

情報科学演習

C++ の class によるハッシュ表の実現

学籍番号: 0000000

氏名: (氏名を記入)

2026 年 7 月 2 日

目次

1	課題	4
2	アルゴリズムの説明	4
2.1	ハッシュ法の原理	4
2.2	ハッシュ関数	4
2.3	衝突とチェイン法	5
2.4	各操作のアルゴリズム	5
2.5	計算量の理論的解析	7
3	プログラムの説明	8
3.1	クラスの構成とデータ構造	8
3.2	コンストラクタとデストラクタ	9
3.3	ハッシュ関数 <code>hash</code>	9
3.4	挿入 <code>insert</code>	10
3.5	検索 <code>operator()</code>	10
3.6	削除 <code>operator-=</code>	11
3.7	補助メンバ関数 <code>size</code> と <code>dump</code>	12
3.8	関数 <code>main</code> (動作確認)	12
3.9	利用方法と各操作の計算量	12
4	実行結果	13
4.1	実行環境	13
4.2	提出プログラム (テンプレート版) の動作確認	13
4.3	非テンプレート版の動作確認	14
4.4	改良版プログラムの動作確認	14
4.5	単体テストによる網羅的な動作確認	14
4.6	実験プログラムの実行	14
5	考察	15
5.1	採用したインタフェース設計について	15
5.1.1	検索を関数呼び出し演算子とする設計	15
5.1.2	削除を複合代入演算子とする設計	16
5.1.3	テンプレート化の効果と限界	16
5.1.4	重複キーの意味論	16
5.2	ハッシュ関数について	17
5.2.1	衝突分布の実験による比較	17
5.2.2	表の大きさとハッシュ関数の相互作用	18
5.2.3	表の大きさは素数でなければならないか	19
5.2.4	アナグラム衝突	19
5.2.5	2020 年以降の知見との対応	20

5.3	負荷率と探索性能について	21
5.4	実行時間について	21
5.4.1	測定方法	21
5.4.2	提出版の性能特性	22
5.4.3	改良版と標準ライブラリの比較	24
5.4.4	キー長の影響	24
5.5	2020 年以降の知見に基づく改良について	25
5.5.1	改良版の設計	25
5.5.2	チェイン法の本質的限界とキャッシュ効率	25
5.5.3	現代的ハッシュ表の設計動向	27
5.5.4	リハッシュの償却コストの実測	27
5.5.5	開放番地法との比較実験	29
5.5.6	タグ付き比較の実装と評価	30
5.6	メモリ使用量について	31
5.7	ハッシュ表の安全性について	32
5.8	C 版との設計比較	33
5.9	総括	33
6	感想	34
A	付録: プログラムソース	36
A.1	hashtable_v1.cpp (非テンプレート版)	36
A.2	hashtable.cpp (テンプレート版・提出プログラム)	37
A.3	hashtable_improved.cpp (改良版)	40
A.4	exp_collision.cpp (実験 1: 衝突分布)	43
A.5	exp_anagram.cpp (実験 2: アナグラム衝突)	45
A.6	exp_loadfactor.cpp (実験 3: 負荷率と比較回数)	46
A.7	exp_timing.cpp (実験 4: 実行時間)	48
A.8	exp_memory.cpp (実験 5: メモリ使用量)	53
A.9	exp_tablesize.cpp (実験 6: 表サイズの影響)	56
A.10	exp_rehash.cpp (実験 7: リハッシュの償却コスト)	58
A.11	exp_openaddr.cpp (実験 8: 開放番地法との比較)	61
A.12	exp_keylen.cpp (実験 9: キー長の影響)	65
A.13	test_hashtable.cpp (単体テスト)	67
A.14	hashtable_tagged.cpp (発展版: タグ付き比較)	71
A.15	exp_tag.cpp (実験 10: タグ付き比較の効果)	75
A.16	exp_hashdos.cpp (実験 11: ハッシュ飽和攻撃)	81
A.17	exp_hashdos.cpp (実験 11: ハッシュ飽和攻撃の実証)	87
A.18	make_figures.py (図の生成スクリプト)	94

B 付録: 出力結果	98
B.1 hashtable_v1 の出力	98
B.2 hashtable の出力	98
B.3 hashtable_improved の出力	99
B.4 exp_anagram の出力	100
B.5 exp_collision の要約出力	100
B.6 exp_loadfactor の出力 (CSV)	102
B.7 exp_timing の出力 (CSV)	102
B.8 exp_memory の出力 (CSV)	104
B.9 exp_tablesize の出力 (CSV)	104
B.10 exp_rehash の統計出力	104
B.11 test_hashtable の出力 (全文)	105
B.12 exp_openaddr の出力 (CSV)	105
B.13 exp_keylen の出力 (CSV)	106
B.14 hashtable_tagged の出力	106
B.15 exp_tag の出力 (CSV)	107
B.16 exp_hashdos の出力 (CSV)	107
C 付録: 参考にした hash.c	107

1 課題

「C 言語 (中級)」で与えられている `hash.c` を参考に、次の仕様を満たすハッシュ表を C++ の class で実現する。

`()`, `insert`, `--` の 3 つのメンバ関数で、キー (文字列) による検索、アイテムの挿入、削除を行える表を C++ で実現し、振舞いを確認する。ハッシュ表オブジェクトを `table` とすると、検索は `table("hoge")`, 挿入は `table.insert("hoge", x)`, 削除は `table -- "hoge"` のように操作するものとする。

また、加算課題として、いろいろな型の値を収納できるよう (キーは文字列でよい)、クラスをテンプレートにする。

2 アルゴリズムの説明

本節では、課題を解くために採用したハッシュ法 (チェイン法) のアルゴリズムを示し、なぜこのアルゴリズムでキーによる検索・挿入・削除が高速に実現できるのかを理論的に解説する。

2.1 ハッシュ法の原理

n 個のデータの集まりから、キー k をもつデータを探し出す問題を考える。データを配列に無秩序に格納した場合、検索には先頭から順に比較していく線形探索が必要であり、比較回数は最悪 n 回、平均でも $n/2$ 回程度となる。データをキーの順に整列して二分探索を行えば比較回数は $\log_2 n$ 回程度に減るが、それでもデータ数に依存した回数の比較が必要であり、さらに挿入・削除のたびに整列状態を保つ操作が必要になる。

ハッシュ法は、キーそのものから格納場所を直接計算することで、データ数によらないほぼ一定の手間で検索・挿入・削除を行う手法である。大きさ m の配列 (以下、この配列の各要素をバケットと呼び、配列全体をハッシュ表と呼ぶ) を用意し、キー k から 0 以上 $m-1$ 以下の整数を計算する関数

$$h(k) \in \{0, 1, \dots, m-1\} \quad (1)$$

を定める。この h をハッシュ関数、 $h(k)$ をキー k のハッシュ値と呼ぶ。データはハッシュ表の $h(k)$ 番目のバケットに格納する。検索時も同じ計算で $h(k)$ を求め、 $h(k)$ 番目のバケットだけを調べればよい。つまり、ハッシュ関数の計算がキー長のみ依存する一定時間で済むならば、探索範囲がデータ全体から 1 バケットへと一挙に絞り込まれる。これがハッシュ法の原理である。

2.2 ハッシュ関数

式 (1) のハッシュ関数 h に要求される性質は次の 2 点である。第一に、同じキーに対しては必ず同じ値を返すこと (決定性)。第二に、異なるキーのハッシュ値ができるだけ 0 から $m-1$ に一様に散らばること (一様性) である。

本課題で参考にする `hash.c` は、キー (文字列) を構成する各文字の文字コードの総和をハッシュ表の大きさ m で割った余りをハッシュ値とする。すなわち、キー k が文字 $c_1, c_2, \dots, c_L$ からなる長さ

L の文字列であるとき、

$$h(k) = \left(\sum_{i=1}^L \text{code}(c_i) \right) \bmod m \quad (2)$$

とする。ここで $\text{code}(c_i)$ は文字 c_i の文字コード（ASCII コード）を表す整数、 m はハッシュ表の大きさ（バケット数）であり、本課題では $m = 17$ とする。 $\bmod$ は剰余を表す。式 (2) は文字コードの加算と剰余のみで計算でき、決定性を満たす。一様性については、剰余をとる際に m を素数にしておくこと、キーの偏り（例えば文字コードがある公約数の倍数に偏る場合）の影響を受けにくいことが知られている。 $m = 17$ が素数であるのはこのためである。なお、式 (2) は文字の並び順を区別しないため、同じ文字の組合せからなるキー同士は必ず同じハッシュ値をもつ。この性質がもたらす問題は考察で詳しく扱う。

2.3 衝突とチェイン法

キーの種類数はバケット数 m よりはるかに多いので、鳩の巣原理より、異なるキーが同じハッシュ値をもつことは原理的に避けられない。これを衝突と呼ぶ。ハッシュ法のアルゴリズムは、衝突をどう処理するかで大きく 2 系統に分かれる。同じバケットに入るデータを連結リストでつないで保持するチェイン法（連鎖法）と、衝突したら表内の別の空きバケットを探して格納する開放番地法である。

本課題では `hash.c` と同じチェイン法を採用する。チェイン法では、ハッシュ表の各バケットは連結リストの先頭を指すポインタであり、キー k をもつデータはリストの節（キー、値、次の節へのポインタの 3 つ組）として $h(k)$ 番目のリストにつながる。チェイン法を採用する理由は次の 3 点である。第一に、衝突がいくら起きても単にリストが伸びるだけであり、表が満杯になって挿入できなくなることがない。第二に、削除がリストからの節の除去として素直に実現でき、開放番地法のように削除跡（墓標）の管理を必要としない。第三に、各操作が「ハッシュ値の計算」と「短いリストの走査」に分離されるため、正しさの確認が容易である。

2.4 各操作のアルゴリズム

チェイン法による挿入・検索・削除のアルゴリズムを以下に示す。いずれの操作も、まず式 (2) でハッシュ値 $v = h(k)$ を計算し、 v 番目のリストのみを対象とする点が共通である。

挿入 キー k と値 x の組を格納する新しい節を作り、 v 番目のリストの先頭につなぐ。すなわち、新しい節の「次」ポインタに現在のリスト先頭を代入し、リスト先頭を新しい節に付け替える。リストの末尾まで辿る必要がないため、挿入そのものはリストの長さによらず一定の手数で完了する。挿入の流れを図 1 に示す。

検索 v 番目のリストを先頭から順に辿り、節のキーと探索キー k を比較する。一致する節が見つければその節（の値）を返し、リストの末尾（空ポインタ）に達したら「存在しない」を返す。検索の流れを図 2 に示す。

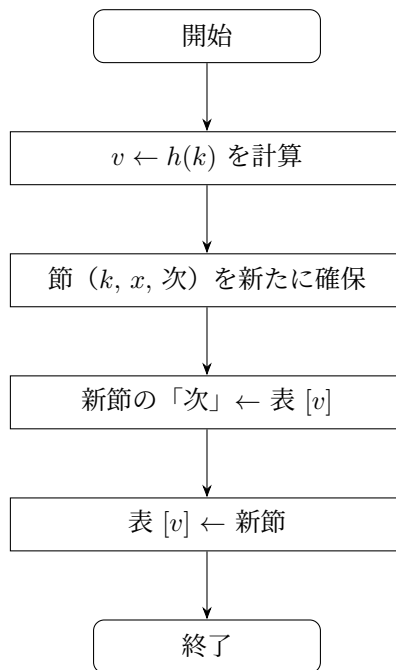


図 1: チェイン法による挿入のフローチャート

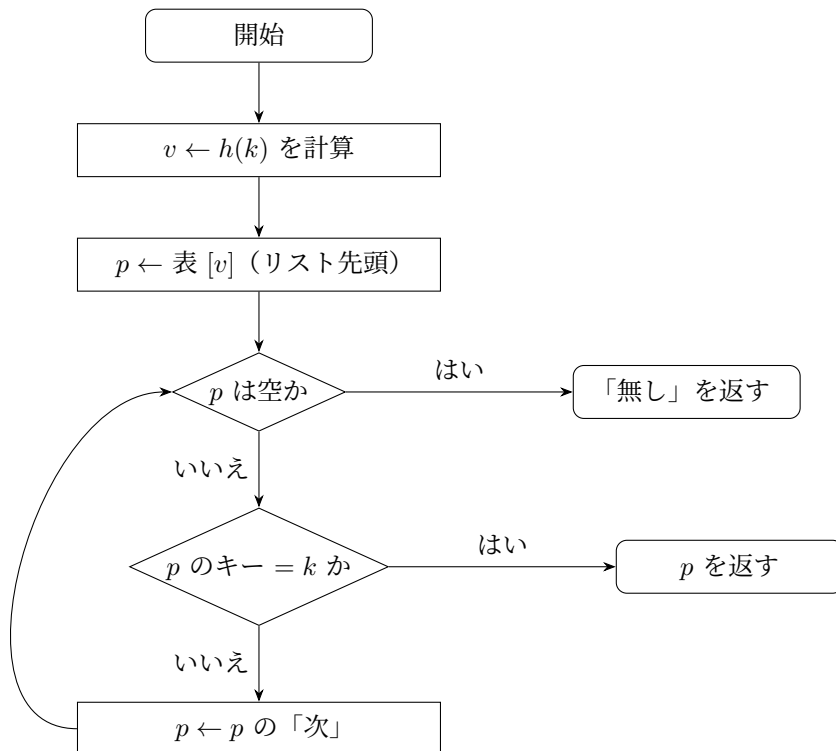


図 2: チェイン法による検索のフローチャート

削除 v 番目のリストを検索と同様に辿るが、付け替えのために「直前の節」を覚えておく必要がある点が検索と異なる。キーが一致する節 p が見つかったとき、 p がリスト先頭ならばリスト先頭を p の「次」に付け替え、そうでなければ直前の節の「次」を p の「次」に付け替える。その後 p の記憶領域を解放する。一致する節がないまま末尾に達した場合は何もしない。削除の流れを図 3 に示す。

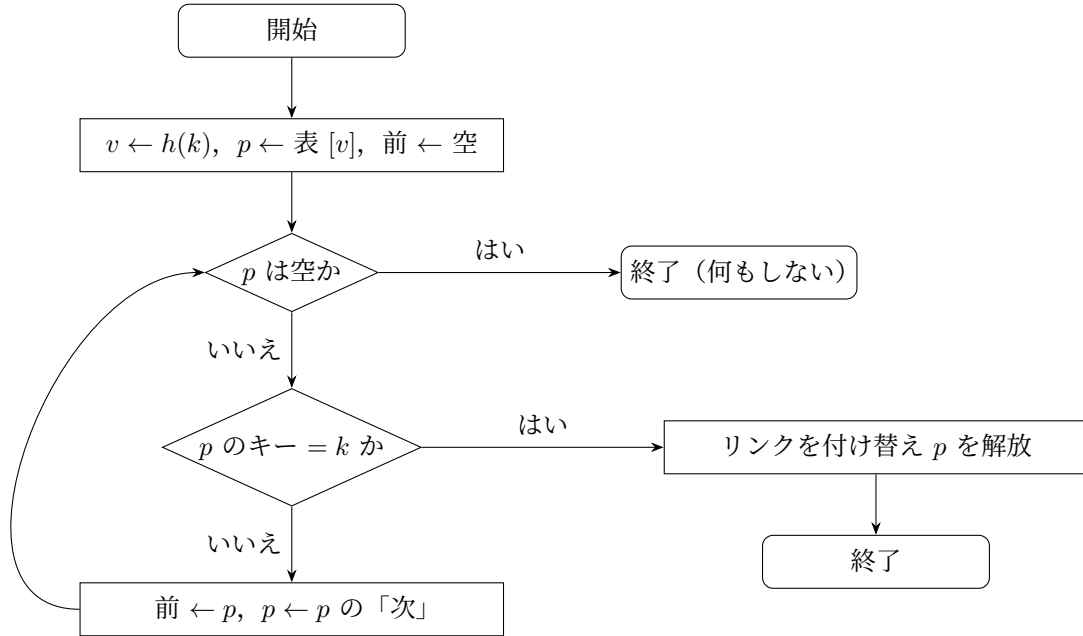


図 3: チェイン法による削除のフローチャート

2.5 計算量の理論的解析

チェイン法の性能は、各バケットのリストの長さで決まる。表に格納されているデータ数を n 、バケット数を m とし、その比

$$\alpha = \frac{n}{m} \quad (3)$$

を負荷率と呼ぶ。 α は 1 バケットあたりの平均データ数に等しい。ここで n は格納済みデータ数、 m はバケット数である。

ハッシュ関数が各キーを m 個のバケットへ等確率かつ独立に割り当てる（単統一様ハッシュ）と仮定すると、各操作の平均比較回数は次のように見積もられる。表に存在しないキーの検索（不成功検索）では、 $h(k)$ 番目のリストを末尾まで辿るので、比較回数の期待値はリスト長の期待値、すなわち α に等しい。一方、表に存在するキーの検索（成功検索）では平均してリストの半ばで一致が見つかるので、期待比較回数はおおよそ

$$1 + \frac{\alpha}{2} \quad (4)$$

となる。挿入はリスト先頭への追加のみなので比較を要せず、ハッシュ値計算を含め一定時間である。削除は検索と同じ走査に付け替えを加えたものなので、期待比較回数は検索と同程度である。

重要なのは、これらの量がデータ数 n 単独ではなく負荷率 $\alpha = n/m$ で決まる点である。 m を n に比例させて確保できるならば α は定数に保たれ、検索・挿入・削除はいずれも平均 $O(1)$ 時間で実行できる。本課題の実装は $m = 17$ 固定なので、 n が大きくなると α が増大し、平均比較回数は式 (4) に

従って n に比例して増える。すなわち m 固定のチェイン法は漸近的には線形探索と同じ $O(n)$ であるが、係数が $1/m$ 倍に抑えられた線形探索とみなせる。また最悪の場合（すべてのキーが同じバケットに衝突する場合）は、 m によらず比較回数が n に達する。この最悪ケースが現実に関起こり得ることも考察で実験的に確認する。

3 プログラムの説明

本節では、前節のアルゴリズムを C++ の class としてどのように実現したかを、最終的に採用したプログラム hashtable.cpp (付録 A.2) に沿って説明する。本プログラムは加算課題であるテンプレート化まで行ったものであり、キーは `std::string` 型、値は任意の型 T を収納できる。なお、比較のために作成した非テンプレート版 hashtable_v1.cpp, 改良版 hashtable_improved.cpp, 発展版 hashtable_tagged.cpp については考察で述べる。

3.1 クラスの構成とデータ構造

ハッシュ表はクラス HashTable として実現した。クラスの骨格を以下に示す（行番号は付録 A.2 のソースに対応する）。

```
5  template <class T>
6  class HashTable {
7  public:
8      static constexpr int HASHSIZE = 17;
9
10     // チェインのノード
11     struct Node {
12         std::string key;
13         T value;
14         Node* next;
15     };
```

5 行目の `template <class T>` により、クラス全体が値の型 T を型引数とするテンプレートになっている。利用側が `HashTable<int>` や `HashTable<std::string>` のように型を指定すると、コンパイラがその型専用のクラスを生成する。8 行目の `HASHSIZE` はバケット数 m に相当する定数であり、hash.c と同じ 17 とした。constexpr 指定によりコンパイル時定数であることを保証している。

11~15 行目の構造体 Node は連結リストの節であり、キー key、値 value、次の節へのポインタ next の 3 つのメンバをもつ。hash.c の struct member_list では名前が `char name[16]` の固定長配列、値が `int score` 固定であったのに対し、本実装ではキーを `std::string`、値をテンプレート引数 T とすることで、長さ 16 バイト以上のキーも任意型の値も扱えるようにした。

表本体は private メンバとして次のように宣言している。

```
117 private:
```

```

118     Node* buckets_[HASHSIZE];
119     int size_;

```

118 行目の `buckets_` が大きさ 17 のバケット配列（各要素が連結リストの先頭を指すポインタ）であり、`hash.c` のグローバル変数 `hashtable` に相当する。C 版ではハッシュ表がグローバル変数であったため 1 個しか作れなかったが、本実装ではメンバ変数としたことで、ハッシュ表オブジェクトを複数個同時に生成できる。119 行目の `size_` は現在の格納要素数であり、挿入・削除のたびに増減させる。

3.2 コンストラクタとデストラクタ

```

17     // 全バケットを nullptr で初期化
18     HashTable() : size_(0) {
19         for (int i = 0; i < HASHSIZE; ++i) {
20             buckets_[i] = nullptr;
21         }
22     }

```

コンストラクタ（18～22 行目）は全バケットを空ポインタ `nullptr` で初期化する。`hash.c` では `main` の先頭で利用者が明示的に初期化ループを書く必要があったが、C++ ではオブジェクト生成時にコンストラクタが自動的に呼ばれるため、初期化忘れが起こらない。

デストラクタ（25～34 行目）は全バケットについて連結リストを先頭から辿り、各節を `delete` で解放する。次の節へのポインタを退避してから `delete` する点が肝要である。これによりオブジェクトの寿命が尽きるときに全メモリが自動的に解放される。`hash.c` には解放処理が存在せず、確保したメモリはプログラム終了まで解放されなかった。

また 37～38 行目でコピーコンストラクタとコピー代入演算子を `= delete` 指定により禁止している。本クラスは生のポインタでノードを管理しているため、既定のコピー（ポインタの浅いコピー）を許すと 2 つのオブジェクトが同じ節を指し、二重解放を起こす。コピーを禁止することでこの誤用をコンパイル時に排除した。

3.3 ハッシュ関数 hash

```

40     // ハッシュ値を計算（文字コードの総和 % HASHSIZE）
41     static int hash(const std::string& key) {
42         int hashval = 0;
43         for (char c : key) {
44             hashval += static_cast<unsigned char>(c);
45         }
46         return hashval % HASHSIZE;
47     }

```

関数 `hash` は、キー文字列を受け取り 0 以上 16 以下のバケット番号を返す。式 (2) をそのまま実装したものであり、範囲 `for` 文（43 行目）で各文字の文字コードを `hashval` に加算し、最後に `HASHSIZE`

で割った余りを返す。44 行目で `unsigned char` にキャストしているのは、`char` が符号付きの処理系で日本語などの多バイト文字を含むキーを渡された場合に加算値が負になり、剰余が負値となってバケット番号が範囲外になる不具合を防ぐためである。この点は `char` をそのまま加算する `hash.c` からの改良である。

3.4 挿入 insert

```
49     // 先頭挿入で key-value を登録
50     bool insert(const std::string& key, const T& value) {
51         int h = hash(key);
52         Node* p = new Node{key, value, buckets_[h]};
53         buckets_[h] = p;
54         ++size_;
55         return true;
56     }
```

メンバ関数 `insert` は、キーと値を受け取り図 1 の手順で表に登録する。51 行目でハッシュ値 `h` を計算し、52 行目で新しい節を確保する。このとき集成体初期化 `{key, value, buckets_[h]}` により、「次」ポインタへ現在のリスト先頭を設定するところまでを 1 文で行っている。53 行目でリスト先頭を新しい節に付け替えれば先頭挿入が完了する。課題の仕様どおり `table.insert("hoge", x)` の形で呼び出せる。値渡しではなく `const T&` (`const` 参照) で受け取ることで、値の型 `T` が大きい場合の無駄なコピーを避けている。

3.5 検索 operator()

```
58     // key に対応する値へのポインタを返す（見つからなければ nullptr）
59     const T* operator()(const std::string& key) const {
60         int h = hash(key);
61         Node* p = buckets_[h];
62         while (p != nullptr) {
63             if (p->key == key) {
64                 return &(p->value);
65             }
66             p = p->next;
67         }
68         return nullptr;
69     }
```

検索は関数呼び出し演算子 `operator()` の多重定義として実現した。これにより課題の仕様どおり `table("hoge")` という関数呼び出しの形でオブジェクトを「適用」でき、表をキーから値への写像として扱う直感的な記法になる。処理は図 2 のとおりで、60 行目でハッシュ値を求め、62～67 行目の

while 文で該当バケットのリストを辿る。63 行目のキー比較には `std::string` の `==` 演算子を用いた。`hash.c` の `strcmp` と異なり、`==` は長さ比較を先に行えるため、長さの異なる文字列同士の比較が高速であり、記述も簡潔になる。

戻り値は値そのものではなく値への `const` ポインタ `const T*` とし、キーが存在しないときは `nullptr` を返す。値そのものを返す設計にすると「存在しない」ことを表す特別な値（番兵）が型 `T` ごとに必要になるが、ポインタを返す設計ならばどんな型 `T` に対しても `nullptr` が一様に「無し」を表せる。これは `hash.c` の `find` が `NULL` または節へのポインタを返す設計を踏襲したものである。末尾の `const` 指定により、検索が表の内容を変更しないことをコンパイラに保証させている。

3.6 削除 `operator-=`

```
71      // key を削除する（存在しなければ何もしない）
72      HashTable& operator-=(const std::string& key) {
73          int h = hash(key);
74          Node* p = buckets_[h];
75          Node* prev = nullptr;
76          while (p != nullptr) {
77              if (p->key == key) {
78                  if (prev == nullptr) {
79                      buckets_[h] = p->next;
80                  } else {
81                      prev->next = p->next;
82                  }
83                  delete p;
84                  --size_;
85                  return *this;
86              }
87              prev = p;
88              p = p->next;
89          }
90          return *this;
91      }
```

削除は複合代入演算子 `operator-=` の多重定義として実現し、課題の仕様どおり `table -= "hoge"` と書ける。処理は図 3 のとおりである。検索と同様にリストを辿るが、付け替えのために直前の節を指すポインタ `prev` (75 行目) を併走させる。一致する節が見つかったとき、それがリスト先頭ならばバケットを次の節に付け替え (78~79 行目)、途中ならば直前の節の `next` を付け替える (81 行目)。その後 83 行目で節を解放する。キーが存在しない場合は何もせずに抜ける。戻り値を自オブジェクトへの参照 `HashTable&` とすることで、組込み型の `-=` と同じく (`table -= "a"`) `-= "b"` のような連鎖も可能な、演算子として自然な意味論になっている。

3.7 補助メンバ関数 size と dump

size (94~96 行目) は現在の格納要素数 size_ を返す. 挿入・削除が正しく行われたことを要素数の増減で確認するために設けた. dump (99~115 行目) は全バケットについて連結リストの中身を (キー → 値) の形式で表示するデバッグ用関数である. どのキーがどのバケットに入り, 衝突がどのバケットで起きているかを直接観察できる.

3.8 関数 main (動作確認)

関数 main (122~180 行目) は 4 部構成で動作確認を行う. (a) では HashTable<int> に hash.c と同じ 4 名の名前と点数を挿入し, dump による格納状況の表示, 検索の成功例・失敗例, -= による削除と削除後の再検索を行う. (b) では HashTable<double> に実数値 (身長) を格納し, テンプレートが int 以外の組込み型でも機能することを確認する. (c) では HashTable<std::string> に日本語文字列を値として格納し, クラス型の値も収納できることを確認する. (d) では存在しないキーの検索と削除後の検索がいずれも nullptr を返すことを, 境界条件として改めて確認する. 各操作の結果は条件演算子を用いて, ポインタが nullptr のとき none, それ以外のとき値を表示する形式で出力する.

3.9 利用方法と各操作の計算量

本クラスの利用者から見た使い方を以下にまとめる. 値の型を指定してオブジェクトを生成し, 3 つの操作を呼び出すだけである.

```
HashTable<int> table;           // 値が int の表を生成
table.insert("hoge", 42);      // 挿入
const int* p = table("hoge");  // 検索
if (p != nullptr) { /* *p が値 */ }
table -= "hoge";               // 削除
```

各操作の計算量を表 1 に示す. n は格納要素数, m はバケット数 (本実装では 17 固定), $\alpha = n/m$ は負荷率である. 挿入は先頭挿入のため常に一定時間であり, 検索・削除は 2.5 節の解析のとおり平均 $O(1 + \alpha)$, 最悪 (全キーが 1 バケットに衝突した場合) $O(n)$ である. m が固定なので, n に対する漸近的な平均計算量はいずれも $O(n/m) = O(n)$ となる点に注意する.

表 1: 各メンバ関数の計算量 (n : 要素数, $\alpha = n/m$: 負荷率)

操作	メンバ関数	平均	最悪
挿入	insert	$O(1)$	$O(1)$
検索	operator()	$O(1 + \alpha)$	$O(n)$
削除	operator-=	$O(1 + \alpha)$	$O(n)$
生成	コンストラクタ	$O(m)$	$O(m)$
破棄	デストラクタ	$O(n + m)$	$O(n + m)$

4 実行結果

本節では、付録 B に添付した実行結果に基づき、作成したプログラムが課題の仕様を満たして動作することを報告する。

4.1 実行環境

実行環境を表 2 に示す。いずれのプログラムも警告なしでコンパイルできた。動作確認プログラムは最適化なし、時間計測を含む実験プログラムは `-O2` を付してコンパイルした。

表 2: 実行環境

項目	内容
CPU	Apple M4 Pro (12 コア)
主記憶	24 GB
OS	macOS 26.5
コンパイラ	Apple clang 21.0.0
コンパイルオプション	<code>-std=c++17 -Wall -Wextra</code> (実験は <code>-O2</code> 付き)
標準ライブラリ	libc++

4.2 提出プログラム (テンプレート版) の動作確認

テンプレート版 `hashtable.cpp` の実行結果の全文を付録 B.2 に示す。その要点は次のとおりである。

まず `HashTable<int>` に対して `hash.c` と同じ 4 組のデータを `insert` で挿入したところ、要素数は 4 となり、`dump` の出力から `matsuzawa` がバケット 3, `takimoto` がバケット 5, `ohmura` がバケット 6, `katsurada` がバケット 8 に格納されたことが確認できた。これは式 (2) のハッシュ関数による配置と一致する。

検索については、次の出力が得られた (付録 B.2 より抜粋)。

```
find takimoto: 42
find katsurada: 122
find nonexistent: none
```

登録済みキーの検索 `table("takimoto")` は挿入した値 42 を返し、未登録キーの検索は `nullptr` (表示は `none`) を返した。削除については、`table -= "katsurada"` の実行後に要素数が 4 から 3 に減り、同じキーの再検索が `none` となった。

```
katsurada 削除後の size: 3
find katsurada after erase: none
```

以上より、課題仕様である `table("hoge")` (検索), `table.insert("hoge", x)` (挿入), `table -= "hoge"` (削除) の 3 操作がいずれも正しく動作している。

さらに加算課題のテンプレート化について、`HashTable<double>` では実数値 175.0 の格納と検索、`HashTable<std::string>` では日本語文字列「こんにちは」「情報科学演習」の格納と検索が正しく行えた。また境界条件として、存在しないキーの検索、削除後の再検索、空でない表での未登録キー検索がすべて `none` を返すことを確認した。すなわち、値の型を `int`, `double`, `std::string` と変えても同一のクラス定義がそのまま機能しており、テンプレート化の要件を満たしている。

4.3 非テンプレート版の動作確認

比較用の非テンプレート版 `hashtable_v1.cpp` についても同様の確認を行い、4 件の挿入、成功検索 4 件、不成功検索 1 件、削除 1 件と削除後の再検索がすべて期待どおりの結果となった（付録 B.1）。

4.4 改良版プログラムの動作確認

考察用の改良版 `hashtable_improved.cpp` (FNV-1a ハッシュ+動的リハッシュ) では、キー `key0000` から `key0999` までの 1000 件を挿入する過程で、バケット数が $17 \rightarrow 37 \rightarrow 79 \rightarrow 163 \rightarrow 331 \rightarrow 673 \rightarrow 1361$ と自動拡張された（付録 B.3）。挿入完了後の要素数は 1000、負荷率は 0.735 であり、負荷率が 1.0 を超えないよう表が成長していることが確認できた。拡張（全要素の再配置）を 6 回経た後も、`key0000`, `key0123`, `key0999` の検索は正しい値を返し、未挿入の `key1000` は `none` を返した。また `key0123` の削除後は再検索が `none` となり要素数が 999 に減った。すなわちリハッシュを挟んでも 3 操作の正しさは保たれている。

4.5 単体テストによる網羅的な動作確認

デモによる確認に加えて、境界条件を網羅する単体テストプログラム `test_hashtable.cpp`（付録 A.13）を作成し実行した。テスト項目は、空の表に対する検索・削除、同一バケットに衝突するキー群（加算ハッシュで同値になるアナグラム）の格納、チェーンの先頭・中間・末尾それぞれの削除、存在しないキーの削除、削除後の再挿入、同一キーの重複挿入時の挙動、要素数の増減、1000 件規模の挿入・削除、および `HashTable<std::string>` での同等の確認である。実行の結果、50 項目すべてが成功した（付録 B.11 に全出力を添付）。

passed 50 / 50

特にチェーンの中間・末尾の削除は、デモ（4.2 節）では通らない経路（削除処理の 81 行目側の分岐）を通るため、単体テストによって初めて網羅的に確認できた項目である。さらに、メモリ誤り検出器（`AddressSanitizer`）と未定義動作検出器（`UndefinedBehaviorSanitizer`）を有効にしたビルドでも、3 本のプログラムと単体テストのすべてが検出ゼロで完走することを確認した。すなわち全ノードの解放漏れ・二重解放・範囲外アクセス・符号あふれは存在しない。

4.6 実験プログラムの実行

考察のために 11 本の実験プログラムを作成し実行した。(1) `exp_collision.cpp` は英単語 5000 語をキーとして 4 種類のハッシュ関数の衝突分布を計測するもの、(2) `exp_anagram.cpp` はアナグラム

(同じ文字の並べ替え) キー群のハッシュ値を印字するもの, (3) `exp_loadfactor.cpp` は負荷率を変えながら検索時の平均キー比較回数を計測するもの, (4) `exp_timing.cpp` は要素数を 10^3 から 10^6 まで変えて挿入・検索時間を 4 種類のデータ構造で計測するもの, (5) `exp_memory.cpp` は挿入前後の最大常駐メモリ量の増分を計測するもの, (6) `exp_tablesize.cpp` はバケット数 (2 の冪と素数) を変えて分布統計を計測するもの, (7) `exp_rehash.cpp` は挿入 1 件ごとのレイテンシを記録してリハッシュの償却コストを観測するもの, (8) `exp_openaddr.cpp` は開放番地法 (線形走査) とチェーン法の平均プローブ数を負荷率別に計測するもの, (9) `exp_keylen.cpp` はキー長を変えてハッシュ計算・検索時間を計測するもの, (10) `exp_tag.cpp` はタグ付き比較 (5.5.6 節で述べる発展版) による文字列比較回数の削減を計測するもの, (11) `exp_hashdos.cpp` は同一バケットに衝突する攻撃キー群による検索コストの劣化 (ハッシュ飽和攻撃) を計測するものである. キーには macOS 付属の英単語辞書 `/usr/share/dict/words` から機械的に抽出した 5000 語, および固定シードの擬似乱数で生成した英数字文字列を用いた. 時間計測を含む実験では, 他プロセスの影響を抑えるため反復測定 (中央値または最小値) を用いた. いずれのプログラムも正常終了し, 出力 (付録 B.4~B.15) が得られた. 各実験の目的と結果の掲載先を表 3 にまとめる. これらの結果の分析は次節の考察で行う.

表 3: 実験プログラムの一覧と結果の掲載先

プログラム	計測対象	使用キー	結果の掲載先
<code>exp_collision</code>	衝突分布・ χ^2 値	英単語 5000 語	図 4, 5, 表 4, 5
<code>exp_anagram</code>	アナグラムのバケット番号	固定アナグラム 16 語	表 7
<code>exp_loadfactor</code>	平均キー比較回数	英単語 5000 語	図 6
<code>exp_timing</code>	挿入・検索の実時間	乱数 12 文字	図 7, 8, 表 8
<code>exp_memory</code>	最大常駐メモリ増分	乱数 12 文字	図 13
<code>exp_tablesize</code>	バケット数と分布統計	英単語 5000 語	表 6
<code>exp_rehash</code>	挿入レイテンシ時系列	乱数 12 文字	図 11
<code>exp_openaddr</code>	平均プローブ数	乱数 12 文字	図 12
<code>exp_keylen</code>	ハッシュ計算・検索時間	乱数 8~256 文字	図 9
<code>exp_tag</code>	文字列比較回数	乱数 12~256 文字	表 10
<code>exp_hashdos</code>	攻撃キーでの検索コスト	アナグラム 2000 語ほか	表 11

5 考察

5.1 採用したインタフェース設計について

5.1.1 検索を関数呼び出し演算子とする設計

検索を `operator()` の多重定義としたことには, 記法上の利点と設計上の利点がある. 記法上は, ハッシュ表が数学的にはキーから値への部分写像であることを踏まえると, `table("hoge")` という「写像の適用」の形で検索を書けるのは自然である. 設計上は, 戻り値を `const T*` とし「無し」を `nullptr` で表現したことで, 任意の型 `T` に対して番兵値を用意することなく成否と値を同時に返せる.

一方、C++ の標準ライブラリの連想コンテナ `std::map` は検索・挿入を兼ねる `operator[]` を提供するが、この演算子は未登録キーへのアクセスで既定値の要素を自動的に挿入するという副作用をもつ。本実装の `operator()` は `const` メンバ関数であり、検索によって表の状態が変化することはない。検索操作が表を書き換えないという性質が型システムで保証される点で、本設計は `operator[]` 方式よりも誤用しにくいと考えられる。ただし、ポインタを返す設計には、返されたポインタの寿命が対象要素の寿命に従属するという注意点がある。すなわち当該キーが削除された後にポインタを参照すると未定義動作となる。この危険は `hash.c` の `find` が返すポインタにも同様に存在するものであり、ポインタを長期間保持しないという利用規約で回避する設計とした。

5.1.2 削除を複合代入演算子とする設計

削除を `operator-=` としたことで、「表からキーを引き去る」という直感に沿った記法 `table -= "hoge"` が得られた。演算子の意味論として、組込み型の `-=` と同様に自オブジェクトへの参照を返すため、`(table -= "a") -= "b"` のような連鎖適用も可能である。一般の設計では `erase` のような名前付きメンバ関数のほうが意図が明瞭であるという立場もあるが、演算子多重定義は「本来の演算子の意味から類推できる場合に限って用いる」という指針に照らすと、集合からの要素の除去を減算と見なす本設計はその範囲に収まっていると考えられる。なお、削除が実際に行われたかどうかを呼び出し側が知る手段として、本実装では削除前後の `size()` の比較が利用できる。

5.1.3 テンプレート化の効果と限界

テンプレート化により、値の型ごとにクラスを書き直すことなく `HashTable<int>`、`HashTable<double>`、`HashTable<std::string>` が同一のソースから得られた (4.2 節)。C 言語で同等の汎用性を得るには、`void*` と明示的キャストを使うか、マクロでコードを複製するか of the どちらかになるが、前者は型安全性を失い (誤った型へのキャストがコンパイル時に検出されない)、後者はデバッグが困難になる。テンプレートはコンパイラが型ごとに専用コードを生成するため、型安全性と実行効率を両立できる。

一方で限界も観察された。第一に、テンプレートの実体化は型ごとにコードを複製するため、多数の型で使うと実行ファイルが大きくなる。第二に、`dump` が `operator<<` で値を出力できる型を暗黙に仮定しているように、テンプレート引数への要求 (コンセプト) がソース上に明示されず、満たさない型を渡したときのコンパイルエラーが読みにくい。C++20 で導入された concepts を用いて要求を明示することが、この限界に対する現代的な解決策である。

5.1.4 重複キーの意味論

本実装の `insert` は既存キーの有無を調べずに先頭挿入するため、同じキーを 2 回挿入すると両方の節がチェーン上に残る。このときの観測可能な挙動を単体テスト (4.5 節) で固定した。検索は先頭から辿るため常に「後から挿入した値」を返し、`-=` を 1 回適用すると新しい節だけが除去されて古い値が再び見えるようになる。すなわち同一キーに対する挿入と削除が、キーごとの後入れ先出し (重ね掛けと剥がし) として振る舞う。

この意味論は標準ライブラリとは異なる。`std::unordered_map` の `insert` は既存キーがあれば挿入せず、`operator[]` は上書きする。本実装を上書き型に変えるには `insert` の先頭で検索を行えばよい

が、その場合、挿入 1 回の手数が「常に一定」から「平均 $1 + \alpha$ 程度の比較を伴う」へ変わる。hash.c の add も本実装と同じく検査なしの先頭挿入であり、本課題では原典の意味論を保存した上で、その帰結を仕様としてテストで文書化するという立場を採った。重複が起こり得る応用で使うならば、上書き型への変更が第一の改造点になる。

5.2 ハッシュ関数について

5.2.1 衝突分布の実験による比較

hash.c 由来の加算ハッシュ（式 (2)）の品質を定量評価するため、英単語辞書から機械的に抽出した 5000 語をキーとし、4 種類のハッシュ関数（加算ハッシュ、djb2、FNV-1a、std::hash）でバケットへの割り当てを比較した。djb2 と FNV-1a は、文字を 1 つ読み込むごとに乗算と加算（または排他的論理和）を繰り返す方式の代表的なハッシュ関数である。

バケット数 $m = 17$ での分布を図 4 に、要約統計を表 4 に示す。一様分布であればバケットあたりの期待値は $5000/17 = 294.1$ 個である。

表 4: ハッシュ関数ごとの衝突分布の要約 ($m = 17$, $n = 5000$)

ハッシュ関数	空バケット数	最長チェーン	分散	χ^2 値
加算ハッシュ	0	329	311.16	17.99
djb2	0	325	295.04	17.05
FNV-1a	0	315	253.75	14.67
std::hash	0	321	210.10	12.14

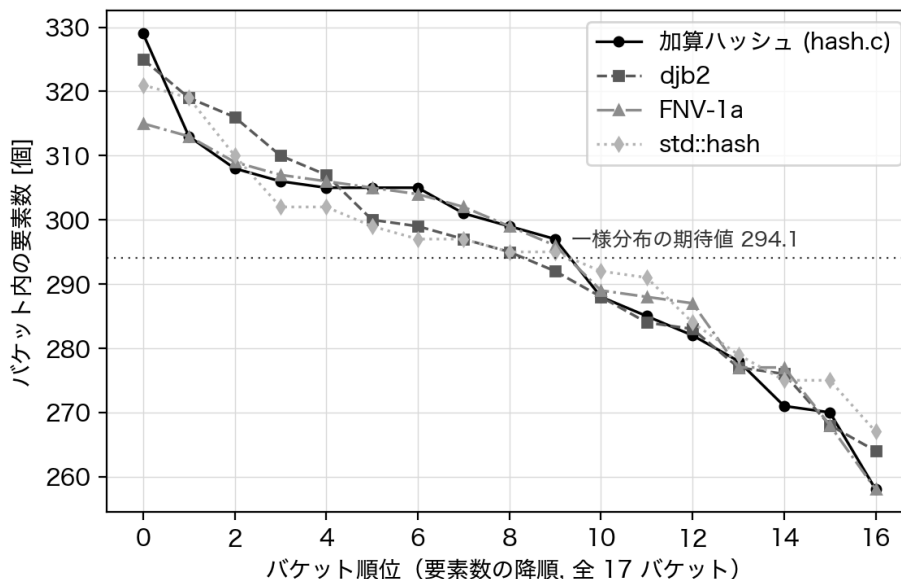


図 4: バケットごとの要素数分布 ($m = 17$, 英単語 5000 語, 要素数の降順)

$m = 17$ では 4 関数の差は小さい。いずれも χ^2 値は自由度 16 のカイ二乗分布の 5% 点 26.3 を下回っており、統計的には一様分布と矛盾しない分布が得られている。なお std::hash の具体的なアル

ゴリズムは処理系定義であり、本実験の `stdhash` の数値は使用した処理系（Apple libc++）に固有である。すなわち、英単語のような自然なキー集合に対しては、 m が小さい間は加算ハッシュでも実用上十分な一様性をもつ。これは、単語の文字コード総和が数百から二千程度の広い範囲に分布し、それを小さな素数 17 で割った余りが十分に混ざるためと考えられる。

5.2.2 表の大きさとハッシュ関数の相互作用

ところが $m = 1031$ （素数）に拡大すると様相が一変する（図 5，表 5）。

表 5: ハッシュ関数ごとの衝突分布の要約 ($m = 1031$, $n = 5000$)

ハッシュ関数	空バケット数	最長チェーン	分散	χ^2 値
加算ハッシュ	57	20	14.54	3091.29
djb2	7	13	4.67	991.76
FNV-1a	6	12	4.78	1016.09
<code>std::hash</code>	8	15	4.93	1048.26

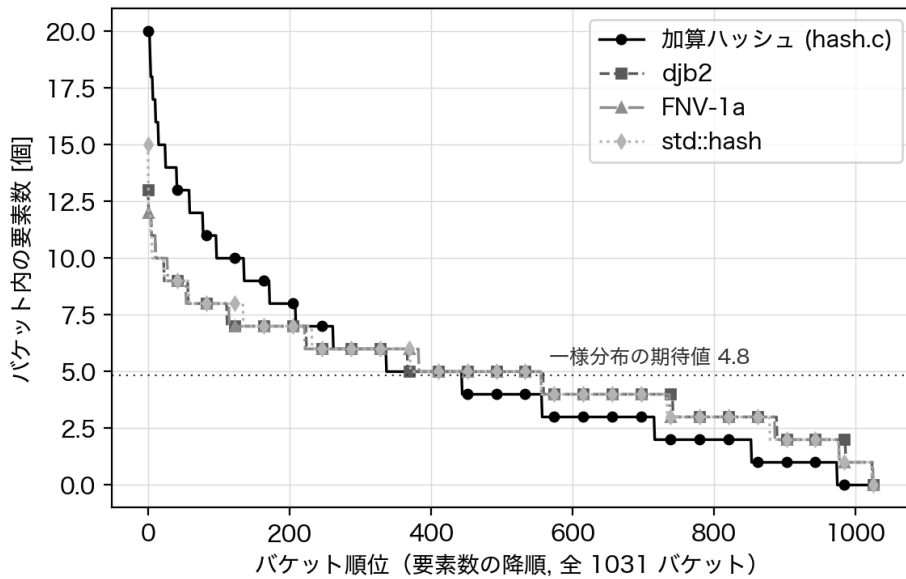


図 5: バケットごとの要素数分布 ($m = 1031$, 英単語 5000 語, 要素数の降順)

自由度 1030 のカイ二乗分布の 5% 点はおおよそ 1106 であり、`djb2`・`FNV-1a`・`std::hash` の χ^2 値 (992~1048) は一様分布と矛盾しないのに対し、加算ハッシュは 3091.29 と決定的に逸脱している。最長チェーンも他の 1.5 倍以上、空バケットは 7 倍以上である。原因は、加算ハッシュの値域の狭さにある。英単語の文字コード総和は高々 2000 程度にしか達しないため、 m がこの値域と同程度以上になると、ハッシュ値が表の一部にしか届かず、剰余を取っても偏りが解消されない。すなわち加算ハッシュは「 m が小さいときに限り成立する」ハッシュ関数であり、表を大きくして性能を上げようとするとかえって偏りが顕在化する。この性質は、5.5 節で述べる動的リハッシュ（表の自動拡大）と致命的に相性が悪く、改良版でハッシュ関数の置き換えが必須であった理由である。

5.2.3 表の大きさは素数でなければならないか

2.2 節では「 m を素数にしておくとの偏りの影響を受けにくい」と述べた。hash.c の $m = 17$ も `std::unordered_map` の実装も素数を用いる。この通説がハッシュ関数の品質とどう関係するかを確かめるため、同じ英単語 5000 語に対して、2 の冪とその近傍の素数の対 ($m = 16/17, 256/257, 1024/1031$) で分布の統計量を比較した。結果を表 6 に示す。

表 6: バケット数 m (2 の冪と近傍の素数) による分布の比較 ($n = 5000$)

ハッシュ関数	m	空バケット数	最長チェーン	分散	χ^2 値
加算ハッシュ	16	0	353	284.62	14.57
	17	0	329	311.16	17.99
	256	0	32	20.44	267.87
	257	0	33	20.84	275.28
	1024	54	21	14.77	3097.79
	1031	57	20	14.54	3091.29
FNV-1a	16	0	339	244.00	12.49
	17	0	315	253.75	14.67
	256	0	31	19.89	260.70
	257	0	31	17.91	236.63
	1024	11	13	4.75	996.54
	1031	6	12	4.78	1016.09

この実験から 2 つの事実が読み取れる。第一に、加算ハッシュの破綻 ($m = 1024$ 以上での χ^2 値の急増) は m が素数かどうかと無関係に起こる。 $m = 1024$ で 3097.79, 素数の $m = 1031$ でも 3091.29 であり、素数を選んでも救われない。5.2.2 節で述べたとおり破綻の原因はハッシュ値の値域の狭さであり、これは剰余の取り方では解消できないからである。第二に、FNV-1a は 2 の冪の m でも近傍の素数と統計的に差のない一様性を示した ($m = 1024$ で 996.54, $m = 1031$ で 1016.09. 自由度約 1000 のカイ二乗分布の 5% 点は約 1100 であり、いずれも一様分布と矛盾しない)。

すなわち「 m は素数にせよ」という通説は、ハッシュ関数が弱い場合の防御策と理解するのが正確である。十分に攪拌するハッシュ関数を使うならば m を 2 の冪に取ってよく、その場合、剰余演算 % をビットマスク (論理積) に置き換えられるため、バケット番号の計算自体も高速化できる。実際、現代の高速ハッシュ表は除算を避ける工夫 (2 の冪サイズや、乗算とシフトで剰余を代替する手法) を標準的に採用している [2]。ただし本実験のキーは英単語 1 種類のデータセットに限られるため、キーの構造によっては (例えば下位ビットに規則性をもつキー群では) 2 の冪の m が不利になり得ることに注意が必要である。

5.2.4 アナグラム衝突

加算ハッシュのもう 1 つの弱点は、文字の並び順を区別しないことである (2.2 節)。これを実験で確認した。表 7 に、アナグラム関係にある 3 群のキーについて加算ハッシュと FNV-1a のバケット番号 ($m = 17$) を示す。

表 7: アナグラムキーに対するバケット番号の比較 ($m = 17$)

キー	加算ハッシュ	FNV-1a
abc	5	1
acb	5	1
bac	5	2
bca	5	1
cab	5	5
cba	5	4
stop	12	13
tops	12	7
pots	12	0
spot	12	3
opts	12	8
post	12	9
listen	9	11
silent	9	10
enlist	9	5
tinsel	9	12

加算ハッシュでは 3 群とも全キーが同一バケットに集中し、最悪ケース（1 本のチェーンへの集中）が容易に発生する．一方 FNV-1a は文字を読むたびに乗算で状態を攪拌するため並び順が結果に反映され、同じ 6 キーが 4～6 個の異なるバケットに分散した．アナグラムは人工的な例に見えるが、「文字の構成が同じで並びだけ違うキー」は識別子やファイル名などで現実に生じ得る．さらに 5.7 節で述べるように、この性質は悪意ある入力に対する脆弱性に直結する．

5.2.5 2020 年以降の知見との対応

ハッシュ関数の品質評価は現在も活発な研究対象である．非暗号ハッシュ関数の標準的な品質検査系 SMHasher は雪崩性（入力 1 ビットの変化が出力の各ビットを確率 $1/2$ で反転させる性質）や分布の一様性を体系的に検査しており [10]、FNV-1a はこの検査で既知の弱点（雪崩性の不足など）が指摘される一方、簡潔さと十分な実用性能から今も広く使われている．より新しい wyhash 系の rapidhash（2024 年公開）は SMHasher 3 の全検査に合格した上で 64 バイト以下の短いキーに対して極めて高速であり [9]、Rust 向けのベンチマーク比較でも短文字列キーに対する wyhash 系の優位が報告されている [11]．また 2024 年には CityHash64・MurmurHash2/3・wyhash などに対して、シードを知らなくても意図的に衝突を起こせる実用的な攻撃が公開されており [8]、「速く一様である」と「敵対的入力に耐える」ことは別の要求であることが明確になっている．本課題の加算ハッシュは前者すら m 依存でしか満たさない最も素朴な段階にあり、FNV-1a への置き換え（改良版）はこの発展系列の第一歩に位置づけられる．

5.3 負荷率と探索性能について

2.5 節の理論解析では、成功検索の平均比較回数は $1 + \alpha/2$ 、不成功検索は α と見積もった (α は式 (3) の負荷率)。この理論値の妥当性を、 $m = 17$ 固定の表に英単語キーを $n = 8$ 件から 1700 件 ($\alpha = 0.47$ から 100) まで挿入して実測した。結果を図 6 に示す。

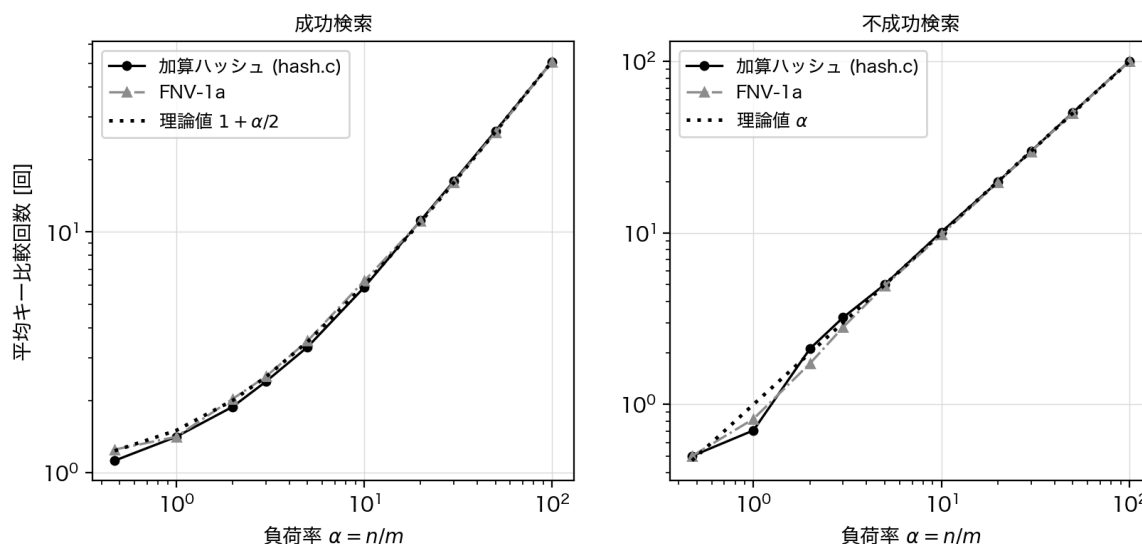


図 6: 負荷率 α と平均キー比較回数の関係 (左: 成功検索, 右: 不成功検索, 両対数)

実測値は理論値と極めてよく一致した。例えば加算ハッシュの成功検索は $\alpha = 10$ で平均 5.88 回 (理論値 6.0), $\alpha = 100$ で 50.87 回 (理論値 51.0) であり、誤差は 2% 以内である。不成功検索も $\alpha = 10$ で 10.13 回, $\alpha = 100$ で 100.27 回と、理論値 α にほぼ重なる。FNV-1a でも同様であり ($\alpha = 100$ で成功 50.83 回・不成功 100.30 回), $m = 17$ においては両ハッシュ関数の差は誤差の範囲に収まる。これは 5.2.1 節で見たとおり $m = 17$ では加算ハッシュも十分一様であることと整合する。

この結果は 2 つのことを意味する。第一に、単純一様ハッシュの仮定に基づく式 (4) が、実在の英単語キーという「作為のない」入力に対して定量的に成立すること。第二に、バケット数固定のハッシュ表の探索コストは α , すなわちデータ数に厳密に比例して劣化していくことである。 $n = 1700$ ($\alpha = 100$) の時点で成功検索に平均 51 回の文字列比較を要しており、これは連結リスト 1 本の線形探索 (平均 850 回) よりも 17 倍速いものの、もはや「ほぼ一定時間」とは言えない。この観察が次節の実行時間測定と 5.5 節の動的リハッシュの動機となる。

5.4 実行時間について

5.4.1 測定方法

比較回数 (前節) ではなく実時間を測るため、固定シードの擬似乱数で生成した 12 文字の英数字キー N 個 ($N = 10^3$ から 10^6 まで対数等間隔の 7 点) を用い、(1) N 件の挿入に要する合計時間、(2) 挿入済みキーからランダムに選んだ 10 万回の検索の 1 回あたり平均時間、を計測した。比較対象は、提出版と同構成の `fixed17` (加算ハッシュ・ $m = 17$ 固定), 改良版 `improved` (FNV-1a・動的リハッシュ), 標準ライブラリの `std::unordered_map` (チェーン法) および `std::map` (平衡二分探索

木) の 4 構造である。時刻取得には `std::chrono::steady_clock` を用い、各条件を 5 回繰り返した中央値を採用した (全測定値は付録 B.7)。なおこの検索時間の測定対象は成功検索のみであり、不成功検索の挙動は比較回数・プローブ数の実験 (5.3 節および 5.5.5 節) で扱う。`fixed17` は $N > 10^5$ では 1 回の検索に平均約 3000 回の文字列比較を要し測定が現実的でないため $N \leq 10^5$ までとした。コンパイルは最適化オプション `-O2` を付して行い、検索結果が最適化で除去されないよう検索の成功回数を集計・出力する構成とした。

結果を表 8 および図 7, 図 8 に示す。

表 8: 要素数 N に対する挿入・検索時間 (5 回の中央値)			
データ構造	N	挿入合計 [ms]	検索平均 [ns]
<code>fixed17</code>	10^3	0.035	567.9
	10^4	0.306	3440.7
	10^5	3.487	374498.1
<code>improved</code>	10^3	0.052	133.7
	10^4	0.592	37.6
	10^5	13.283	27.8
	10^6	1496.228	952.6
<code>std::unordered_map</code>	10^3	0.054	26.6
	10^4	0.783	28.3
	10^5	20.205	31.6
	10^6	3584.558	1321.6
<code>std::map</code>	10^3	0.155	331.6
	10^4	3.825	240.6
	10^5	95.797	872.1
	10^6	6074.825	12907.7

5.4.2 提出版の性能特性

`fixed17` の検索時間は $N = 10^3$ で 568 ns, $N = 10^4$ で 3.44 μ s, $N = 10^5$ で 374 μ s と, N にほぼ比例して増加した。これは 5.3 節の比較回数の実験と整合し, バケット数固定のハッシュ表が漸近的には $O(N)$ の構造であることを実時間でも裏付けている。挿入は先頭挿入なので 1 件あたりほぼ一定 ($N = 10^5$ でも合計 3.5 ms) であり, 「挿入は速いが検索が伸びる」という理論どおりの非対称性が観察された。

注目すべきは, 検索時間の増加が比較回数の増加 (N に比例, 100 倍) を上回る 659 倍に達している点である。1 ノードあたりの実時間に換算すると, $N = 10^3$ では約 19 ns, $N = 10^5$ では約 127 ns と 6 倍以上悪化している。チェーンが長くなるとノード群がキャッシュに収まらなくなり, 1 ノード辿るごとに主記憶アクセスが発生するためであり, 5.5.2 節で述べる「比較回数の理論だけでは実時間を説明できない」というキャッシュ効率の問題を, 本課題の規模でも定量的に観察できたことになる。

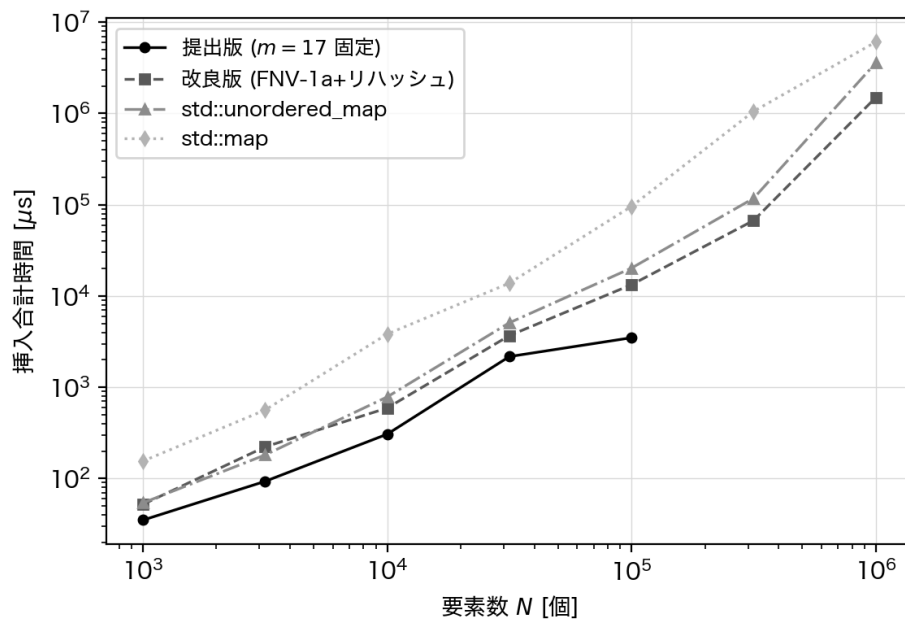


図 7: 要素数 N と挿入合計時間の関係（両対数, 5 回の中央値）

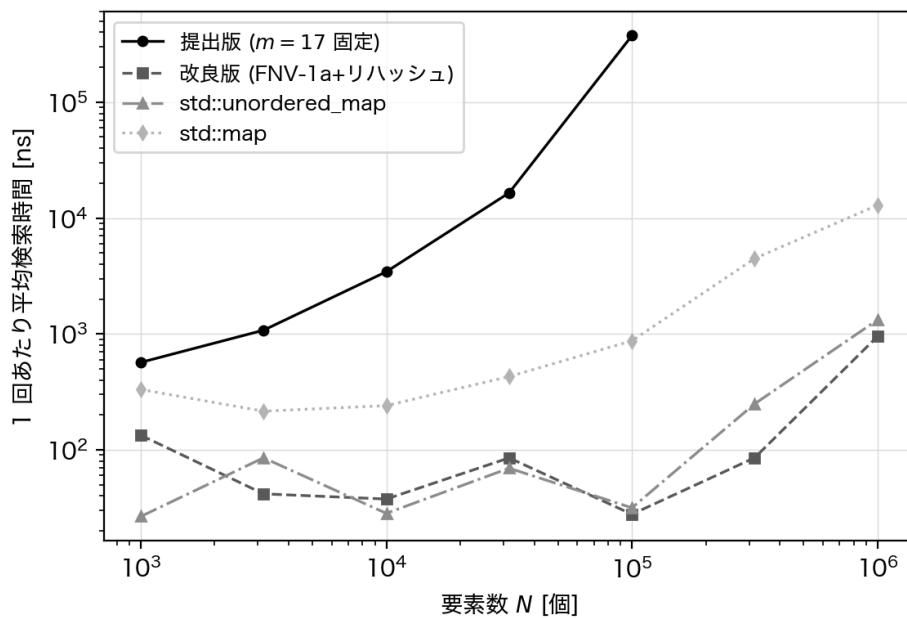


図 8: 要素数 N と 1 回あたり平均検索時間の関係（両対数, 5 回の中央値）

5.4.3 改良版と標準ライブラリの比較

`improved` の検索時間は $N = 10^4 \sim 10^5$ で 30 ns 前後とほぼ一定であり、動的リハッシュによって負荷率が抑えられ、平均 $O(1)$ 検索が実時間でも成立している。 $N = 10^6$ で 953 ns へ増加するのは、表とノード群の総量が CPU キャッシュを超え、1 回の検索に伴うキャッシュミスが増えるためと考えられ、同じ増加傾向は `std::unordered_map` (31.6 ns $\rightarrow$ 1321.6 ns) にも現れている。

構造間の比較では、第一に、`improved` は全域で `std::unordered_map` と同等以上の性能を示した。 $N = 10^6$ では挿入合計 1.50 s 対 3.58 s、検索平均 953 ns 対 1322 ns である。これは本実装が優れているというより、`std::unordered_map` が規格上の制約（要素のポインタ安定性等）を満たすための機構を抱えている [2] のに対し、本実装は機能を 3 操作に絞った単純な構造であることの帰結と解釈すべきである。第二に、`std::map` は検索が $\log N$ に従って増加し ($N = 10^6$ で 12.9 μ s)、 $N = 10^6$ における検索は `improved` の約 13.5 倍遅い。キーの順序走査が不要で検索・挿入・削除だけが必要な用途では、ハッシュ表が平衡二分探索木より明確に有利であることが確認できた。

なお、個々の測定値には実行ごとに数倍程度のばらつきがあり（付録 B.7）、これは測定を汎用 OS 上で行ったことによる他プロセスの影響が主因と考えられる。本考察では 5 回の中央値を用いることでこの影響を抑えた。また本測定では 4 構造を共通の仮想関数インタフェース越しに呼び出して条件を揃えているため、仮想関数呼び出しの固定費が全構造に様に上乗せされている。この固定費は操作自体が軽い構造ほど相対的に大きく効くので、本実装と標準ライブラリの差は実際にはここで示した値よりわずかに開く方向にあると考えられる。

5.4.4 キー長の影響

ここまでの実験はキー長をほぼ一定（英単語または 12 文字）としてきたが、ハッシュ関数の計算量はキー長 L に比例するため、長いキーでは様相が変わるはずである。キー長を 8 から 256 文字まで倍々に変えて、ハッシュ計算単体と成功検索の平均時間（15 回反復の最小値）を計測した。結果を図 9 に示す。

FNV-1a の計算時間は $L = 8$ で 5.0 ns、 $L = 64$ で 70.6 ns、 $L = 256$ で 529.9 ns と、おおむね L に比例して増加した。一方、加算ハッシュは $L = 256$ でも 6.9 ns とほぼ一定のままである。この差は品質と速度のトレードオフの構造を示している。加算ハッシュの総和計算は繰返し間に依存関係がないためコンパイラが SIMD 命令で並列化できるのに対し、FNV-1a は「排他的論理和してから乗算」の結果が次の文字の計算に必要という逐次依存をもつため、1 文字ずつしか進められない。5.2 節で見た攪拌の良さは、まさにこの逐次的な混ぜ込みから来ているので、両者は表裏の関係にある。検索時間はキーが長くなるとハッシュ計算に支配され、 $L = 256$ では 567.9 ns 中の約 530 ns (9 割超) がハッシュ計算である。

この観察は、2020 年代のハッシュ関数が採る設計をよく説明する。wyhash 系の rapidhash などは、キーを 16~48 バイト単位のブロックで読み、ブロック内は並列に処理できる乗算の構造を使うことで、逐次依存の連鎖を断ちつつ攪拌を保つ [9, 11]。すなわち「品質は保ったまま並列性を回復する」ことが長いキーへの現代的な答えである。本実装の範囲でも、ノードにハッシュ値を保存しておきリハッシュ時の再計算を省く、比較の前にハッシュ値のタグで棄却する、といった「ハッシュ計算・比較の回数自体を減らす」改良が、長いキーに対して特に効果的になると考えられる。

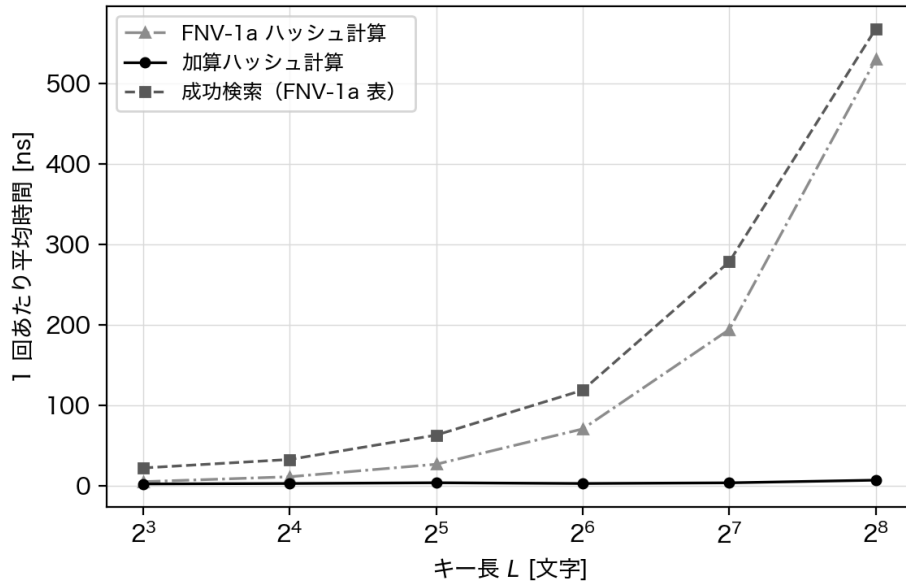


図 9: キー長 L とハッシュ計算・検索時間の関係 ($n = 10000$, 15 回反復の最小値)

5.5 2020 年以降の知見に基づく改良について

5.5.1 改良版の設計

以上の実験結果と近年の文献を踏まえ、2 点の改良を加えた `hashtable_improved.cpp` を作成した。第一の改良はハッシュ関数の FNV-1a への置き換えである。5.2 節で示したとおり、加算ハッシュは表の拡大とともに偏りが顕在化するため、表の大きさによらず一様性を保つハッシュ関数が必要である。FNV-1a は 1 文字あたり排他的論理和 1 回と乗算 1 回という最小限の計算で実用的な一様性を達成し (表 5)、実装も 5 行程度と教育的な文脈でも見通しがよい。第二の改良は動的リハッシュである。負荷率が 1.0 を超えた時点でバケット数を「現在の 2 倍以上で最小の素数」($17 \rightarrow 37 \rightarrow 79 \rightarrow \dots$) に拡張し全要素を再配置することで、 α を常に一定以下に保ち、平均探索コストを式 (4) の意味で $O(1)$ に維持する。拡張のたびに全要素の再配置 ($O(n)$) が発生するが、表の大きさが等比数列的に成長するため、挿入 1 回あたりに均した償却コストは定数に収まる。これは 4.4 節で確認した拡張列と一致する。

この 2 つの改良が連動している点が本質的である。リハッシュだけを加算ハッシュのまま導入すると、 m が大きくなった時点で 5.2.2 節の偏りが現れ、負荷率を下げてでもチェーンが短くならないという形で破綻する。すなわち「よいハッシュ関数」は「表の成長」の前提条件である。リハッシュ処理の流れを図 10 に示す。

5.5.2 チェイン法の本質的限界とキャッシュ効率

本課題のようなノードを `new` で個別確保するチェイン法は、C++ 標準ライブラリの `std::unordered_map` と同系統の構造である。実は C++ 標準規格が要素へのポインタ安定性を要求するため、`std::unordered_map` は事実上チェイン法での実装を強制されていることが知られている [2]。しかし 2020 年代の複数の大規模ベンチマークは、この方式が現代の計算機では不利であることを一致して報告している。Leitner-Ankerl による 29 実装の総合ベンチマーク (2022 年) では、`std::unordered_map` は検索・挿入・メモリ使

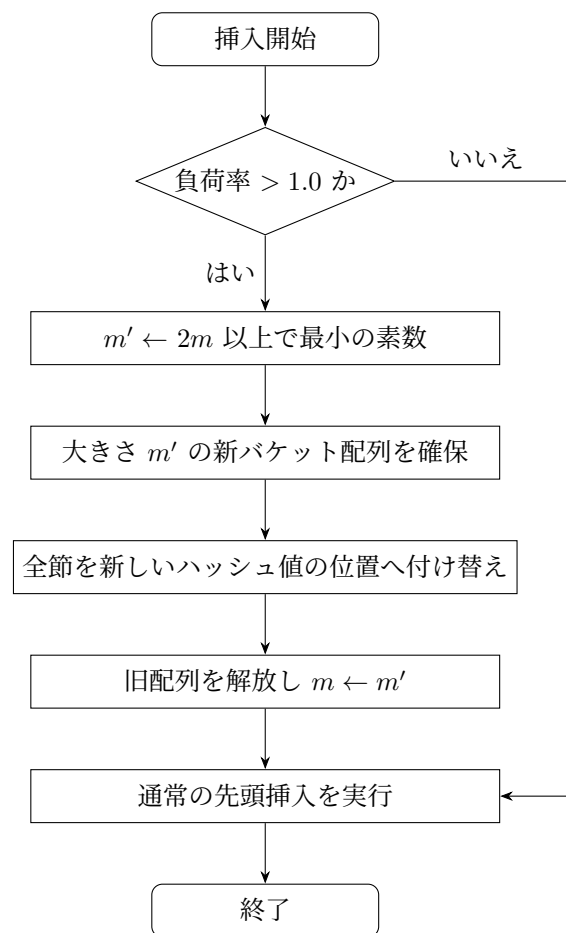


図 10: 改良版における動的リハッシュを含む挿入の流れ

用量のほぼ全項目で開放番地法系の実装に劣り [3], Allan による 2024 年のベンチマークでも, 表がキャッシュに収まらない規模ではチェーンのノードを 1 つ辿るごとにキャッシュミスが 1 回発生することが劣位の主因と分析されている [4].

この知見は本実装の性能特性の解釈に直結する. 2.5 節の理論は「比較回数」を数えるが, 現代の CPU では主記憶へのアクセス (キャッシュミス) 1 回が数百命令分の時間に相当するため, 同じ比較回数でもメモリ配置によって実時間は大きく変わる. チェイン法のノードはヒープ上に散在するため, チェインを辿る操作は本質的にキャッシュミスの連鎖である. 比較回数の理論 (図 6) が正確に成立してもなお, 実時間では連続メモリ配置の構造に及ばないという二層構造を理解することが重要である.

5.5.3 現代的ハッシュ表の設計動向

チェーン法の限界に対する 2020 年代の主流の答えは, 「要素を連続配列に直接格納し (開放番地法), ハッシュ値の一部をメタデータとして分離保存し, SIMD 命令で複数スロットを並列照合する」設計である. Boost 1.81 (2022 年) で導入された `boost::unordered_flat_map` は, 15 スロットごとに 16 バイトのメタデータ語を持ち, 縮約ハッシュ値を SSE2/Neon 命令で一括比較することで, 1 回のメモリアクセスあたりの照合数を大幅に増やしている [1]. 同系統の設計は Google の Swiss Tables に始まり, Go 言語 1.24 (2025 年) の組込み `map` にも採用された. Go の実装ではハッシュ値を上位 57 ビット (グループ選択用) と下位 7 ビット (メタデータ用) に分割し, 8 スロット分の状態を 64 ビット制御語 1 つで並列照合することで, 負荷率 7/8 まで性能劣化なく運用でき, マイクロベンチマークで最大 60% の高速化を得たと報告されている [6]. Datadog はこの Go 1.24 の `map` への移行だけで単一サービスのヒープを約 500 MiB 削減したと報告しており [7], 設計の違いが実運用のメモリ・速度に直接効くことがわかる. また Leitner-Ankerl の `ankerl::unordered_dense` (2022 年) は全要素を `std::vector` に密に格納することで反復速度とメモリ効率を高めており [5], 「ノードを個別確保しない」ことが共通の設計原理になっている.

本課題の実装をこの系譜に位置づけると, 改良版 (FNV-1a + リハッシュ) は「ハッシュ関数の品質」と「負荷率の制御」までを解決した段階にあり, 残る差分は「メモリ配置」である. さらなる改良としては, (1) ノードを可変長配列にまとめて確保しチェーンを添字でつなぐ (キャッシュミス削減), (2) 各ノードにハッシュ値の下位バイトをタグとして保存し, 文字列比較の前にタグ比較で棄却する (比較コスト削減, Swiss Tables の縮約ハッシュに相当), の 2 点が, クラスの外部仕様 (`() / insert / -=`) を変えずに適用できる現実的な候補である.

5.5.4 リハッシュの償却コストの実測

動的リハッシュは「たまに全要素を再配置する」操作であるから, 個々の挿入の所要時間は一様ではない. 理論上は, 表の大きさが等比数列的に成長する限り再配置の総仕事量は挿入数の定数倍に収まり, 1 回あたりの償却コストは $O(1)$ となる. これを実測で確認するため, 改良版と同じ構成の表に 20 万件のキーを挿入し, 挿入 1 件ごとの所要時間を個別に記録した. 結果を図 11 に示す.

20 万件の挿入中に発生したリハッシュの一覧を表 9 に示す. リハッシュは 14 回発生し (バケット数は 17 から 350899 まで成長), 挿入 1 件のレイテンシは最大 23.8 ms に達した.

このとき移動される節数はその時点の要素数 (表 9 の挿入番号にほぼ等しい) である. 発生間隔とバケット数がともに等比数列的に増えている様子が読み取れる. これは通常の挿入 (数十～数百 ns) の

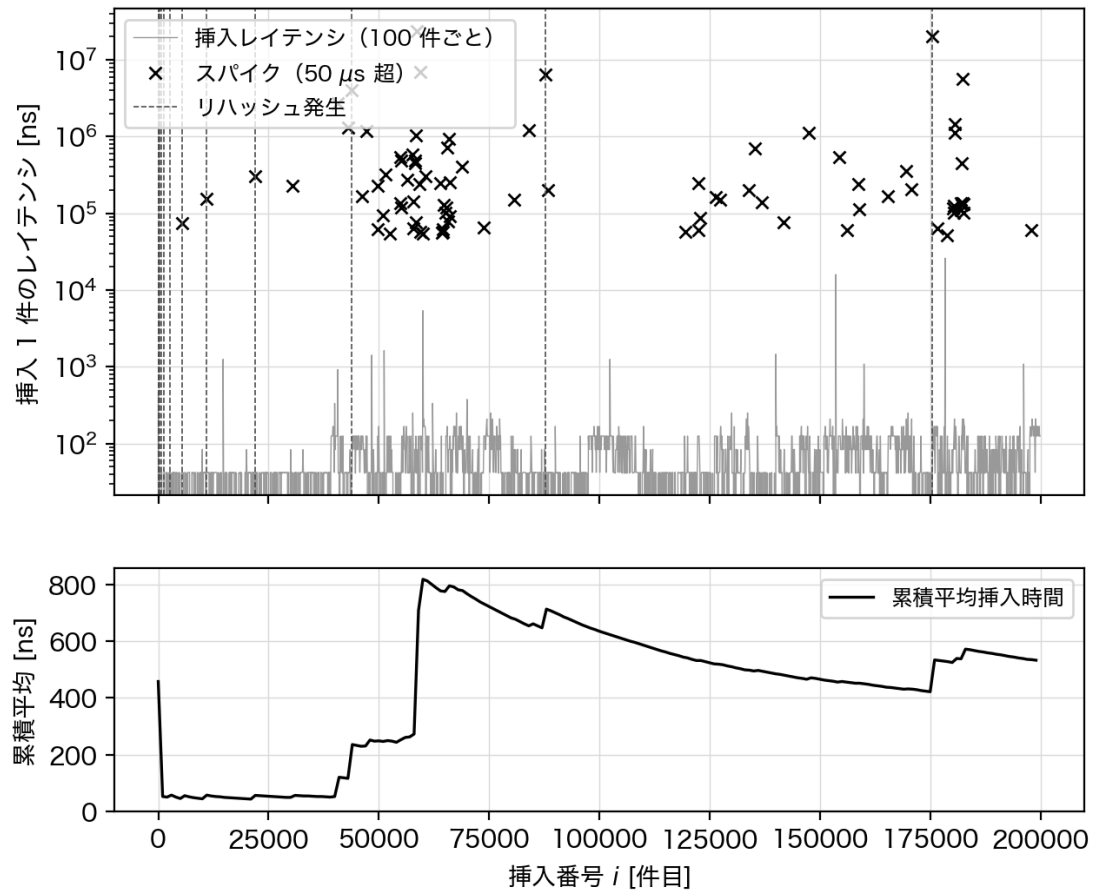


図 11: 挿入レイテンシの時系列（上: 個別レイテンシと リハッシュ発生位置, 対数軸）と累積平均挿入時間（下）（ $N = 200000$ ）

表 9: 20 万件挿入中に発生したリハッシュの一覧

発生した挿入番号	新バケット数
18	37
38	79
80	163
164	331
332	673
674	1361
1362	2729
2730	5471
5472	10949
10950	21911
21912	43853
43854	87719
87720	175447
175448	350899

数万倍である．それにもかかわらず全挿入の平均は 531 ns に収まった．図 11 下段の累積平均は，大きなスパイクの直後に段状に持ち上がるものの，その後続く大量の高速な挿入によって押し下げられて収束していく．スパイク 1 回の仕事量（全要素の再配置）は表の成長とともに倍々に増えるが，発生間隔も倍々に開くため，平均への寄与が一定に抑えられるのである．これが償却 $O(1)$ の実体である．なお図中の $50\ \mu\text{s}$ 超のスパイクにはリハッシュ発生位置（破線）と一致しないものも多数見られるが，これらは主にメモリ確保に伴う OS のページ割り当てなど測定環境側の要因と考えられ，リハッシュ由来の最大級のスパイク（ms 単位）とは規模が 1～2 桁異なる．

この結果は同時に，動的リハッシュの限界も示している．平均は速くても，最悪の 1 回は 24 ms 止まるため，1 回ごとの応答時間に上限が要求される用途（対話処理や実時間制御）では，再配置を 1 度に行わず複数の操作に分割して漸進的に進める方式や，あらかじめ最終規模で表を確保しておく方式を検討する必要がある．償却計算量と最悪計算量の区別が実用上意味をもつことを示す好例といえる．

5.5.5 開放番地法との比較実験

2.3 節で述べたとおり，衝突処理にはチェーン法のほかに開放番地法があり，両方式の対比は近年の解説記事でも標準的な題材である [16, 17]．5.5.3 節で見た現代の実装がすべて開放番地法系であることを踏まえ，両者の探索コストの性質の違いを自分の実装で確かめた．開放番地法として最も基本的な線形走査（衝突したら次のスロットを順に調べる方式，削除は墓標で表現）を FNV-1a と 2 の幂の容量で実装し，容量 131072 を固定して挿入数を変えることで負荷率 α を 0.25 から 0.95 まで制御し，成功・不成功検索の平均プローブ数（スロットまたはバケットの参照回数）を計測した．結果を図 12 に示す．図中の点線は理論値であり，チェーン法は成功 $1 + \alpha/2$ ・不成功 $1 + \alpha$ ，線形走査は成功 $\frac{1}{2}(1 + \frac{1}{1-\alpha})$ ・不成功 $\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$ である（線形走査の式は Knuth による古典的解析として知られる）．

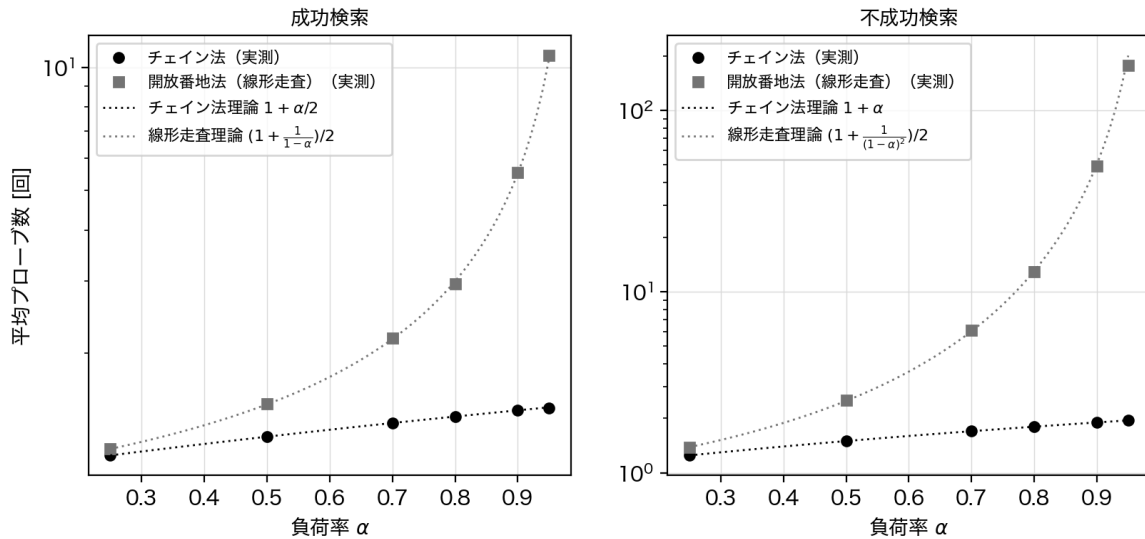


図 12: チェイン法と開放番地法（線形走査）の平均プローブ数の比較（左: 成功検索，右: 不成功検索，容量 131072）

実測は両方式とも理論値にほぼ完全に一致した．例えば $\alpha = 0.9$ における線形走査の成功検索は実測 5.53 回（理論 5.50 回），不成功検索は実測 49.4 回（理論 50.5 回）である．プローブ数だけを見る

と、チェーン法は $\alpha = 0.95$ でも成功 1.47 回・不成功 1.95 回と穏やかに劣化するのに対し、線形走査は $\alpha \rightarrow 1$ で爆発的に悪化する ($\alpha = 0.95$ で不成功 176.8 回)。衝突した要素が連続スロットを占有して塊 (クラスタ) を作り、塊が大きいほどさらに衝突を吸い寄せるといった正帰還が原因である。

この結果は一見、チェーン法の優位を示すように見える。それでも現代の高速実装が開放番地法を選ぶのは、プローブ 1 回あたりの実コストが桁違いに安いからである。線形走査のプローブは連続メモリ上の隣接スロットへの参照であり、多くはキャッシュに載っている。対してチェーン法のプローブはヒープ上に散在するノードへのポインタ追跡であり、1 回ごとにキャッシュミスを起こし得る (5.4.2 節で実測したとおり、チェーンを 1 ノード辿る実時間は表の規模によって 19 ns から 127 ns まで変動した)。さらに実装側は負荷率に上限を課してプローブ数の爆発を防いでいる。boost::unordered_flat_map は最大負荷率 0.875 [1]、Go 1.24 の map は 7/8 [6] であり、図 12 の横軸でいえば爆発が始まる手前で必ずリハッシュする設計になっている。すなわち両方式の選択は「プローブ数で有利なチェーン法」対「プローブ単価で有利な開放番地法」のトレードオフであり、後者は負荷率制御という運用条件を追加することで成立している。

一方、本課題の仕様には削除 == が含まれるため、チェーン法の利点も明確である。チェーン法の削除はリストからの節の除去で完結するのに対し、開放番地法の削除は墓標 (削除済み印) を残す必要があり、墓標が蓄積すると不成功検索がさらに劣化して定期的な作り直しが要る。3 操作が対等に要求される本課題の範囲では、チェーン法は実装の単純さと削除の素直さの点で妥当な選択であったと考えられる。

5.5.6 タグ付き比較の実装と評価

5.5.3 節で見た現代的設計のうち、チェーン法という枠組みを保ったまま適用できる要素として「縮約ハッシュの事前照合」がある。Swiss Tables 系の実装はハッシュ値の一部をメタデータとして保存し、本来のキー比較の前にこの短い値を照合して候補を絞り込む [1, 6]。この着想をチェーン法に移植した発展版 hashtable_tagged.cpp (付録 A.14) を作成した。変更点は 3 つである。第一に、節 (Node) に挿入時に計算した FNV-1a の 32 ビット値全体をタグとして保存する。第二に、検索・削除のチェーン走査で、まず探索キーのハッシュ値と節のタグを整数比較し、一致したときに限り文字列比較を行う。第三に、リハッシュ時には保存済みタグから新しいバケット番号を計算し、キー文字列の再ハッシュを行わない。なお本実装の節は詰め物を含めて実質 48 バイト境界に丸められるため、タグ 4 バイトの追加は実メモリ消費を増やさない。

効果は、環境ノイズに左右されない決定的な指標として「検索 1 回あたりの文字列比較回数」で評価した ($N = 200000$, $\alpha = 0.570$, 10 万回検索)。結果を表 10 に示す。

タグなしの実測値は理論値 (成功 $1 + \alpha/2 = 1.285$, 不成功 $\alpha = 0.570$) と一致する。タグ付きでは、成功検索の文字列比較が「一致する節の 1 回」だけに削減され、不成功検索ではほぼ完全にゼロ (10 万回あたり 4~7 回) となった。残る数回は 32 ビットタグの偶然の一致であり、その確率はチェーン上の節 1 つあたり 2^{-32} 程度という見積もりと整合する。すなわち、不要な文字列比較という仕事そのものを原理的に消し去る改良であることが確認できた。また挿入 (リハッシュ 14 回を含む) の合計時間は、キー再ハッシュの省略によりキー長 64 で 301 ms から 58 ms へと一貫して短縮された。一方、検索の実時間は本実験の規模では実行ごとのばらつき (OS のスケジューリングによる数倍の変動) に埋もれ、安定した差として観測できなかった。文字列比較 0.3 回分の削減は数 ns 規模であり、チェイ

表 10: タグ付き比較による文字列比較回数の削減 ($N = 200000$, $\alpha = 0.570$, 10 万回検索の平均)

実装	キー長	成功検索 [回/検索]	不成功検索 [回/検索]
タグなし (改良版相当)	12	1.28728	0.56954
	64	1.28554	0.56925
	256	1.28809	0.56863
タグ付き	12	1.00000	0.00006
	64	1.00001	0.00007
	256	1.00002	0.00004

ン走査のキャッシュミス (数十～百 ns) が支配的な本構造では、タグの真価はキー比較が高価な場合 (長いキー、比較関数が重い型) や、メタデータを連続配置して SIMD で並列照合する段階 (5.5.3 節) まで進んで初めて実時間に現れると考えられる。

5.6 メモリ使用量について

実行時間に加えて、メモリ使用量も構造選択の重要な判断軸である。12 文字のランダムキー N 件の挿入前後でプロセスの最大常駐メモリ量 (getrusage の `ru_maxrss`) を測り、その増分を要素数で割った「1 要素あたりの増分メモリ」を比較した。結果を図 13 に示す。

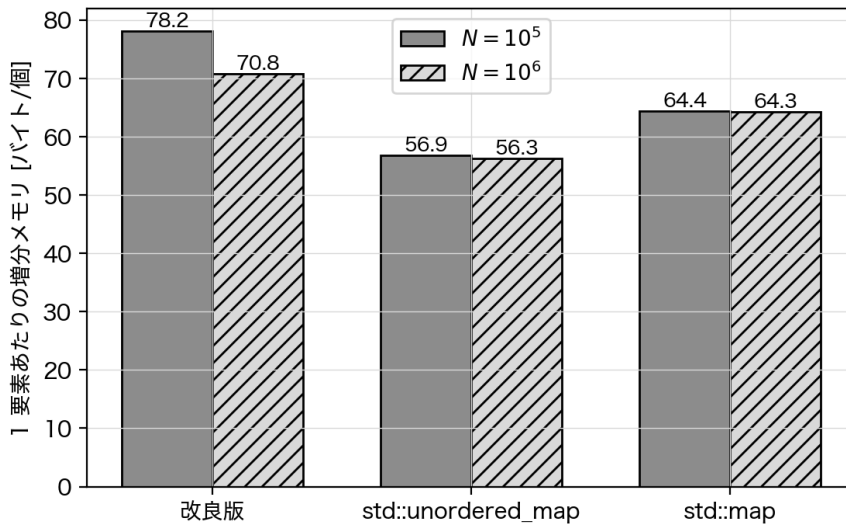


図 13: 1 要素あたりの増分メモリ使用量の比較 (12 文字キー, 値は int)

$N = 10^6$ における 1 要素あたりの増分は、改良版 70.8 バイト, `std::unordered_map` 56.3 バイト, `std::map` 64.3 バイトであった。実行時間では優位だった改良版が、メモリでは最も多く消費している。この内訳は構造から説明できる。改良版のノードは `std::string` (24 バイト) + `int` (4 バイト + 詰め物 4 バイト) + 次ポインタ (8 バイト) の 40 バイトだが、メモリ確保器が 16 バイト境界に切り上げるため実質 48 バイトを占める。これにバケット配列 (最終的に約 140 万バケット $\times$ 8 バイト $\approx$ 11 バイト/要素) が加わって約 59 バイト/要素となり、さらにリハッシュ中は新旧のバケット配列が一時的に共存して最大常駐量を押し上げる。残余は確保器の管理領域と考えられる。一方 `std::map` の実測

値 64.3 バイトは、キー・値対 32 バイト + 子ポインタ 2 本と親ポインタ・色情報 32 バイトという赤黒木ノードの理論値 64 バイトとほぼ一致する。

なお 12 文字のキーは `std::string` の短文字列最適化 (SSO) によりオブジェクト内に格納されるため、本測定にはキー本体のヒープ確保が含まれていない。キーが長い場合は全構造に同じだけの上乗せが生じる。ノードを個別確保するチェイン法がメモリ面でも不利になりやすいことは近年のベンチマークでも指摘されており [4]、全要素を連続領域に密に格納する設計 [5] や、メタデータをビット単位に圧縮する設計 [6] がこの問題への現代的な回答である。実運用でもハッシュ表の内部設計の変更だけで数百 MiB 規模のメモリ削減が得られた事例が報告されている [7]。

5.7 ハッシュ表の安全性について

5.2.4 節のアナグラム衝突は、性能問題であると同時に安全性の問題でもある。攻撃者が意図的に同一バケットへ衝突するキーを大量に送り込むと、ハッシュ表は 1 本の連結リストに退化し、操作あたり $O(n)$ の時間を消費させられる。これはハッシュ飽和攻撃 (HashDoS) として知られ、Web サーバがリクエストパラメータをハッシュ表に格納する実装を狙った攻撃が現実成立している [12]。加算ハッシュではアナグラムを列挙するだけで衝突キーを無限に生成できるため、本課題の提出版は敵対的環境では使用できない。

この攻撃が本課題の実装に対して実際に成立することを実験で確かめた。同一の文字マルチセット (a~d 各 3 文字) の並べ替えからなる攻撃キー 2000 個と、通常のランダムキー 2000 個をそれぞれ挿入し、全キーの検索コストを計測した。結果を表 11 に示す。

表 11: ハッシュ飽和攻撃の実証 ($n = 2000$, 検索 1 回あたりの平均)

実装	キー群	最長チェイン	平均比較回数	平均検索時間 [ns]
提出版 (加算, $m = 17$)	攻撃キー	2000	1000.50	9656.94
	ランダムキー	144	59.89	250.43
改良版 (FNV-1a)	攻撃キー	5	1.36	31.26
	ランダムキー	5	1.38	30.75

提出版では攻撃キー 2000 個の加算ハッシュ値がすべて一致してバケット 1 つに集中し (最長チェイン 2000)、平均比較回数は 1000.50 回と、チェイン 1 本の線形探索の理論値 $(n + 1)/2$ に正確に一致した。同じ要素数のランダムキー (59.89 回) と比べて約 17 倍、検索時間では約 39 倍の劣化である。攻撃者は要素数を増やすだけでこの劣化を際限なく深められる。一方、改良版では同じ攻撃キーが FNV-1a により 1416 バケットへ分散し (最長チェイン 5)、比較回数はランダムキーと区別がつかない 1.36 回に留まった。すなわちアナグラム型の攻撃に対しては、ハッシュ関数の置き換えだけで防御が成立する。ただしこれは FNV-1a の衝突を作ることが加算ハッシュより難しいというだけであり、FNV-1a も鍵をもたない公開のアルゴリズムである以上、衝突キーを計算で列挙する攻撃は原理的に可能である。

現代の言語ランタイムはこの攻撃へのより根本的な対策として、鍵付きハッシュ関数 SipHash と実行ごとに変わる乱数シードを採用している。Rust の `HashMap` は既定で SipHash-1-3 を用い [12]、Python の `hash()` も文字列に対して起動ごとに異なるシードを混ぜるため、同じ文字列のハッシュ値が実行ごとに変わる [13]。シードが秘密である限り攻撃者は衝突キーを事前計算できない。一方 2024 年の研究

では、シードがあっても設計が弱いハッシュ関数（wyhash など）はシード非依存の衝突を構成できることが示されており [8]、攻撃と防御の均衡は現在も動いている。理論面でも、攻撃下のハッシュ表の資源消費を保証付きで抑える手法が 2023 年に提案されるなど [14]、ハッシュ表は完成した技術ではなく現役の研究対象である。本課題の文脈では、「ハッシュ関数は性能特性であると同時にセキュリティ境界である」という視点を、加算ハッシュという最も壊れやすい実例から学べたことが収穫である。

5.8 C 版との設計比較

最後に、出発点である `hash.c` と本実装を設計の観点から比較する。

第一に資源管理である。`hash.c` は `malloc` で確保した節を一切解放せず、表の初期化も利用者が `main` で明示的に行う。C++ 版はコンストラクタが初期化を、デストラクタが全節の解放を自動的に行うため、初期化忘れ・解放忘れが構文的に起こらない。これは C++ の RAII（資源確保は初期化時に）という設計原理の直接の恩恵である。

第二に安全性である。`hash.c` の `add` は `strcpy(ptr->name, c)` で 16 バイト固定の配列にキーを複写するため、15 文字を超えるキーを渡すとバッファあふれ（未定義動作）が起こる。実際、本課題の実験で用いた英単語 5000 語には 16 文字以上の単語が含まれており、`hash.c` をそのまま実験に流用することはできなかった。C++ 版はキーを `std::string` で保持するため長さの制限自体が存在しない。C 言語でハッシュ表を実装する際の落とし穴と対策は近年の解説記事でも繰り返し指摘されており [15]、文字列の所有権管理は C 実装の主要な誤り源である。

第三に多重性と汎用性である。`hash.c` の表はグローバル変数なのでプログラム中に 1 個しか存在できず、値の型も `int` 固定である。C++ 版はオブジェクトなので必要なだけ生成でき、テンプレートにより任意の値型を収納できる（4.2 節で `int・double・std::string` を確認）。

第四にインタフェースの表現力である。`find(c)・add(c, s)` という関数名ベースの C 版に対し、C++ 版は演算子多重定義により `table("hoge")・table -= "hoge"` という宣言的な記法を提供する。どちらが読みやすいかは文脈によるが、演算子という選択肢が言語機能として存在すること自体が、ライブラリの利用感を設計対象にできるという C++ の特徴である。

以上のとおり、同じアルゴリズム（チェーン法＋加算ハッシュ）であっても、実装言語の機構によって安全性・汎用性・保守性は大きく変わる。アルゴリズムの正しさ（2 節）とプログラムとしての堅牢さ（本節）は独立の関心事であり、後者において C++ の `class` は明確な利点を提供した。

5.9 総括

本課題で実装・評価した各版と標準ライブラリの特徴を表 12 にまとめる。メモリの欄は 5.6 節の実測値（ $N = 10^6$ 、12 文字キー）である。

一連の実験を通じて得られた知見は次の 3 点に要約できる。第一に、課題仕様の 3 操作（`()・insert・-=`）は、チェーン法により少ないコードで正しく実現でき、その正しさは理論（式 (4)）どおりの実測性能として定量的に裏付けられた。第二に、素朴な実装と実用的な実装を分ける要素は、アルゴリズムの骨格ではなく「ハッシュ関数の品質」「負荷率の制御」「メモリ配置」という 3 つの工学的判断であり、それぞれの効果と限界を個別の実験（5.2 節、5.5.4 節、5.4.2 節）で切り分けて確認できた。第三に、2020 年代の実装（Swiss Tables 系・`boost::unordered_flat_map`・Go 1.24 の `map`）が到達して

表 12: 実装各版と標準ライブラリの特徴のまとめ

実装	ハッシュ関数	表の成長	検索 (平均)	メモリ [B/要素]	主な限界
提出版	加算	なし ($m = 17$)	$O(n/17)$	–	n 増で線形劣化
改良版	FNV-1a	素数倍化	$O(1)$	70.8	検索ごとのポインタ追跡
発展版 (タグ付き)	FNV-1a	素数倍化	$O(1)$	70.8	同上 (比較回数は削減)
<code>std::unordered_map</code>	処理系定義	素数系列	$O(1)$	56.3	規格制約による間接化
<code>std::map</code>	– (比較木)	平衡木	$O(\log n)$	64.3	キー比較が $\log n$ 回

いる設計は、本課題の実装が抱える限界（ポインタ追跡・比較コスト・負荷率と性能の緊張関係）を段階的に解消してきた延長線上にあり、データ構造の教科書的な基礎と最先端の実装が地続きであることを実感できた。配列・リスト・木・ハッシュ表といった基本データ構造の得失を「何を解決し何を失うか」で整理する近年の視点 [18] に照らしても、本課題のハッシュ表は「検索の速さを得る代わりに順序とメモリ局所性を失う」構造として明確に位置づけられる。

今後の課題としては、(1) ノードの連続領域への配置によるキャッシュミス削減の実装と計測、(2) 上書き型 `insert` への変更 (5.1.4 節)、(3) C++20 concepts によるテンプレート引数要求の明示 (5.1.3 節)、の 3 点が挙げられる。いずれも本レポートで確立した測定基盤（実験プログラム群と反復測定の手順）の上で定量評価が可能である。

6 感想

ハッシュ表そのものは C 言語の演習で一度実装していたが、同じアルゴリズムを C++ の `class` として書き直し、さらに実験でハッシュ関数の性質を測ってみると、「動くコード」と「よい設計」の間に多くの論点があることが分かった。特に、加算ハッシュが表の大きさ次第で破綻する様子や、理論式 $1 + \alpha/2$ が実測とほぼ完全に一致する様子を自分の実験で確認できたのは面白かった。また、ハッシュ表が 2020 年代でも言語ランタイムの改良対象であり続けていることを文献調査で知り、基礎的なデータ構造の奥行きを感じた。

参考文献

- [1] Joaquín M López Muñoz: “Inside `boost::unordered_flat_map`”, Bannalia blog, 2022 年 11 月.
<https://bannalia.blogspot.com/2022/11/inside-boostunorderedflatmap.html>
- [2] Joaquín M López Muñoz: “Advancing the state of the art for `std::unordered_map` implementations”, Bannalia blog, 2022 年 6 月.
<https://bannalia.blogspot.com/2022/06/advancing-state-of-art-for.html>
- [3] Martin Leitner-Ankerl: “Comprehensive C++ Hashmap Benchmarks 2022”, 2022 年 8 月.
<https://martin.ankerl.com/2022/08/27/hashmap-bench-01/>
- [4] Jackson Allan: “An Extensive Benchmark of C and C++ Hash Tables”, 2024 年 5 月.
https://jacksonallan.github.io/c_cpp_hash_tables_benchmark/

- [5] Martin Leitner-Ankerl: “ankerl::unordered_dense::{map, set} — A fast & densely stored hashmap”, GitHub, 2022 年（初版）.
https://github.com/martinus/unordered_dense
- [6] Michael Pratt: “Faster Go maps with Swiss Tables”, The Go Blog, 2025 年 2 月.
<https://go.dev/blog/swisstable>
- [7] Nayef Ghattas: “How Go 1.24’s Swiss Tables saved us hundreds of gigabytes”, Datadog Engineering Blog, 2025 年 7 月.
<https://www.datadoghq.com/blog/engineering/go-swiss-tables/>
- [8] Orson Peters: “Breaking CityHash64, MurmurHash2/3, wyhash, and more...”, 2024 年.
<https://orlp.net/blog/breaking-hash-functions/>
- [9] Nicolas De Carli: “rapidhash — Very fast, high quality, platform-independent hashing algorithm”, GitHub, 2024 年.
<https://github.com/Nicoshev/rapidhash>
- [10] Reini Urban: “smhasher: Hash function quality and speed tests”, GitHub, 2024 年（継続更新）.
<https://github.com/rurban/smhasher>
- [11] Timofey Prodanov: “Benchmarking non-cryptographic hash functions in Rust”, Medium, 2023 年.
<https://medium.com/@tprodanov/benchmarking-non-cryptographic-hash-functions-in-rust-2e6091077d11>
- [12] Sling Academy: “Security considerations: HashDoS and Rust’s default SipHash”, 2025 年.
<https://www.slingacademy.com/article/security-considerations-hashdos-and-rusts-default-siphash/>
- [13] Chenna Kautilya: “Python hash() is not deterministic”, 2023 年 12 月.
<https://chenna.me/blog/2023/12/25/python-hash-is-not-deterministic/>
- [14] Trisha Chakraborty, Jared Saia, Maxwell Young: “Defending Hash Tables from Subterfuge with Depth Charge”, arXiv:2308.04305, 2023 年.
<https://arxiv.org/abs/2308.04305>
- [15] Ben Hoyt: “How to implement a hash table (in C)”, 2021 年.
<https://benhoyt.com/writings/hash-table-in-c/>
- [16] 情報工学入門チャンネル: “Python で学ぶハッシュ探索法 — チェーン法とオープンアドレス法”, Zenn, 2023 年.
<https://zenn.dev/fikastudio/articles/efcfe246642553>
- [17] tachu: “ハッシュテーブルを実装して理解するぜ”, Qiita, 2025 年.
<https://qiita.com/tachu/items/df25bbd3648fd187dc37>
- [18] hideyuki: “配列、リスト、木、ハッシュは何を解決し、何を失ったのか”, Zenn, 2026 年.
https://zenn.dev/tamai_hideyuki/articles/data-structure-history-and-tradeoffs

A 付録: プログラムソース

以下に、実際にコンパイル・実行したプログラムファイルを無加工で添付する。

A.1 hashtable_v1.cpp (非テンプレート版)

```
#include <iostream>
#include <string>

// hash.c をそのまま C++ class に移植した非テンプレート版 (値は int 固定)
class HashTable {
public:
    static constexpr int HASHSIZE = 17;

    // チェインのノード
    struct Node {
        std::string key;
        int value;
        Node* next;
    };

    // 全バケットを nullptr で初期化
    HashTable() {
        for (int i = 0; i < HASHSIZE; ++i) {
            buckets_[i] = nullptr;
        }
    }

    // 全ノードを削除
    ~HashTable() {
        for (int i = 0; i < HASHSIZE; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                delete p;
                p = next;
            }
        }
    }

    // コピー禁止
    HashTable(const HashTable&) = delete;
    HashTable& operator=(const HashTable&) = delete;

    // ハッシュ値を計算 (文字コードの総和 % HASHSIZE)
    static int hash(const std::string& key) {
        int hashval = 0;
        for (char c : key) {
            hashval += static_cast<unsigned char>(c);
        }
        return hashval % HASHSIZE;
    }

    // 先頭挿入で key-value を登録
    bool insert(const std::string& key, int value) {
        int h = hash(key);
        Node* p = new Node{key, value, buckets_[h]};
        buckets_[h] = p;
        return true;
    }

    // key に対応する値へのポインタを返す (見つからなければ nullptr)
    const int* operator()(const std::string& key) const {
        int h = hash(key);
        Node* p = buckets_[h];
        while (p != nullptr) {
            if (p->key == key) {
                return &(p->value);
            }
            p = p->next;
        }
    }
};
```

```

        return nullptr;
    }

    // key を削除する (存在しなければ何もしない)
    HashTable& operator-=(const std::string& key) {
        int h = hash(key);
        Node* p = buckets_[h];
        Node* prev = nullptr;
        while (p != nullptr) {
            if (p->key == key) {
                if (prev == nullptr) {
                    buckets_[h] = p->next;
                } else {
                    prev->next = p->next;
                }
                delete p;
                return *this;
            }
            prev = p;
            p = p->next;
        }
        return *this;
    }

private:
    Node* buckets_[HASHSIZE];
};

int main() {
    HashTable ht;

    std::cout << "--- insert ---" << std::endl;
    ht.insert("takimoto", 42);
    ht.insert("katsurada", 122);
    ht.insert("matsuzawa", 35);
    ht.insert("ohmura", 12);
    std::cout << "takimoto=42, katsurada=122, matsuzawa=35, ohmura=12 を挿入しました" << std::endl;

    std::cout << "--- find (success) ---" << std::endl;
    const int* p;
    p = ht("takimoto");
    std::cout << "takimoto: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;
    p = ht("katsurada");
    std::cout << "katsurada: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;
    p = ht("matsuzawa");
    std::cout << "matsuzawa: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;
    p = ht("ohmura");
    std::cout << "ohmura: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;

    std::cout << "--- find (failure) ---" << std::endl;
    p = ht("nonexistent");
    std::cout << "nonexistent: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;

    std::cout << "--- erase ---" << std::endl;
    ht -= "katsurada";
    std::cout << "katsurada を削除しました" << std::endl;
    p = ht("katsurada");
    std::cout << "katsurada (after erase): " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;

    p = ht("takimoto");
    std::cout << "takimoto (after erase): " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;

    return 0;
}

```

A.2 hashtable.cpp (テンプレート版・提出プログラム)

```

#include <iostream>
#include <string>

// キーは std::string 固定、値は任意型 T のテンプレート版ハッシュ表 (チェーン法)
template <class T>

```

```

class HashTable {
public:
    static constexpr int HASHSIZE = 17;

    // チェインのノード
    struct Node {
        std::string key;
        T value;
        Node* next;
    };

    // 全バケットを nullptr で初期化
    HashTable() : size_(0) {
        for (int i = 0; i < HASHSIZE; ++i) {
            buckets_[i] = nullptr;
        }
    }

    // 全ノードを削除
    ~HashTable() {
        for (int i = 0; i < HASHSIZE; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                delete p;
                p = next;
            }
        }
    }

    // コピー禁止
    HashTable(const HashTable&) = delete;
    HashTable& operator=(const HashTable&) = delete;

    // ハッシュ値を計算 (文字コードの総和 % HASHSIZE)
    static int hash(const std::string& key) {
        int hashval = 0;
        for (char c : key) {
            hashval += static_cast<unsigned char>(c);
        }
        return hashval % HASHSIZE;
    }

    // 先頭挿入で key-value を登録
    bool insert(const std::string& key, const T& value) {
        int h = hash(key);
        Node* p = new Node{key, value, buckets_[h]};
        buckets_[h] = p;
        ++size_;
        return true;
    }

    // key に対応する値へのポインタを返す (見つからなければ nullptr)
    const T* operator()(const std::string& key) const {
        int h = hash(key);
        Node* p = buckets_[h];
        while (p != nullptr) {
            if (p->key == key) {
                return &(p->value);
            }
            p = p->next;
        }
        return nullptr;
    }

    // key を削除する (存在しなければ何もしない)
    HashTable& operator-=(const std::string& key) {
        int h = hash(key);
        Node* p = buckets_[h];
        Node* prev = nullptr;
        while (p != nullptr) {
            if (p->key == key) {
                if (prev == nullptr) {
                    buckets_[h] = p->next;
                }
            }
            prev = p;
            p = p->next;
        }
    }
};

```

```

        } else {
            prev->next = p->next;
        }
        delete p;
        --size_;
        return *this;
    }
    prev = p;
    p = p->next;
}
return *this;
}

// 登録されている要素数を返す
int size() const {
    return size_;
}

// 全バケットの内容を表示 (デバッグ用)
void dump() const {
    for (int i = 0; i < HASHSIZE; ++i) {
        std::cout << " bucket[" << i << "]: ";
        Node* p = buckets[i];
        if (p == nullptr) {
            std::cout << "(empty)";
        }
        while (p != nullptr) {
            std::cout << "(" << p->key << " -> " << p->value << ")";
            if (p->next != nullptr) {
                std::cout << " -> ";
            }
            p = p->next;
        }
        std::cout << std::endl;
    }
}

private:
    Node* buckets_[HASHSIZE];
    int size_;
};

int main() {
    std::cout << "=== (a) HashTable<int> ===" << std::endl;
    HashTable<int> ht_int;
    ht_int.insert("takimoto", 42);
    ht_int.insert("katsurada", 122);
    ht_int.insert("matsuzawa", 35);
    ht_int.insert("ohmura", 12);
    std::cout << "insert 後の size: " << ht_int.size() << std::endl;
    ht_int.dump();

    const int* pi = ht_int("takimoto");
    std::cout << "find takimoto: " << (pi != nullptr ? std::to_string(*pi) : "none") << std::endl;
    pi = ht_int("katsurada");
    std::cout << "find katsurada: " << (pi != nullptr ? std::to_string(*pi) : "none") << std::endl;
    pi = ht_int("nonexistent");
    std::cout << "find nonexistent: " << (pi != nullptr ? std::to_string(*pi) : "none") << std::endl;

    ht_int -= "katsurada";
    std::cout << "katsurada 削除後の size: " << ht_int.size() << std::endl;
    pi = ht_int("katsurada");
    std::cout << "find katsurada after erase: " << (pi != nullptr ? std::to_string(*pi) : "none") << std::endl;

    std::cout << std::endl << "=== (b) HashTable<double> ===" << std::endl;
    HashTable<double> ht_double;
    ht_double.insert("takimoto", 172.5);
    ht_double.insert("katsurada", 168.2);
    ht_double.insert("matsuzawa", 175.0);
    std::cout << "insert 後の size: " << ht_double.size() << std::endl;
    const double* pd = ht_double("matsuzawa");
    std::cout << "find matsuzawa: " << (pd != nullptr ? std::to_string(*pd) : "none") << std::endl;
    pd = ht_double("ohmura");
    std::cout << "find ohmura: " << (pd != nullptr ? std::to_string(*pd) : "none") << std::endl;
}

```

```

std::cout << std::endl << "=== (c) HashTable<std::string> ===" << std::endl;
HashTable<std::string> ht_str;
ht_str.insert("greeting", "こんにちは");
ht_str.insert("subject", "情報科学演習");
ht_str.insert("name", "哈希表");
std::cout << "insert 後の size: " << ht_str.size() << std::endl;
const std::string* ps = ht_str("greeting");
std::cout << "find greeting: " << (ps != nullptr ? *ps : "none") << std::endl;
ps = ht_str("subject");
std::cout << "find subject: " << (ps != nullptr ? *ps : "none") << std::endl;
ps = ht_str("farewell");
std::cout << "find farewell: " << (ps != nullptr ? *ps : "none") << std::endl;

std::cout << std::endl << "=== (d) 存在しないキーと削除後の検索 ===" << std::endl;
HashTable<int> ht_check;
ht_check.insert("present", 99);
const int* pc = ht_check("absent");
std::cout << "find absent (before erase present): " << (pc != nullptr ? std::to_string(*pc) : "none") << std::endl;
ht_check -= "present";
pc = ht_check("present");
std::cout << "find present (after erase present): " << (pc != nullptr ? std::to_string(*pc) : "none") << std::endl;
pc = ht_check("absent");
std::cout << "find absent (after erase present): " << (pc != nullptr ? std::to_string(*pc) : "none") << std::endl;

return 0;
}

```

A.3 hashtable_improved.cpp (改良版)

```

#include <cmath>
#include <iostream>
#include <string>

// 動的バケット拡張と FNV-1a ハッシュを用いた改良版ハッシュ表
template <class T>
class HashTable {
public:
    // チェインのノード
    struct Node {
        std::string key;
        T value;
        Node* next;
    };

    // 初期バケット数で空のハッシュ表を作成
    HashTable() : buckets_(nullptr), bucket_count_(17), size_(0) {
        buckets_ = new Node*[bucket_count_];
        for (int i = 0; i < bucket_count_; ++i) {
            buckets_[i] = nullptr;
        }
    }

    // 全ノードとバケット配列を削除
    ~HashTable() {
        for (int i = 0; i < bucket_count_; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                delete p;
                p = next;
            }
        }
        delete[] buckets_;
    }

    // コピー禁止
    HashTable(const HashTable&) = delete;
    HashTable& operator=(const HashTable&) = delete;

    // FNV-1a 32bit ハッシュ値を計算
    static std::uint32_t fnv1a(const std::string& key) {

```

```

        std::uint32_t hashval = 2166136261u;
        for (unsigned char c : key) {
            hashval ^= c;
            hashval *= 16777619u;
        }
        return hashval;
    }

int hash(const std::string& key) const {
    return static_cast<int>(fnv1a(key) % static_cast<std::uint32_t>(bucket_count_));
}

// n が素数かどうか判定
static bool is_prime(int n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (int i = 3; i * i <= n; i += 2) {
        if (n % i == 0) return false;
    }
    return true;
}

// 現在のバケット数の 2 倍以上で最小の素数を返す
static int next_prime(int current) {
    int candidate = current * 2 + 1;
    while (!is_prime(candidate)) {
        candidate += 2;
    }
    return candidate;
}

// バケット数を拡張して全要素を再配置
void rehash(int new_bucket_count) {
    Node** new_buckets = new Node*[new_bucket_count];
    for (int i = 0; i < new_bucket_count; ++i) {
        new_buckets[i] = nullptr;
    }

    for (int i = 0; i < bucket_count_; ++i) {
        Node* p = buckets_[i];
        while (p != nullptr) {
            Node* next = p->next;
            int h = static_cast<int>(fnv1a(p->key) % static_cast<std::uint32_t>(new_bucket_count));
            p->next = new_buckets[h];
            new_buckets[h] = p;
            p = next;
        }
    }
    delete[] buckets_;
    buckets_ = new_buckets;
    bucket_count_ = new_bucket_count;
}

// 負荷率が 1.0 を超えたら自動拡張して先頭挿入
bool insert(const std::string& key, const T& value) {
    if (load_factor() > 1.0) {
        rehash(next_prime(bucket_count_));
    }
    int h = hash(key);
    Node* p = new Node{key, value, buckets_[h]};
    buckets_[h] = p;
    ++size_;
    return true;
}

// key に対応する値へのポインタを返す（見つからなければ nullptr）
const T* operator()(const std::string& key) const {
    int h = hash(key);
    Node* p = buckets_[h];
    while (p != nullptr) {
        if (p->key == key) {
            return &(p->value);
        }
    }
}

```

```

        p = p->next;
    }
    return nullptr;
}

// key を削除する (存在しなければ何もしない)
HashTable& operator-=(const std::string& key) {
    int h = hash(key);
    Node* p = buckets_[h];
    Node* prev = nullptr;
    while (p != nullptr) {
        if (p->key == key) {
            if (prev == nullptr) {
                buckets_[h] = p->next;
            } else {
                prev->next = p->next;
            }
            delete p;
            --size_;
            return *this;
        }
        prev = p;
        p = p->next;
    }
    return *this;
}

// 要素数を返す
int size() const {
    return size_;
}

// バケット数を返す
int bucket_count() const {
    return bucket_count_;
}

// 負荷率を返す
double load_factor() const {
    return static_cast<double>(size_) / static_cast<double>(bucket_count_);
}

// 全バケットの内容を表示 (デバッグ用、max_buckets >= 0 なら先頭のみ)
void dump(int max_buckets = -1) const {
    int limit = (max_buckets >= 0 && max_buckets < bucket_count_) ? max_buckets : bucket_count_;
    for (int i = 0; i < limit; ++i) {
        std::cout << "    bucket[" << i << "]: ";
        Node* p = buckets_[i];
        if (p == nullptr) {
            std::cout << "(empty)";
        }
        while (p != nullptr) {
            std::cout << "(" << p->key << " -> " << p->value << ")";
            if (p->next != nullptr) {
                std::cout << " -> ";
            }
            p = p->next;
        }
        std::cout << std::endl;
    }
    if (limit < bucket_count_) {
        std::cout << "    ... (残り " << (bucket_count_ - limit) << " バケットは省略)" << std::endl;
    }
}

private:
    Node** buckets_;
    int bucket_count_;
    int size_;
};

int main() {
    HashTable<int> ht;

```

```

std::cout << "=== 大量挿入前 ===" << std::endl;
std::cout << "bucket_count: " << ht.bucket_count() << ", size: " << ht.size()
    << ", load_factor: " << ht.load_factor() << std::endl;

std::cout << "=== key0000 ~ key0999 を 0 ~ 999 とともに挿入 ===" << std::endl;
int prev_bucket_count = ht.bucket_count();
for (int i = 0; i < 1000; ++i) {
    std::string key = "key";
    if (i < 10) {
        key += "000";
    } else if (i < 100) {
        key += "00";
    } else if (i < 1000) {
        key += "0";
    }
    key += std::to_string(i);
    ht.insert(key, i);
    if (ht.bucket_count() != prev_bucket_count) {
        std::cout << " 拡張: bucket_count " << prev_bucket_count << " -> " << ht.bucket_count()
            << " (size=" << ht.size() << ", load_factor=" << ht.load_factor() << ")" << std::endl;
        prev_bucket_count = ht.bucket_count();
    }
}

std::cout << "=== 挿入後 ===" << std::endl;
std::cout << "bucket_count: " << ht.bucket_count() << ", size: " << ht.size()
    << ", load_factor: " << ht.load_factor() << std::endl;

std::cout << "=== 検索テスト ===" << std::endl;
const int* p;
p = ht("key0000");
std::cout << "find key0000: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;
p = ht("key0123");
std::cout << "find key0123: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;
p = ht("key0999");
std::cout << "find key0999: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;
p = ht("key1000");
std::cout << "find key1000: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;

std::cout << "=== 削除テスト ===" << std::endl;
ht -= "key0123";
std::cout << "key0123 を削除しました" << std::endl;
p = ht("key0123");
std::cout << "find key0123 after erase: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;
std::cout << "size: " << ht.size() << ", load_factor: " << ht.load_factor() << std::endl;

std::cout << "=== 先頭 30 パケットの内容 (拡張後も整合性確認) ===" << std::endl;
ht.dump(30);

return 0;
}

```

A.4 exp_collision.cpp (実験 1: 衝突分布)

```

#include <cstdint>
#include <cstdlib>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

static int add_hash(const std::string& key, int m) {
    int h = 0;
    for (unsigned char c : key) {
        h += static_cast<int>(c);
    }
    return h % m;
}

static int djb2_hash(const std::string& key, int m) {
    unsigned long h = 5381UL;

```

```

    for (unsigned char c : key) {
        h = h * 33UL + static_cast<unsigned long>(c);
    }
    return static_cast<int>(h % static_cast<unsigned long>(m));
}

static std::uint32_t fnv1a_hash_raw(const std::string& key) {
    std::uint32_t h = 2166136261u;
    for (unsigned char c : key) {
        h ^= static_cast<std::uint32_t>(c);
        h *= 16777619u;
    }
    return h;
}

static int fnv1a_hash(const std::string& key, int m) {
    return static_cast<int>(fnv1a_hash_raw(key) % static_cast<std::uint32_t>(m));
}

static int std_hash(const std::string& key, int m) {
    std::size_t h = std::hash<std::string>{}(key);
    return static_cast<int>(h % static_cast<std::size_t>(m));
}

struct Stats {
    std::string name;
    std::vector<int> counts;
};

int main(int argc, char** argv) {
    if (argc != 3) {
        std::cerr << "Usage: " << argv[0] << " keyfile m\n";
        return 1;
    }

    const std::string keyfile = argv[1];
    const int m = std::atoi(argv[2]);
    if (m <= 0) {
        std::cerr << "m must be positive\n";
        return 1;
    }

    std::ifstream ifs(keyfile);
    if (!ifs) {
        std::cerr << "cannot open " << keyfile << "\n";
        return 1;
    }

    std::vector<std::string> keys;
    std::string line;
    while (std::getline(ifs, line)) {
        keys.push_back(line);
    }

    std::vector<Stats> stats = {
        {"add", std::vector<int>(m, 0)},
        {"djb2", std::vector<int>(m, 0)},
        {"fnv1a", std::vector<int>(m, 0)},
        {"stdhash", std::vector<int>(m, 0)},
    };

    for (const auto& key : keys) {
        ++stats[0].counts[add_hash(key, m)];
        ++stats[1].counts[djb2_hash(key, m)];
        ++stats[2].counts[fnv1a_hash(key, m)];
        ++stats[3].counts[std_hash(key, m)];
    }

    std::cout << "hash,bucket,count\n";
    for (const auto& s : stats) {
        for (int b = 0; b < m; ++b) {
            std::cout << s.name << ", " << b << ", " << s.counts[b] << "\n";
        }
    }
}

```

```

const double n = static_cast<double>(keys.size());
const double expected = n / static_cast<double>(m);

std::cerr << "-----\n";
std::cerr << "Hash collision distribution summary (m=" << m << ", n=" << keys.size() << ")\n";
std::cerr << "-----\n";
std::cerr << std::left << std::setw(10) << "function"
    << std::right << std::setw(8) << "empty"
    << std::setw(10) << "longest"
    << std::setw(18) << "avg(non-empty)"
    << std::setw(14) << "variance"
    << std::setw(14) << "chi2" << "\n";
std::cerr << "-----\n";

for (const auto& s : stats) {
    int empty = 0;
    int longest = 0;
    double sum = 0.0;
    double sumsq = 0.0;
    int nonempty = 0;

    for (int c : s.counts) {
        if (c == 0) ++empty;
        if (c > longest) longest = c;
        sum += static_cast<double>(c);
        sumsq += static_cast<double>(c) * static_cast<double>(c);
        if (c > 0) ++nonempty;
    }

    const double mean_all = sum / static_cast<double>(m);
    const double variance = sumsq / static_cast<double>(m) - mean_all * mean_all;
    const double mean_nonempty = (nonempty > 0) ? (sum / static_cast<double>(nonempty)) : 0.0;

    double chi2 = 0.0;
    if (expected > 0.0) {
        for (int c : s.counts) {
            const double diff = static_cast<double>(c) - expected;
            chi2 += diff * diff / expected;
        }
    }

    std::cerr << std::left << std::setw(10) << s.name
        << std::right << std::setw(8) << empty
        << std::setw(10) << longest
        << std::fixed << std::setprecision(2)
        << std::setw(18) << mean_nonempty
        << std::setw(14) << variance
        << std::setw(14) << chi2 << "\n";
}
std::cerr << "-----\n";

return 0;
}

```

A.5 exp_anagram.cpp (実験 2: アナグラム衝突)

```

#include <stdint>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

static int add_hash(const std::string& key) {
    int h = 0;
    for (unsigned char c : key) {
        h += static_cast<int>(c);
    }
    return h % 17;
}

static int fnv1a_hash(const std::string& key) {

```

```

        std::uint32_t h = 2166136261u;
        for (unsigned char c : key) {
            h ^= static_cast<std::uint32_t>(c);
            h *= 16777619u;
        }
        return static_cast<int>(h % 17u);
    }

int main() {
    const std::vector<std::vector<std::string>> groups = {
        {"abc", "acb", "bac", "bca", "cab", "cba"},
        {"stop", "tops", "pots", "spot", "opts", "post"},
        {"listen", "silent", "enlist", "tinsel"},
    };

    std::cout << std::left << std::setw(14) << "key"
               << std::right << std::setw(16) << "add_hash(m=17)"
               << std::setw(20) << "fnv1a_hash(m=17)" << "\n";
    std::cout << std::string(50, '-') << "\n";

    for (const auto& group : groups) {
        for (const auto& key : group) {
            std::cout << std::left << std::setw(14) << key
                      << std::right << std::setw(16) << add_hash(key)
                      << std::setw(20) << fnv1a_hash(key) << "\n";
        }
        std::cout << std::string(50, '-') << "\n";
    }

    std::cout << "[まとめ] すべてのグループで add は全キーが同一バケットになり、"
               << "fnv1a は異なるバケットに分散している。\\n";

    return 0;
}

```

A.6 exp_loadfactor.cpp (実験 3: 負荷率と比較回数)

```

#include <cstdint>
#include <fstream>
#include <iomanip>
#include <cstdlib>
#include <iostream>
#include <string>
#include <vector>

static constexpr int M = 17;

struct AddHash {
    static int hash(const std::string& key) {
        int h = 0;
        for (unsigned char c : key) {
            h += static_cast<int>(c);
        }
        return h % M;
    }
};

struct FnvHash {
    static int hash(const std::string& key) {
        std::uint32_t h = 2166136261u;
        for (unsigned char c : key) {
            h ^= static_cast<std::uint32_t>(c);
            h *= 16777619u;
        }
        return static_cast<int>(h % static_cast<std::uint32_t>(M));
    }
};

template <typename Hasher>
class ChainTable {
public:
    struct Node {

```

```

        std::string key;
        int value;
        Node* next;
    };

    ChainTable() : size_(0) {
        for (int i = 0; i < M; ++i) {
            buckets_[i] = nullptr;
        }
    }

    ~ChainTable() {
        clear();
    }

    ChainTable(const ChainTable&) = delete;
    ChainTable& operator=(const ChainTable&) = delete;

    void insert(const std::string& key, int value) {
        const int h = Hasher::hash(key);
        buckets_[h] = new Node{key, value, buckets_[h]};
        ++size_;
    }

    // 探索に必要なだったキー比較回数を返す。found に成否を書き込む。
    int search_comparisons(const std::string& key, bool& found) const {
        const int h = Hasher::hash(key);
        int cmp = 0;
        for (Node* p = buckets_[h]; p != nullptr; p = p->next) {
            ++cmp;
            if (p->key == key) {
                found = true;
                return cmp;
            }
        }
        found = false;
        return cmp;
    }

    void clear() {
        for (int i = 0; i < M; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                delete p;
                p = next;
            }
            buckets_[i] = nullptr;
        }
        size_ = 0;
    }

private:
    Node* buckets_[M];
    int size_;
};

template <typename Hasher>
static void run_experiment(const std::vector<std::string>& keys, const std::string& name) {
    const std::vector<int> ns = {8, 17, 34, 51, 85, 170, 340, 510, 850, 1700};

    for (int n : ns) {
        if (n > static_cast<int>(keys.size())) {
            continue;
        }

        ChainTable<Hasher> table;
        for (int i = 0; i < n; ++i) {
            table.insert(keys[i], i);
        }

        long long succ_sum = 0;
        for (int i = 0; i < n; ++i) {
            bool found = false;

```

```

        succ_sum += table.search_comparisons(keys[i], found);
    }
    const double succ_avg = static_cast<double>(succ_sum) / static_cast<double>(n);

    long long unsucc_sum = 0;
    // 不変条件: 不成功検索用のキー (末尾 n 個) が挿入済みキー (先頭 n 個) と
    // 重ならないよう n <= keys.size()/2 を要求する
    if (static_cast<std::size_t>(n) > keys.size() / 2) {
        std::cerr << "error: n=" << n << " exceeds half of key file; "
            << "unsuccessful-search keys would overlap inserted keys\n";
        std::exit(1);
    }
    const std::size_t start = keys.size() - static_cast<std::size_t>(n);
    for (std::size_t i = start; i < keys.size(); ++i) {
        bool found = false;
        unsucc_sum += table.search_comparisons(keys[i], found);
    }
    const double unsucc_avg = static_cast<double>(unsucc_sum) / static_cast<double>(n);
    const double alpha = static_cast<double>(n) / static_cast<double>(M);

    std::cout << name << ", " << n << ", "
        << std::fixed << std::setprecision(6) << alpha << ", "
        << std::fixed << std::setprecision(6) << succ_avg << ", "
        << std::fixed << std::setprecision(6) << unsucc_avg << "\n";
    }
}

int main(int argc, char** argv) {
    if (argc != 2) {
        std::cerr << "Usage: " << argv[0] << " keyfile\n";
        return 1;
    }

    std::ifstream ifs(argv[1]);
    if (!ifs) {
        std::cerr << "cannot open " << argv[1] << "\n";
        return 1;
    }

    std::vector<std::string> keys;
    std::string line;
    while (std::getline(ifs, line)) {
        keys.push_back(line);
    }

    std::cout << "hash,n,alpha,succ_avg,unsucc_avg\n";
    run_experiment<AddHash>(keys, "add");
    run_experiment<FnvHash>(keys, "fnv1a");

    return 0;
}

```

A.7 exp_timing.cpp (実験 4: 実行時間)

```

#include <chrono>
#include <stdint>
#include <cstdlib>
#include <iomanip>
#include <iostream>
#include <map>
#include <memory>
#include <random>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <vector>

// =====
// fixed17: 加算ハッシュ、バケット数 17 固定のチェイン法
// =====
class Fixed17Table {
public:

```

```

struct Node {
    std::string key;
    int value;
    Node* next;
};

Fixed17Table() : size_(0) {
    for (int i = 0; i < 17; ++i) {
        buckets_[i] = nullptr;
    }
}

~Fixed17Table() {
    clear();
}

Fixed17Table(const Fixed17Table&) = delete;
Fixed17Table& operator=(const Fixed17Table&) = delete;

void insert(const std::string& key, int value) {
    const int h = hash_(key);
    buckets_[h] = new Node{key, value, buckets_[h]};
    ++size_;
}

bool find(const std::string& key) const {
    const int h = hash_(key);
    for (Node* p = buckets_[h]; p != nullptr; p = p->next) {
        if (p->key == key) {
            return true;
        }
    }
    return false;
}

void clear() {
    for (int i = 0; i < 17; ++i) {
        Node* p = buckets_[i];
        while (p != nullptr) {
            Node* next = p->next;
            delete p;
            p = next;
        }
        buckets_[i] = nullptr;
    }
    size_ = 0;
}

private:
    static int hash_(const std::string& key) {
        int h = 0;
        for (unsigned char c : key) {
            h += static_cast<int>(c);
        }
        return h % 17;
    }

    Node* buckets_[17];
    int size_;
};

// =====
// improved: FNV-1a + 負荷率 1.0 超で素数リハッシュ
// =====
class ImprovedTable {
public:
    struct Node {
        std::string key;
        int value;
        Node* next;
    };

    ImprovedTable() : buckets_(nullptr), bucket_count_(17), size_(0) {
        buckets_ = new Node*[bucket_count_];
    }
};

```

```

        for (int i = 0; i < bucket_count_; ++i) {
            buckets_[i] = nullptr;
        }
    }

    ~ImprovedTable() {
        clear();
        delete[] buckets_;
    }

    ImprovedTable(const ImprovedTable&) = delete;
    ImprovedTable& operator=(const ImprovedTable&) = delete;

    void insert(const std::string& key, int value) {
        if (load_factor() > 1.0) {
            rehash(next_prime_(bucket_count_));
        }
        const int h = hash_(key);
        buckets_[h] = new Node{key, value, buckets_[h]};
        ++size_;
    }

    bool find(const std::string& key) const {
        const int h = hash_(key);
        for (Node* p = buckets_[h]; p != nullptr; p = p->next) {
            if (p->key == key) {
                return true;
            }
        }
        return false;
    }

    void clear() {
        for (int i = 0; i < bucket_count_; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                delete p;
                p = next;
            }
            buckets_[i] = nullptr;
        }
        size_ = 0;
    }

private:
    static std::uint32_t fnv1a_(const std::string& key) {
        std::uint32_t h = 2166136261u;
        for (unsigned char c : key) {
            h ^= static_cast<std::uint32_t>(c);
            h *= 16777619u;
        }
        return h;
    }

    int hash_(const std::string& key) const {
        return static_cast<int>(fnv1a_(key) % static_cast<std::uint32_t>(bucket_count_));
    }

    static bool is_prime_(int n) {
        if (n < 2) return false;
        if (n == 2) return true;
        if (n % 2 == 0) return false;
        for (int i = 3; i * i <= n; i += 2) {
            if (n % i == 0) return false;
        }
        return true;
    }

    static int next_prime_(int current) {
        int candidate = current * 2 + 1;
        while (!is_prime_(candidate)) {
            candidate += 2;
        }
    }

```

```

        return candidate;
    }

    void rehash(int new_bucket_count) {
        Node** new_buckets = new Node*[new_bucket_count];
        for (int i = 0; i < new_bucket_count; ++i) {
            new_buckets[i] = nullptr;
        }
        for (int i = 0; i < bucket_count_; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                const int h = static_cast<int>(
                    fnv1a_(p->key) % static_cast<std::uint32_t>(new_bucket_count));
                p->next = new_buckets[h];
                new_buckets[h] = p;
                p = next;
            }
        }
        delete[] buckets_;
        buckets_ = new_buckets;
        bucket_count_ = new_bucket_count;
    }

    double load_factor() const {
        return static_cast<double>(size_) / static_cast<double>(bucket_count_);
    }

    Node** buckets_;
    int bucket_count_;
    int size_;
};

// =====
// 共通ラッパー
// =====
struct WrapperBase {
    virtual ~WrapperBase() = default;
    virtual void insert(const std::string& key, int value) = 0;
    virtual bool find(const std::string& key) = 0;
};

template <class Table>
struct GenericWrapper : public WrapperBase {
    Table table;
    void insert(const std::string& key, int value) override {
        table.insert(key, value);
    }
    bool find(const std::string& key) override {
        return table.find(key);
    }
};

struct UnorderedMapWrapper : public WrapperBase {
    std::unordered_map<std::string, int> table;
    void insert(const std::string& key, int value) override {
        table.emplace(key, value);
    }
    bool find(const std::string& key) override {
        return table.find(key) != table.end();
    }
};

struct MapWrapper : public WrapperBase {
    std::map<std::string, int> table;
    void insert(const std::string& key, int value) override {
        table.emplace(key, value);
    }
    bool find(const std::string& key) override {
        return table.find(key) != table.end();
    }
};

struct Case {

```

```

        std::string name;
        bool skip_if_too_slow;
        std::unique_ptr<WrapperBase> (*create)();
    };

    // =====
    // キー生成
    // =====
    static std::string generate_key(std::mt19937& rng) {
        static const char chars[] =
            "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
        static const int L = 62;
        std::uniform_int_distribution<int> dist(0, L - 1);

        std::string key(12, ' ');
        for (int i = 0; i < 12; ++i) {
            key[i] = chars[dist(rng)];
        }
        return key;
    }

    static std::vector<std::string> generate_unique_keys(int N) {
        std::mt19937 rng(42);
        std::unordered_set<std::string> seen;
        seen.reserve(static_cast<std::size_t>(N) * 2);
        std::vector<std::string> keys;
        keys.reserve(static_cast<std::size_t>(N));

        while (static_cast<int>(keys.size()) < N) {
            const std::string k = generate_key(rng);
            if (seen.insert(k).second) {
                keys.push_back(k);
            }
        }
        return keys;
    }

    static std::vector<std::string> generate_lookups(
        const std::vector<std::string>& keys, int count) {
        std::mt19937 rng(42);
        std::uniform_int_distribution<int> dist(0, static_cast<int>(keys.size()) - 1);
        std::vector<std::string> lookups;
        lookups.reserve(static_cast<std::size_t>(count));
        for (int i = 0; i < count; ++i) {
            lookups.push_back(keys[dist(rng)]);
        }
        return lookups;
    }

    // =====
    // メイン
    // =====
    int main() {
        const std::vector<int> Ns = {
            1000, 3162, 10000, 31623, 100000, 316228, 1000000
        };
        const int lookup_count = 100000;

        const std::vector<Case> cases = {
            {"fixed17", true, []() -> std::unique_ptr<WrapperBase> {
                return std::make_unique<GenericWrapper<Fixed17Table>>();
            }},
            {"improved", false, []() -> std::unique_ptr<WrapperBase> {
                return std::make_unique<GenericWrapper<ImprovedTable>>();
            }},
            {"unordered_map", false, []() -> std::unique_ptr<WrapperBase> {
                return std::make_unique<UnorderedMapWrapper>();
            }},
            {"map", false, []() -> std::unique_ptr<WrapperBase> {
                return std::make_unique<MapWrapper>();
            }},
        };

        std::cout << "structure,N,rep,insert_total_us,lookup_avg_ns\n";
    }

```

```

using clock = std::chrono::steady_clock;

for (int N : Ns) {
    const auto keys = generate_unique_keys(N);
    const auto lookups = generate_lookups(keys, lookup_count);

    for (const auto& c : cases) {
        if (c.skip_if_too_slow && N > 100000) {
            std::cerr << "skip " << c.name << " for N=" << N << " (too slow)\n";
            continue;
        }
        for (int rep = 1; rep <= 5; ++rep) {
            std::cerr << "Running " << c.name << " N=" << N << " rep=" << rep << "\n";
            auto table = c.create();

            const auto t0 = clock::now();
            for (int i = 0; i < N; ++i) {
                table->insert(keys[i], i);
            }
            const auto t1 = clock::now();

            long long found = 0;
            const auto t2 = clock::now();
            for (const auto& k : lookups) {
                found += table->find(k) ? 1 : 0;
            }
            const auto t3 = clock::now();
            std::cerr << " found=" << found << "/" << lookup_count << "\n";

            const double insert_us = static_cast<double>(
                std::chrono::duration_cast<std::chrono::microseconds>(t1 - t0).count());
            const double lookup_total_ns = static_cast<double>(
                std::chrono::duration_cast<std::chrono::nanoseconds>(t3 - t2).count());
            const double lookup_avg_ns = lookup_total_ns / static_cast<double>(lookup_count);

            std::cout << c.name << ", " << N << ", " << rep << ", "
                << std::fixed << std::setprecision(1) << insert_us << ", "
                << std::fixed << std::setprecision(1) << lookup_avg_ns << "\n";
        }
    }
}

return 0;
}

```

A.8 exp_memory.cpp (実験 5: メモリ使用量)

```

#include <cerrno>
#include <cstdint>
#include <cstdlib>
#include <cstring>
#include <iostream>
#include <map>
#include <random>
#include <string>
#include <sys/resource.h>
#include <unordered_map>
#include <vector>

// =====
// improved: FNV-1a + 負荷率 1.0 超で素数リハッシュ
// =====
class ImprovedTable {
public:
    struct Node {
        std::string key;
        int value;
        Node* next;
    };

    ImprovedTable() : buckets_(nullptr), bucket_count_(17), size_(0) {

```

```

        buckets_ = new Node*[bucket_count_];
        for (int i = 0; i < bucket_count_; ++i) {
            buckets_[i] = nullptr;
        }
    }

    ~ImprovedTable() {
        clear();
        delete[] buckets_;
    }

    ImprovedTable(const ImprovedTable&) = delete;
    ImprovedTable& operator=(const ImprovedTable&) = delete;

    void insert(const std::string& key, int value) {
        if (load_factor() > 1.0) {
            rehash(next_prime_(bucket_count_));
        }
        const int h = hash_(key);
        buckets_[h] = new Node{key, value, buckets_[h]};
        ++size_;
    }

    bool find(const std::string& key) const {
        const int h = hash_(key);
        for (Node* p = buckets_[h]; p != nullptr; p = p->next) {
            if (p->key == key) {
                return true;
            }
        }
        return false;
    }

    void clear() {
        for (int i = 0; i < bucket_count_; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                delete p;
                p = next;
            }
            buckets_[i] = nullptr;
        }
        size_ = 0;
    }

    static std::size_t node_size() {
        return sizeof(Node);
    }

private:
    static std::uint32_t fnv1a_(const std::string& key) {
        std::uint32_t h = 2166136261u;
        for (unsigned char c : key) {
            h ^= static_cast<std::uint32_t>(c);
            h *= 16777619u;
        }
        return h;
    }

    int hash_(const std::string& key) const {
        return static_cast<int>(fnv1a_(key) % static_cast<std::uint32_t>(bucket_count_));
    }

    static bool is_prime_(int n) {
        if (n < 2) return false;
        if (n == 2) return true;
        if (n % 2 == 0) return false;
        for (int i = 3; i * i <= n; i += 2) {
            if (n % i == 0) return false;
        }
        return true;
    }
}

```

```

static int next_prime_(int current) {
    int candidate = current * 2 + 1;
    while (!is_prime_(candidate)) {
        candidate += 2;
    }
    return candidate;
}

void rehash(int new_bucket_count) {
    Node** new_buckets = new Node*[new_bucket_count];
    for (int i = 0; i < new_bucket_count; ++i) {
        new_buckets[i] = nullptr;
    }
    for (int i = 0; i < bucket_count_; ++i) {
        Node* p = buckets_[i];
        while (p != nullptr) {
            Node* next = p->next;
            const int h = static_cast<int>(
                fnv1a_(p->key) % static_cast<std::uint32_t>(new_bucket_count));
            p->next = new_buckets[h];
            new_buckets[h] = p;
            p = next;
        }
    }
    delete[] buckets_;
    buckets_ = new_buckets;
    bucket_count_ = new_bucket_count;
}

double load_factor() const {
    return static_cast<double>(size_) / static_cast<double>(bucket_count_);
}

Node** buckets_;
int bucket_count_;
int size_;
};

// =====
// キー生成
// =====
static std::string generate_key(std::mt19937& rng) {
    static const char chars[] =
        "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
    static const int L = 62;
    std::uniform_int_distribution<int> dist(0, L - 1);

    std::string key(12, ' ');
    for (int i = 0; i < 12; ++i) {
        key[i] = chars[dist(rng)];
    }
    return key;
}

static std::vector<std::string> generate_unique_keys(int N) {
    std::mt19937 rng(42);
    std::vector<std::string> keys;
    keys.reserve(static_cast<std::size_t>(N));

    while (static_cast<int>(keys.size()) < N) {
        keys.push_back(generate_key(rng));
    }
    return keys;
}

// =====
// RSS 取得 (macOS では ru_maxrss がバイト単位)
// =====
static long long get_max_rss_bytes() {
    struct rusage usage;
    if (getrusage(RUSAGE_SELF, &usage) != 0) {
        std::cerr << "getrusage failed: " << std::strerror(errno) << "\n";
        std::exit(1);
    }
}

```

```

    return static_cast<long long>(usage.ru_maxrss);
}

// =====
// メイン
// =====
int main(int argc, char** argv) {
    if (argc != 3) {
        std::cerr << "Usage: " << argv[0] << " <structure> <N>\n";
        std::cerr << " structure: improved | unordered_map | map\n";
        return 1;
    }

    const std::string structure = argv[1];
    const int N = std::atoi(argv[2]);
    if (N <= 0) {
        std::cerr << "N must be positive\n";
        return 1;
    }

    std::cerr << "sizeof(std::string)=" << sizeof(std::string)
        << ", sizeof(ImprovedTable::Node)=" << ImprovedTable::node_size() << "\n";

    // キーを先にすべて生成してから RSS を測定
    std::cerr << "Generating " << N << " keys...\n";
    const auto keys = generate_unique_keys(N);

    const long long rss_before = get_max_rss_bytes();
    std::cerr << "rss_before=" << rss_before << " bytes\n";

    if (structure == "improved") {
        ImprovedTable table;
        for (int i = 0; i < N; ++i) {
            table.insert(keys[i], i);
        }
    } else if (structure == "unordered_map") {
        std::unordered_map<std::string, int> table;
        table.reserve(static_cast<std::size_t>(N));
        for (int i = 0; i < N; ++i) {
            table.emplace(keys[i], i);
        }
    } else if (structure == "map") {
        std::map<std::string, int> table;
        for (int i = 0; i < N; ++i) {
            table.emplace(keys[i], i);
        }
    } else {
        std::cerr << "unknown structure: " << structure << "\n";
        return 1;
    }

    const long long rss_after = get_max_rss_bytes();
    const long long delta = rss_after - rss_before;

    std::cout << structure << ", " << N << ", "
        << rss_before << ", " << rss_after << ", " << delta << "\n";

    return 0;
}

```

A.9 exp_tablesize.cpp (実験 6: 表サイズの影響)

```

#include <cmath>
#include <cstdint>
#include <cstdlib>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

static int add_hash(const std::string& key, int m) {

```

```

    int h = 0;
    for (unsigned char c : key) {
        h += static_cast<int>(c);
    }
    return h % m;
}

static std::uint32_t fnv1a_hash_raw(const std::string& key) {
    std::uint32_t h = 2166136261u;
    for (unsigned char c : key) {
        h ^= static_cast<std::uint32_t>(c);
        h *= 16777619u;
    }
    return h;
}

static int fnv1a_hash(const std::string& key, int m) {
    return static_cast<int>(fnv1a_hash_raw(key) % static_cast<std::uint32_t>(m));
}

struct Stats {
    std::string name;
    std::vector<int> counts;
};

int main(int argc, char** argv) {
    if (argc != 2) {
        std::cerr << "Usage: " << argv[0] << " <keyfile>\n";
        return 1;
    }

    const std::string keyfile = argv[1];
    std::ifstream ifs(keyfile);
    if (!ifs) {
        std::cerr << "cannot open " << keyfile << "\n";
        return 1;
    }

    std::vector<std::string> keys;
    std::string line;
    while (std::getline(ifs, line)) {
        keys.push_back(line);
    }

    const int n = static_cast<int>(keys.size());
    const std::vector<int> ms = {16, 17, 256, 257, 1024, 1031};

    std::cout << "hash,m,n,empty,longest,mean_nonempty,variance,chi2\n";

    for (int m : ms) {
        std::vector<Stats> stats = {
            {"add", std::vector<int>(m, 0)},
            {"fnv1a", std::vector<int>(m, 0)},
        };

        for (const auto& key : keys) {
            ++stats[0].counts[add_hash(key, m)];
            ++stats[1].counts[fnv1a_hash(key, m)];
        }

        const double expected = static_cast<double>(n) / static_cast<double>(m);

        for (const auto& s : stats) {
            int empty = 0;
            int longest = 0;
            double sum = 0.0;
            double sumsq = 0.0;
            int nonempty = 0;

            for (int c : s.counts) {
                if (c == 0) ++empty;
                if (c > longest) longest = c;
                sum += static_cast<double>(c);
                sumsq += static_cast<double>(c) * static_cast<double>(c);
            }
        }
    }
}

```

```

        if (c > 0) ++nonempty;
    }

    const double mean_all = sum / static_cast<double>(m);
    const double variance = sumsq / static_cast<double>(m) - mean_all * mean_all;
    const double mean_nonempty = (nonempty > 0) ? (sum / static_cast<double>(nonempty)) : 0.0;

    double chi2 = 0.0;
    if (expected > 0.0) {
        for (int c : s.counts) {
            const double diff = static_cast<double>(c) - expected;
            chi2 += diff * diff / expected;
        }
    }

    std::cout << s.name << ", " << m << ", " << n << ", "
                << empty << ", " << longest << ", "
                << std::fixed << std::setprecision(2) << mean_nonempty << ", "
                << std::fixed << std::setprecision(2) << variance << ", "
                << std::fixed << std::setprecision(2) << chi2 << "\n";
    }
}

return 0;
}

```

A.10 exp_rehash.cpp (実験 7: リハッシュの償却コスト)

```

#include <algorithm>
#include <chrono>
#include <cstdlib>
#include <cstdliblib>
#include <iostream>
#include <random>
#include <string>
#include <unordered_set>
#include <vector>

// =====
// improved: FNV-1a + 負荷率 1.0 超で素数リハッシュ
// =====
class ImprovedTable {
public:
    struct Node {
        std::string key;
        int value;
        Node* next;
    };

    ImprovedTable() : buckets_(nullptr), bucket_count_(17), size_(0) {
        buckets_ = new Node*[bucket_count_];
        for (int i = 0; i < bucket_count_; ++i) {
            buckets_[i] = nullptr;
        }
    }

    ~ImprovedTable() {
        clear();
        delete[] buckets_;
    }

    ImprovedTable(const ImprovedTable&) = delete;
    ImprovedTable& operator=(const ImprovedTable&) = delete;

    void insert(const std::string& key, int value) {
        if (load_factor() > 1.0) {
            rehash(next_prime_(bucket_count_));
        }
        const int h = hash_(key);
        buckets_[h] = new Node{key, value, buckets_[h]};
        ++size_;
    }
}

```

```

bool find(const std::string& key) const {
    const int h = hash_(key);
    for (Node* p = buckets_[h]; p != nullptr; p = p->next) {
        if (p->key == key) {
            return true;
        }
    }
    return false;
}

void clear() {
    for (int i = 0; i < bucket_count_; ++i) {
        Node* p = buckets_[i];
        while (p != nullptr) {
            Node* next = p->next;
            delete p;
            p = next;
        }
        buckets_[i] = nullptr;
    }
    size_ = 0;
}

int bucket_count() const {
    return bucket_count_;
}

int size() const {
    return size_;
}

private:
static std::uint32_t fnv1a_(const std::string& key) {
    std::uint32_t h = 2166136261u;
    for (unsigned char c : key) {
        h ^= static_cast<std::uint32_t>(c);
        h *= 16777619u;
    }
    return h;
}

int hash_(const std::string& key) const {
    return static_cast<int>(fnv1a_(key) % static_cast<std::uint32_t>(bucket_count_));
}

static bool is_prime_(int n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (int i = 3; i * i <= n; i += 2) {
        if (n % i == 0) return false;
    }
    return true;
}

static int next_prime_(int current) {
    int candidate = current * 2 + 1;
    while (!is_prime_(candidate)) {
        candidate += 2;
    }
    return candidate;
}

void rehash(int new_bucket_count) {
    Node** new_buckets = new Node*[new_bucket_count];
    for (int i = 0; i < new_bucket_count; ++i) {
        new_buckets[i] = nullptr;
    }
    for (int i = 0; i < bucket_count_; ++i) {
        Node* p = buckets_[i];
        while (p != nullptr) {
            Node* next = p->next;
            const int h = static_cast<int>(

```

```

        fnvia_(p->key) % static_cast<std::uint32_t>(new_bucket_count));
        p->next = new_buckets[h];
        new_buckets[h] = p;
        p = next;
    }
}
delete[] buckets_;
buckets_ = new_buckets;
bucket_count_ = new_bucket_count;
}

double load_factor() const {
    return static_cast<double>(size_) / static_cast<double>(bucket_count_);
}

Node** buckets_;
int bucket_count_;
int size_;
};

// =====
// キー生成
// =====
static std::string generate_key(std::mt19937& rng) {
    static const char chars[] =
        "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
    static const int L = 62;
    std::uniform_int_distribution<int> dist(0, L - 1);

    std::string key(12, ' ');
    for (int i = 0; i < 12; ++i) {
        key[i] = chars[dist(rng)];
    }
    return key;
}

static std::vector<std::string> generate_unique_keys(int N) {
    std::mt19937 rng(42);
    std::unordered_set<std::string> seen;
    seen.reserve(static_cast<std::size_t>(N) * 2);
    std::vector<std::string> keys;
    keys.reserve(static_cast<std::size_t>(N));

    while (static_cast<int>(keys.size()) < N) {
        const std::string k = generate_key(rng);
        if (seen.insert(k).second) {
            keys.push_back(k);
        }
    }
    return keys;
}

// =====
// メイン
// =====
int main() {
    const int N = 200000;
    const long long spike_threshold_ns = 50000;

    std::cerr << "Generating " << N << " unique keys...\n";
    const auto keys = generate_unique_keys(N);

    ImprovedTable table;
    using clock = std::chrono::steady_clock;

    std::cout << "kind,i,value_ns\n";

    long long total_ns = 0;
    long long max_ns = 0;
    int rehash_count = 0;
    int prev_bucket_count = table.bucket_count();

    for (int i = 0; i < N; ++i) {
        if (i % 10000 == 0) {

```

```

        std::cerr << "progress: " << i << "/" << N << "\n";
    }

    const auto t0 = clock::now();
    table.insert(keys[i], i);
    const auto t1 = clock::now();

    const long long ns = std::chrono::duration_cast<std::chrono::nanoseconds>(
        t1 - t0).count();
    total_ns += ns;
    if (ns > max_ns) {
        max_ns = ns;
    }

    if (table.bucket_count() != prev_bucket_count) {
        std::cout << "rehash," << i << ", " << table.bucket_count() << "\n";
        prev_bucket_count = table.bucket_count();
        ++rehash_count;
    }

    if (i % 100 == 0) {
        std::cout << "sample," << i << ", " << ns << "\n";
    }

    if (ns > spike_threshold_ns) {
        std::cout << "spike," << i << ", " << ns << "\n";
    }

    if (i % 1000 == 0) {
        const long long cumavg = total_ns / static_cast<long long>(i + 1);
        std::cout << "cumavg," << i << ", " << cumavg << "\n";
    }
}

const double avg_ns = static_cast<double>(total_ns) / static_cast<double>(N);

std::cerr << "-----\n";
std::cerr << "Final stats (N=" << N << ") \n";
std::cerr << "  total insert time: " << total_ns << " ns\n";
std::cerr << "  average latency:   " << avg_ns << " ns\n";
std::cerr << "  max latency:           " << max_ns << " ns\n";
std::cerr << "  rehash count:         " << rehash_count << "\n";
std::cerr << "  final bucket_count:" << table.bucket_count() << "\n";
std::cerr << "  final load_factor:    "
    << static_cast<double>(table.size()) / static_cast<double>(table.bucket_count())
    << "\n";
std::cerr << "-----\n";

return 0;
}

```

A.11 exp_openaddr.cpp (実験 8: 開放番地法との比較)

```

#include <algorithm>
#include <chrono>
#include <stdint>
#include <iostream>
#include <random>
#include <string>
#include <vector>

// FNV-1a 64-bit hash
static std::uint64_t fnv1a(const std::string& key) {
    std::uint64_t h = 0xcbf29ce484222325ULL;
    for (unsigned char c : key) {
        h ^= c;
        h *= 0x100000001b3ULL;
    }
    return h;
}

static std::string random_key(std::mt19937& rng, int len) {

```

```

static const char alnum[] =
    "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    "abcdefghijklmnopqrstuvwxyz"
    "0123456789";
std::uniform_int_distribution<int> dist(0, 61);
std::string s;
s.reserve(len);
for (int i = 0; i < len; ++i) {
    s.push_back(alnum[dist(rng)]);
}
return s;
}

// ----- chained hashing -----
class ChainedTable {
    struct Node {
        std::string key;
        Node* next;
    };
    std::vector<Node*> buckets_;

public:
    explicit ChainedTable(std::size_t cap) : buckets_(cap, nullptr) {}

    ~ChainedTable() {
        for (Node* head : buckets_) {
            while (head) {
                Node* n = head;
                head = head->next;
                delete n;
            }
        }
    }

    std::size_t capacity() const { return buckets_.size(); }

    void insert(const std::string& key) {
        std::size_t h = fnv1a(key) & (buckets_.size() - 1);
        Node* n = new Node{key, buckets_[h]};
        buckets_[h] = n;
    }

    // return number of probes (bucket reference + key comparisons)
    int find(const std::string& key) const {
        std::size_t h = fnv1a(key) & (buckets_.size() - 1);
        int probes = 1; // bucket reference
        for (Node* p = buckets_[h]; p; p = p->next) {
            if (p->key == key) {
                return probes;
            }
            ++probes; // key comparison that failed
        }
        return probes;
    }
};

// ----- open addressing with linear probing -----
class OpenAddrTable {
    enum State : std::uint8_t { EMPTY = 0, USED = 1, DELETED = 2 };
    struct Slot {
        std::string key;
        State state = EMPTY;
    };
    std::vector<Slot> slots_;
    std::size_t mask_;

public:
    explicit OpenAddrTable(std::size_t cap) : slots_(cap), mask_(cap - 1) {}

    std::size_t capacity() const { return slots_.size(); }

    void insert(const std::string& key) {
        std::size_t i = fnv1a(key) & mask_;
        while (slots_[i].state == USED) {

```

```

        if (slots_[i].key == key) {
            return; // already present; not expected in this experiment
        }
        i = (i + 1) & mask_;
    }
    slots_[i].key = key;
    slots_[i].state = USED;
}

// successful search: probe count = slots examined until match
int find_success(const std::string& key) const {
    std::size_t i = fnv1a(key) & mask_;
    int probes = 0;
    for (;;) {
        ++probes;
        if (slots_[i].state == EMPTY) {
            return probes; // not found, but this path is not used for success
        }
        if (slots_[i].state == USED && slots_[i].key == key) {
            return probes;
        }
        i = (i + 1) & mask_;
    }
}

// unsuccessful search: probe count until EMPTY slot is seen
int find_unsucc(const std::string& key) const {
    std::size_t i = fnv1a(key) & mask_;
    int probes = 0;
    for (;;) {
        ++probes;
        if (slots_[i].state == EMPTY) {
            return probes;
        }
        if (slots_[i].state == USED && slots_[i].key == key) {
            return probes; // found, not expected for unsucc set
        }
        i = (i + 1) & mask_;
    }
}

};

int main() {
    constexpr std::size_t CAPACITY = 1ULL << 17; // 131072
    constexpr int KEY_LEN = 12;

    std::mt19937 rng(42);

    const std::vector<double> alphas = {0.25, 0.5, 0.7, 0.8, 0.9, 0.95};

    std::cout << "method,alpha,n,succ_probes,unsucc_probes,succ_ns,unsucc_ns\n";

    for (double alpha : alphas) {
        std::size_t n = static_cast<std::size_t>(std::llround(alpha * CAPACITY));

        // generate distinct inserted keys and distinct non-inserted keys
        std::vector<std::string> inserted;
        inserted.reserve(n);
        while (inserted.size() < n) {
            std::string k = random_key(rng, KEY_LEN);
            inserted.push_back(std::move(k));
        }

        std::vector<std::string> not_inserted;
        not_inserted.reserve(n);
        while (not_inserted.size() < n) {
            std::string k = random_key(rng, KEY_LEN);
            not_inserted.push_back(std::move(k));
        }

        // ---- chained ----
        {
            ChainedTable ct(CAPACITY);
            for (const auto& k : inserted) {

```

```

        ct.insert(k);
    }

    long long succ_found = 0;
    auto t0 = std::chrono::high_resolution_clock::now();
    double succ_probes = 0.0;
    for (const auto& k : inserted) {
        int p = ct.find(k);
        succ_probes += p;
        ++succ_found;
    }
    auto t1 = std::chrono::high_resolution_clock::now();
    double succ_ns = static_cast<double>(
        std::chrono::duration_cast<std::chrono::nanoseconds>(t1 - t0).count()) / n;

    auto t2 = std::chrono::high_resolution_clock::now();
    double unsucc_probes = 0.0;
    for (const auto& k : not_inserted) {
        unsucc_probes += ct.find(k);
    }
    auto t3 = std::chrono::high_resolution_clock::now();
    double unsucc_ns = static_cast<double>(
        std::chrono::duration_cast<std::chrono::nanoseconds>(t3 - t2).count()) / n;

    std::cerr << "[chained] alpha=" << alpha
        << " inserted=" << succ_found
        << " not_inserted=" << not_inserted.size() << "\n";

    std::cout << "chained," << alpha << "," << n << ","
        << (succ_probes / n) << "," << (unsucc_probes / n) << ","
        << succ_ns << "," << unsucc_ns << "\n";
}

// ---- open addressing ----
{
    OpenAddrTable ot(CAPACITY);
    for (const auto& k : inserted) {
        ot.insert(k);
    }

    long long succ_found = 0;
    auto t0 = std::chrono::high_resolution_clock::now();
    double succ_probes = 0.0;
    for (const auto& k : inserted) {
        int p = ot.find_success(k);
        succ_probes += p;
        ++succ_found;
    }
    auto t1 = std::chrono::high_resolution_clock::now();
    double succ_ns = static_cast<double>(
        std::chrono::duration_cast<std::chrono::nanoseconds>(t1 - t0).count()) / n;

    auto t2 = std::chrono::high_resolution_clock::now();
    double unsucc_probes = 0.0;
    for (const auto& k : not_inserted) {
        unsucc_probes += ot.find_unsucc(k);
    }
    auto t3 = std::chrono::high_resolution_clock::now();
    double unsucc_ns = static_cast<double>(
        std::chrono::duration_cast<std::chrono::nanoseconds>(t3 - t2).count()) / n;

    std::cerr << "[openaddr] alpha=" << alpha
        << " inserted=" << succ_found
        << " not_inserted=" << not_inserted.size() << "\n";

    std::cout << "openaddr," << alpha << "," << n << ","
        << (succ_probes / n) << "," << (unsucc_probes / n) << ","
        << succ_ns << "," << unsucc_ns << "\n";
}
}

return 0;
}

```

A.12 exp_keylen.cpp (実験 9: キー長の影響)

```
#include <algorithm>
#include <chrono>
#include <cstdint>
#include <iostream>
#include <random>
#include <string>
#include <vector>

static constexpr std::size_t N = 10000;
static constexpr std::size_t BUCKET_COUNT = 16384; // 2^14, load factor < 1
static constexpr std::size_t BUCKET_MASK = BUCKET_COUNT - 1;

static std::string random_key(std::mt19937& rng, int len) {
    static const char alnum[] =
        "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        "abcdefghijklmnopqrstuvwxyz"
        "0123456789";
    std::uniform_int_distribution<int> dist(0, 61);
    std::string s;
    s.reserve(len);
    for (int i = 0; i < len; ++i) {
        s.push_back(alnum[dist(rng)]);
    }
    return s;
}

// FNV-1a 64-bit hash
static std::uint64_t fnv1a(const std::string& key) {
    std::uint64_t h = 0xcbf29ce484222325ULL;
    for (unsigned char c : key) {
        h ^= c;
        h *= 0x100000001b3ULL;
    }
    return h;
}

// additive hash (same style as hashtable.cpp but with a larger bucket count)
static int additive_hash(const std::string& key) {
    int sum = 0;
    for (unsigned char c : key) {
        sum += c;
    }
    return sum % static_cast<int>(BUCKET_COUNT);
}

// chained hash table using FNV-1a + bucket mask
class ChainedTable {
    struct Node {
        std::string key;
        int value;
        Node* next;
    };
    std::vector<Node*> buckets_;

public:
    ChainedTable() : buckets_(BUCKET_COUNT, nullptr) {}
    ~ChainedTable() {
        for (Node* head : buckets_) {
            while (head) {
                Node* n = head;
                head = head->next;
                delete n;
            }
        }
    }

    void insert(const std::string& key, int value) {
        std::size_t h = fnv1a(key) & BUCKET_MASK;
        Node* n = new Node{key, value, buckets_[h]};
        buckets_[h] = n;
    }
}
```

```

const int* find(const std::string& key) const {
    std::size_t h = fnv1a(key) & BUCKET_MASK;
    for (Node* p = buckets_[h]; p; p = p->next) {
        if (p->key == key) {
            return &(p->value);
        }
    }
    return nullptr;
}

};

// OS 由来のノイズを抑えるため反復測定の最小値を採る（マイクロベンチマークの慣行）
static double min_of(std::vector<double> v) {
    std::sort(v.begin(), v.end());
    return v[0];
}

int main() {
    std::mt19937 rng(42);
    const std::vector<int> lengths = {8, 16, 32, 64, 128, 256};
    const int reps = 15;

    std::cout << "keylen,fnv1a_hash_ns,add_hash_ns,lookup_ns\n";

    for (int len : lengths) {
        std::vector<std::string> keys;
        keys.reserve(N);
        for (std::size_t i = 0; i < N; ++i) {
            keys.push_back(random_key(rng, len));
        }

        std::uint64_t fnv_total = 0;
        std::uint64_t add_total = 0;
        long long found = 0;
        std::vector<double> fnv_samples, add_samples, lookup_samples;

        ChainedTable table;
        for (std::size_t i = 0; i < keys.size(); ++i) {
            table.insert(keys[i], static_cast<int>(i));
        }

        for (int r = 0; r < reps; ++r) {
            // (1) FNV-1a hash only
            auto t0 = std::chrono::steady_clock::now();
            for (const auto& k : keys) {
                fnv_total += fnv1a(k);
            }
            auto t1 = std::chrono::steady_clock::now();
            fnv_samples.push_back(static_cast<double>(
                std::chrono::duration_cast<std::chrono::nanoseconds>(t1 - t0).count() / N));

            // (2) additive hash only
            auto t2 = std::chrono::steady_clock::now();
            for (const auto& k : keys) {
                add_total += static_cast<std::uint64_t>(additive_hash(k));
            }
            auto t3 = std::chrono::steady_clock::now();
            add_samples.push_back(static_cast<double>(
                std::chrono::duration_cast<std::chrono::nanoseconds>(t3 - t2).count() / N));

            // (3) look up all keys in the chained table
            auto t4 = std::chrono::steady_clock::now();
            for (const auto& k : keys) {
                const int* p = table.find(k);
                if (p != nullptr) {
                    ++found;
                }
            }
            auto t5 = std::chrono::steady_clock::now();
            lookup_samples.push_back(static_cast<double>(
                std::chrono::duration_cast<std::chrono::nanoseconds>(t5 - t4).count() / N));
        }

        std::cerr << "[keylen=" << len << "] found=" << found

```

```

        << " fnv_total=" << fnv_total
        << " add_total=" << add_total << "\n";

    std::cout << len << ", " << min_of(fnv_samples) << ", "
        << min_of(add_samples) << ", " << min_of(lookup_samples) << "\n";
}

return 0;
}

```

A.13 test_hashtable.cpp (単体テスト)

```

#include <iostream>
#include <string>

// キーは std::string 固定、値は任意型 T のテンプレート版ハッシュ表 (チェイン法)
template <class T>
class HashTable {
public:
    static constexpr int HASHSIZE = 17;

    // チェインのノード
    struct Node {
        std::string key;
        T value;
        Node* next;
    };

    // 全バケットを nullptr で初期化
    HashTable() : size_(0) {
        for (int i = 0; i < HASHSIZE; ++i) {
            buckets_[i] = nullptr;
        }
    }

    // 全ノードを削除
    ~HashTable() {
        for (int i = 0; i < HASHSIZE; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                delete p;
                p = next;
            }
        }
    }

    // コピー禁止
    HashTable(const HashTable&) = delete;
    HashTable& operator=(const HashTable&) = delete;

    // ハッシュ値を計算 (文字コードの総和 % HASHSIZE)
    static int hash(const std::string& key) {
        int hashval = 0;
        for (char c : key) {
            hashval += static_cast<unsigned char>(c);
        }
        return hashval % HASHSIZE;
    }

    // 先頭挿入で key-value を登録
    bool insert(const std::string& key, const T& value) {
        int h = hash(key);
        Node* p = new Node{key, value, buckets_[h]};
        buckets_[h] = p;
        ++size_;
        return true;
    }

    // key に対応する値へのポインタを返す (見つからなければ nullptr)
    const T* operator()(const std::string& key) const {
        int h = hash(key);

```

```

        Node* p = buckets_[h];
        while (p != nullptr) {
            if (p->key == key) {
                return &(p->value);
            }
            p = p->next;
        }
        return nullptr;
    }

    // key を削除する (存在しなければ何もしない)
    HashTable& operator-=(const std::string& key) {
        int h = hash(key);
        Node* p = buckets_[h];
        Node* prev = nullptr;
        while (p != nullptr) {
            if (p->key == key) {
                if (prev == nullptr) {
                    buckets_[h] = p->next;
                } else {
                    prev->next = p->next;
                }
                delete p;
                --size_;
                return *this;
            }
            prev = p;
            p = p->next;
        }
        return *this;
    }

    // 登録されている要素数を返す
    int size() const {
        return size_;
    }

    // 全バケットの内容を表示 (デバッグ用)
    void dump() const {
        for (int i = 0; i < HASHSIZE; ++i) {
            std::cout << " bucket[" << i << "]: ";
            Node* p = buckets_[i];
            if (p == nullptr) {
                std::cout << "(empty)";
            }
            while (p != nullptr) {
                std::cout << "(" << p->key << " -> " << p->value << ")";
                if (p->next != nullptr) {
                    std::cout << " -> ";
                }
                p = p->next;
            }
            std::cout << std::endl;
        }
    }

private:
    Node* buckets_[HASHSIZE];
    int size_;
};

static int total_tests = 0;
static int passed_tests = 0;

static void CHECK(bool cond, const char* name) {
    ++total_tests;
    if (cond) {
        ++passed_tests;
        std::cout << "OK: " << name << "\n";
    } else {
        std::cout << "NG: " << name << "\n";
    }
}

```

```

int main() {
    // 1. empty table
    {
        HashTable<int> ht;
        CHECK(ht("missing") == nullptr, "empty lookup returns nullptr");
        CHECK(ht.size() == 0, "empty size is 0");
        ht -= "missing";
        CHECK(ht.size() == 0, "erase on empty table is safe");
    }

    // 2. single insert and lookup
    {
        HashTable<int> ht;
        ht.insert("one", 1);
        const int* p = ht("one");
        CHECK(p != nullptr && *p == 1, "single insert/lookup");
    }

    // 3. colliding anagram keys (same additive hash)
    {
        HashTable<int> ht;
        ht.insert("abc", 1);
        ht.insert("acb", 2);
        ht.insert("bac", 3);
        ht.insert("bca", 4);
        ht.insert("cab", 5);
        ht.insert("cba", 6);
        const int* a = ht("abc");
        const int* b = ht("acb");
        const int* c = ht("bac");
        const int* d = ht("bca");
        const int* e = ht("cab");
        const int* f = ht("cba");
        CHECK(a && b && c && d && e && f, "all anagram keys found");
        CHECK(*a == 1 && *b == 2 && *c == 3 && *d == 4 && *e == 5 && *f == 6,
            "anagram values are correct");
    }

    // 4. erase head/middle/tail of a chain
    {
        HashTable<int> ht;
        ht.insert("abc", 1);
        ht.insert("acb", 2);
        ht.insert("bac", 3);
        // chain order after head-insertions: bac -> acb -> abc
        ht -= "bac"; // erase head
        CHECK(ht("bac") == nullptr, "erase head removes it");
        CHECK(ht("acb") != nullptr && *ht("acb") == 2, "head erase leaves middle");
        CHECK(ht("abc") != nullptr && *ht("abc") == 1, "head erase leaves tail");

        ht.insert("bac", 3); // restore head
        ht -= "acb"; // erase middle
        CHECK(ht("acb") == nullptr, "erase middle removes it");
        CHECK(ht("bac") != nullptr && *ht("bac") == 3, "middle erase leaves head");
        CHECK(ht("abc") != nullptr && *ht("abc") == 1, "middle erase leaves tail");

        ht.insert("acb", 2); // restore middle
        ht -= "abc"; // erase tail
        CHECK(ht("abc") == nullptr, "erase tail removes it");
        CHECK(ht("bac") != nullptr && *ht("bac") == 3, "tail erase leaves head");
        CHECK(ht("acb") != nullptr && *ht("acb") == 2, "tail erase leaves middle");
    }

    // 5. erasing non-existent key does not break the table
    {
        HashTable<int> ht;
        ht.insert("abc", 1);
        ht.insert("acb", 2);
        ht -= "notpresent";
        CHECK(ht.size() == 2, "erase missing key keeps size");
        CHECK(ht("abc") != nullptr && *ht("abc") == 1, "table intact after missing erase");
        CHECK(ht("acb") != nullptr && *ht("acb") == 2, "table intact after missing erase 2");
    }
}

```

```

// 6. erase then re-insert
{
    HashTable<int> ht;
    ht.insert("abc", 1);
    ht -= "abc";
    CHECK(ht("abc") == nullptr, "after erase key absent");
    ht.insert("abc", 10);
    const int* p = ht("abc");
    CHECK(p != nullptr && *p == 10, "re-inserted key found");
}

// 7. duplicate insert: front insert returns newest, one erase reveals old value
{
    HashTable<int> ht;
    ht.insert("dup", 100);
    ht.insert("dup", 200);
    const int* p = ht("dup");
    CHECK(p != nullptr && *p == 200, "duplicate insert returns newer value");
    ht -= "dup";
    p = ht("dup");
    CHECK(p != nullptr && *p == 100, "one erase reveals old value");
}

// 8. size tracks inserts and erases
{
    HashTable<int> ht;
    CHECK(ht.size() == 0, "initial size 0");
    ht.insert("a", 1);
    ht.insert("b", 2);
    CHECK(ht.size() == 2, "size after 2 inserts");
    ht -= "a";
    CHECK(ht.size() == 1, "size after 1 erase");
    ht -= "b";
    CHECK(ht.size() == 0, "size after all erased");
    ht -= "a";
    CHECK(ht.size() == 0, "size unchanged by missing erase");
}

// 9. bulk 1000 inserts/lookups and 500 erases
{
    HashTable<int> ht;
    for (int i = 0; i < 1000; ++i) {
        ht.insert("key" + std::to_string(i), i);
    }
    bool all_found = true;
    for (int i = 0; i < 1000; ++i) {
        const int* p = ht("key" + std::to_string(i));
        if (p == nullptr || *p != i) {
            all_found = false;
            break;
        }
    }
    CHECK(all_found, "1000 inserted keys all found");

    for (int i = 0; i < 500; ++i) {
        ht -= "key" + std::to_string(i);
    }
    bool remaining_ok = true;
    for (int i = 500; i < 1000; ++i) {
        const int* p = ht("key" + std::to_string(i));
        if (p == nullptr || *p != i) {
            remaining_ok = false;
            break;
        }
    }
    bool erased_ok = true;
    for (int i = 0; i < 500; ++i) {
        if (ht("key" + std::to_string(i)) != nullptr) {
            erased_ok = false;
            break;
        }
    }
    CHECK(remaining_ok, "remaining 500 keys still found");
    CHECK(erased_ok, "erased 500 keys return nullptr");
}

```

```

        CHECK(ht.size() == 500, "size after 500 erases is 500");
    }

// 10. HashTable<std::string> passes equivalents of tests 1-6
{
    // 1 empty
    HashTable<std::string> ht;
    CHECK(ht("missing") == nullptr, "string empty lookup returns nullptr");
    CHECK(ht.size() == 0, "string empty size is 0");
    ht -= "missing";
    CHECK(ht.size() == 0, "string erase on empty table is safe");

    // 2 single insert
    ht.insert("greeting", "hello");
    const std::string* ps = ht("greeting");
    CHECK(ps != nullptr && *ps == "hello", "string single insert/lookup");

    // 3 anagram collisions
    ht.insert("abc", "A");
    ht.insert("acb", "B");
    ht.insert("bac", "C");
    ps = ht("abc");
    const std::string* ps2 = ht("acb");
    const std::string* ps3 = ht("bac");
    CHECK(ps && ps2 && ps3, "string anagram keys found");
    CHECK(*ps == "A" && *ps2 == "B" && *ps3 == "C", "string anagram values correct");

    // 4 erase head/middle/tail
    ht.insert("bca", "D");
    ht.insert("cab", "E");
    ht.insert("cba", "F");
    // current chain for this bucket: cba -> cab -> bca -> bac -> acb -> abc
    ht -= "cba"; // head of newly extended chain
    CHECK(ht("cba") == nullptr, "string erase head removes it");
    CHECK(ht("cab") != nullptr && *ht("cab") == "E", "string head erase leaves middle");
    CHECK(ht("bca") != nullptr && *ht("bca") == "D", "string head erase leaves tail-ish");

    ht -= "bca"; // tail of remaining chain
    CHECK(ht("bca") == nullptr, "string erase tail removes it");
    CHECK(ht("cab") != nullptr && *ht("cab") == "E", "string tail erase leaves head");
    CHECK(ht("bac") != nullptr && *ht("bac") == "C", "string tail erase leaves middle");

    ht -= "cab"; // middle of remaining chain
    CHECK(ht("cab") == nullptr, "string erase middle removes it");
    CHECK(ht("bac") != nullptr && *ht("bac") == "C", "string middle erase leaves one side");
    CHECK(ht("abc") != nullptr && *ht("abc") == "A", "string middle erase leaves other side");

    // 5 erase non-existent does not break table
    ht -= "notpresent";
    CHECK(ht("abc") != nullptr && *ht("abc") == "A", "string table intact after missing erase");
    CHECK(ht("bac") != nullptr && *ht("bac") == "C", "string table intact after missing erase 2");

    // 6 erase then re-insert
    ht -= "abc";
    CHECK(ht("abc") == nullptr, "string after erase key absent");
    ht.insert("abc", "newA");
    ps = ht("abc");
    CHECK(ps != nullptr && *ps == "newA", "string re-inserted key found");
}

std::cout << "passed " << passed_tests << " / " << total_tests << "\n";
return (passed_tests == total_tests) ? 0 : 1;
}

```

A.14 hashtable_tagged.cpp (発展版: タグ付き比較)

```

#include <cmath>
#include <cstdint>
#include <iostream>
#include <string>
#include <utility>

```

```

// 動的バケット拡張、FNV-1a ハッシュ、およびタグ付き比較を用いた改良版ハッシュ表
template <class T>
class HashTable {
public:
    // チェインのノード（ハッシュ値全体を保存）
    struct Node {
        std::string key;
        T value;
        std::uint32_t hash;
        Node* next;
    };

    // 初期バケット数で空のハッシュ表を作成
    HashTable() : buckets_(nullptr), bucket_count_(17), size_(0),
        tag_comparisons_(0), string_comparisons_(0) {
        buckets_ = new Node*[bucket_count_];
        for (int i = 0; i < bucket_count_; ++i) {
            buckets_[i] = nullptr;
        }
    }

    // 全ノードとバケット配列を削除
    ~HashTable() {
        for (int i = 0; i < bucket_count_; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                delete p;
                p = next;
            }
            delete[] buckets_;
        }
    }

    // コピー禁止
    HashTable(const HashTable&) = delete;
    HashTable& operator=(const HashTable&) = delete;

    // カウンタをリセット
    void reset_counters() const {
        tag_comparisons_ = 0;
        string_comparisons_ = 0;
    }

    // カウンタ値を取得（first: タグ比較回数, second: 文字列比較回数）
    std::pair<std::uint64_t, std::uint64_t> counters() const {
        return {tag_comparisons_, string_comparisons_};
    }

    // FNV-1a 32bit ハッシュ値を計算
    static std::uint32_t fnv1a(const std::string& key) {
        std::uint32_t hashval = 2166136261u;
        for (unsigned char c : key) {
            hashval ^= c;
            hashval *= 16777619u;
        }
        return hashval;
    }

    int hash(const std::string& key) const {
        return static_cast<int>(fnv1a(key) % static_cast<std::uint32_t>(bucket_count_));
    }

    // n が素数かどうか判定
    static bool is_prime(int n) {
        if (n < 2) return false;
        if (n == 2) return true;
        if (n % 2 == 0) return false;
        for (int i = 3; i * i <= n; i += 2) {
            if (n % i == 0) return false;
        }
        return true;
    }
};

```

```

// 現在のバケット数の 2 倍以上で最小の素数を返す
static int next_prime(int current) {
    int candidate = current * 2 + 1;
    while (!is_prime(candidate)) {
        candidate += 2;
    }
    return candidate;
}

// バケット数を拡張して全要素を再配置（保存済みハッシュ値を再利用）
void rehash(int new_bucket_count) {
    Node** new_buckets = new Node*[new_bucket_count];
    for (int i = 0; i < new_bucket_count; ++i) {
        new_buckets[i] = nullptr;
    }

    for (int i = 0; i < bucket_count_; ++i) {
        Node* p = buckets_[i];
        while (p != nullptr) {
            Node* next = p->next;
            int h = static_cast<int>(p->hash % static_cast<std::uint32_t>(new_bucket_count));
            p->next = new_buckets[h];
            new_buckets[h] = p;
            p = next;
        }
    }
    delete[] buckets_;
    buckets_ = new_buckets;
    bucket_count_ = new_bucket_count;
}

// 負荷率が 1.0 を超えたら自動拡張して先頭挿入
bool insert(const std::string& key, const T& value) {
    if (load_factor() > 1.0) {
        rehash(next_prime(bucket_count_));
    }
    std::uint32_t hval = fnv1a(key);
    int h = static_cast<int>(hval % static_cast<std::uint32_t>(bucket_count_));
    Node* p = new Node{key, value, hval, buckets_[h]};
    buckets_[h] = p;
    ++size_;
    return true;
}

// key に対応する値へのポインタを返す（見つからなければ nullptr）
// タグ（保存ハッシュ値）が一致した場合のみ文字列比較を行う
const T* operator()(const std::string& key) const {
    std::uint32_t hval = fnv1a(key);
    int h = static_cast<int>(hval % static_cast<std::uint32_t>(bucket_count_));
    Node* p = buckets_[h];
    while (p != nullptr) {
        ++tag_comparisons_;
        if (p->hash == hval) {
            ++string_comparisons_;
            if (p->key == key) {
                return &(p->value);
            }
        }
        p = p->next;
    }
    return nullptr;
}

// key を削除する（存在しなければ何もしない）
HashTable& operator-=(const std::string& key) {
    std::uint32_t hval = fnv1a(key);
    int h = static_cast<int>(hval % static_cast<std::uint32_t>(bucket_count_));
    Node* p = buckets_[h];
    Node* prev = nullptr;
    while (p != nullptr) {
        ++tag_comparisons_;
        if (p->hash == hval) {
            ++string_comparisons_;
            if (p->key == key) {

```

```

        if (prev == nullptr) {
            buckets_[h] = p->next;
        } else {
            prev->next = p->next;
        }
        delete p;
        --size_;
        return *this;
    }
}

prev = p;
p = p->next;
}

return *this;
}

// 要素数を返す
int size() const {
    return size_;
}

// バケット数を返す
int bucket_count() const {
    return bucket_count_;
}

// 負荷率を返す
double load_factor() const {
    return static_cast<double>(size_) / static_cast<double>(bucket_count_);
}

// 全バケットの内容を表示 (デバッグ用、max_buckets >= 0 なら先頭のみ)
void dump(int max_buckets = -1) const {
    int limit = (max_buckets >= 0 && max_buckets < bucket_count_) ? max_buckets : bucket_count_;
    for (int i = 0; i < limit; ++i) {
        std::cout << "    bucket[" << i << "]: ";
        Node* p = buckets_[i];
        if (p == nullptr) {
            std::cout << "(empty)";
        }
        while (p != nullptr) {
            std::cout << "(" << p->key << " -> " << p->value << ")";
            if (p->next != nullptr) {
                std::cout << " -> ";
            }
            p = p->next;
        }
        std::cout << std::endl;
    }
    if (limit < bucket_count_) {
        std::cout << "    ... (残り " << (bucket_count_ - limit) << " バケットは省略)" << std::endl;
    }
}

private:
    Node** buckets_;
    int bucket_count_;
    int size_;
    mutable std::uint64_t tag_comparisons_;
    mutable std::uint64_t string_comparisons_;
};

int main() {
    HashTable<int> ht;

    std::cout << "=== 大量挿入前 ===" << std::endl;
    std::cout << "bucket_count: " << ht.bucket_count() << ", size: " << ht.size()
        << ", load_factor: " << ht.load_factor() << std::endl;

    std::cout << "=== key0000 ~ key0999 を 0 ~ 999 とともに挿入 ===" << std::endl;
    int prev_bucket_count = ht.bucket_count();
    for (int i = 0; i < 1000; ++i) {
        std::string key = "key";
        if (i < 10) {

```

```

        key += "000";
    } else if (i < 100) {
        key += "00";
    } else if (i < 1000) {
        key += "0";
    }
    key += std::to_string(i);
    ht.insert(key, i);
    if (ht.bucket_count() != prev_bucket_count) {
        std::cout << "  拡張: bucket_count " << prev_bucket_count << " -> " << ht.bucket_count()
            << " (size=" << ht.size() << ", load_factor=" << ht.load_factor() << ")" << std::endl;
        prev_bucket_count = ht.bucket_count();
    }
}

std::cout << "=== 挿入後 ===" << std::endl;
std::cout << "bucket_count: " << ht.bucket_count() << ", size: " << ht.size()
    << ", load_factor: " << ht.load_factor() << std::endl;

std::cout << "=== 検索テスト ===" << std::endl;
const int* p;
p = ht("key0000");
std::cout << "find key0000: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;
p = ht("key0123");
std::cout << "find key0123: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;
p = ht("key0999");
std::cout << "find key0999: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;
p = ht("key1000");
std::cout << "find key1000: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;

std::cout << "=== 全キー検索でのタグ照合効果 ===" << std::endl;
ht.reset_counters();
for (int i = 0; i < 1000; ++i) {
    std::string key = "key";
    if (i < 10) {
        key += "000";
    } else if (i < 100) {
        key += "00";
    } else if (i < 1000) {
        key += "0";
    }
    key += std::to_string(i);
    const int* v = ht(key);
    if (v == nullptr) {
        std::cerr << "unexpected missing key: " << key << std::endl;
    }
}

auto c = ht.counters();
std::cout << "1000 件全キー検索: tag comparisons = " << c.first
    << ", string comparisons = " << c.second << std::endl;
std::cout << "  -> タグ不一致により文字列比較を " << (c.first - c.second)
    << "  回省略しました" << std::endl;

std::cout << "=== 削除テスト ===" << std::endl;
ht -= "key0123";
std::cout << "key0123 を削除しました" << std::endl;
p = ht("key0123");
std::cout << "find key0123 after erase: " << (p != nullptr ? std::to_string(*p) : "none") << std::endl;
std::cout << "size: " << ht.size() << ", load_factor: " << ht.load_factor() << std::endl;

std::cout << "=== 先頭 30 バケットの内容 (拡張後も整合性確認) ===" << std::endl;
ht.dump(30);

return 0;
}

```

A.15 exp_tag.cpp (実験 10: タグ付き比較の効果)

```

#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdint>

```

```

#include <iomanip>
#include <iostream>
#include <random>
#include <string>
#include <unordered_map>
#include <vector>

// ---- (a) improved 相当：タグなし、FNV-1a + 素数リハッシュのチェイン法 ----
template <class T>
class HashTableBaseline {
public:
    struct Node {
        std::string key;
        T value;
        Node* next;
    };

    HashTableBaseline()
        : buckets_(nullptr), bucket_count_(17), size_(0) {
        buckets_ = new Node*[bucket_count_];
        for (int i = 0; i < bucket_count_; ++i) {
            buckets_[i] = nullptr;
        }
    }

    ~HashTableBaseline() {
        for (int i = 0; i < bucket_count_; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                delete p;
                p = next;
            }
        }
        delete[] buckets_;
    }

    HashTableBaseline(const HashTableBaseline&) = delete;
    HashTableBaseline& operator=(const HashTableBaseline&) = delete;

    static std::uint32_t fnv1a(const std::string& key) {
        std::uint32_t hashval = 2166136261u;
        for (unsigned char c : key) {
            hashval ^= c;
            hashval *= 16777619u;
        }
        return hashval;
    }

    int hash(const std::string& key) const {
        return static_cast<int>(fnv1a(key) % static_cast<std::uint32_t>(bucket_count_));
    }

    static bool is_prime(int n) {
        if (n < 2) return false;
        if (n == 2) return true;
        if (n % 2 == 0) return false;
        for (int i = 3; i * i <= n; i += 2) {
            if (n % i == 0) return false;
        }
        return true;
    }

    static int next_prime(int current) {
        int candidate = current * 2 + 1;
        while (!is_prime(candidate)) {
            candidate += 2;
        }
        return candidate;
    }

    void rehash(int new_bucket_count) {
        Node** new_buckets = new Node*[new_bucket_count];
        for (int i = 0; i < new_bucket_count; ++i) {

```

```

        new_buckets[i] = nullptr;
    }
    for (int i = 0; i < bucket_count_; ++i) {
        Node* p = buckets_[i];
        while (p != nullptr) {
            Node* next = p->next;
            int h = static_cast<int>(fnvia(p->key) % static_cast<std::uint32_t>(new_bucket_count));
            p->next = new_buckets[h];
            new_buckets[h] = p;
            p = next;
        }
    }
    delete[] buckets_;
    buckets_ = new_buckets;
    bucket_count_ = new_bucket_count;
}

bool insert(const std::string& key, const T& value) {
    if (load_factor() > 1.0) {
        rehash(next_prime(bucket_count_));
    }
    int h = hash(key);
    Node* p = new Node{key, value, buckets_[h]};
    buckets_[h] = p;
    ++size_;
    return true;
}

const T* operator()(const std::string& key) const {
    int h = hash(key);
    Node* p = buckets_[h];
    while (p != nullptr) {
        ++strcmp_count_;
        if (p->key == key) {
            return &(p->value);
        }
        p = p->next;
    }
    return nullptr;
}

int size() const { return size_; }
int bucket_count() const { return bucket_count_; }
double load_factor() const {
    return static_cast<double>(size_) / static_cast<double>(bucket_count_);
}
void reset_counters() const { strcmp_count_ = 0; }
long long strcmp_count() const { return strcmp_count_; }

private:
    Node** buckets_;
    int bucket_count_;
    int size_;
    mutable long long strcmp_count_ = 0;
};

```

// ---- (b) tagged: タグ付き比較 + リハッシュ時は保存済みハッシュを利用 ----

```

template <class T>
class HashTableTagged {
public:
    struct Node {
        std::string key;
        T value;
        std::uint32_t hash;
        Node* next;
    };

    HashTableTagged()
        : buckets_(nullptr), bucket_count_(17), size_(0) {
        buckets_ = new Node[bucket_count_];
        for (int i = 0; i < bucket_count_; ++i) {
            buckets_[i] = nullptr;
        }
    }
};

```

```

~HashTableTagged() {
    for (int i = 0; i < bucket_count_; ++i) {
        Node* p = buckets_[i];
        while (p != nullptr) {
            Node* next = p->next;
            delete p;
            p = next;
        }
    }
    delete[] buckets_;
}

HashTableTagged(const HashTableTagged&) = delete;
HashTableTagged& operator=(const HashTableTagged&) = delete;

static std::uint32_t fnv1a(const std::string& key) {
    std::uint32_t hashval = 2166136261u;
    for (unsigned char c : key) {
        hashval ^= c;
        hashval *= 16777619u;
    }
    return hashval;
}

static bool is_prime(int n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (int i = 3; i * i <= n; i += 2) {
        if (n % i == 0) return false;
    }
    return true;
}

static int next_prime(int current) {
    int candidate = current * 2 + 1;
    while (!is_prime(candidate)) {
        candidate += 2;
    }
    return candidate;
}

void rehash(int new_bucket_count) {
    Node** new_buckets = new Node*[new_bucket_count];
    for (int i = 0; i < new_bucket_count; ++i) {
        new_buckets[i] = nullptr;
    }
    for (int i = 0; i < bucket_count_; ++i) {
        Node* p = buckets_[i];
        while (p != nullptr) {
            Node* next = p->next;
            int h = static_cast<int>(p->hash % static_cast<std::uint32_t>(new_bucket_count));
            p->next = new_buckets[h];
            new_buckets[h] = p;
            p = next;
        }
    }
    delete[] buckets_;
    buckets_ = new_buckets;
    bucket_count_ = new_bucket_count;
}

bool insert(const std::string& key, const T& value) {
    if (load_factor() > 1.0) {
        rehash(next_prime(bucket_count_));
    }
    std::uint32_t hval = fnv1a(key);
    int h = static_cast<int>(hval % static_cast<std::uint32_t>(bucket_count_));
    Node* p = new Node{key, value, hval, buckets_[h]};
    buckets_[h] = p;
    ++size_;
    return true;
}

```

```

const T* operator()(const std::string& key) const {
    std::uint32_t hval = fnv1a(key);
    int h = static_cast<int>(hval % static_cast<std::uint32_t>(bucket_count_));
    Node* p = buckets_[h];
    while (p != nullptr) {
        if (p->hash == hval) {
            ++strcmp_count_;
            if (p->key == key) {
                return &(p->value);
            }
        }
        p = p->next;
    }
    return nullptr;
}

int size() const { return size_; }
int bucket_count() const { return bucket_count_; }
double load_factor() const {
    return static_cast<double>(size_) / static_cast<double>(bucket_count_);
}
void reset_counters() const { strcmp_count_ = 0; }
long long strcmp_count() const { return strcmp_count_; }

private:
    Node** buckets_;
    int bucket_count_;
    int size_;
    mutable long long strcmp_count_ = 0;
};

// ---- ユーティリティ ----
static std::string make_key(std::mt19937& gen, int len) {
    static const char chars[] =
        "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
    std::uniform_int_distribution<int> dist(0, 61);
    std::string s;
    s.reserve(len);
    for (int i = 0; i < len; ++i) {
        s += chars[dist(gen)];
    }
    return s;
}

template <class Table>
static void measure(const std::string& name, int keylen,
    const std::vector<std::string>& keys,
    const std::vector<std::string>& missing_keys,
    long long& out_insert_us,
    long long& out_succ_ns,
    long long& out_unsucc_ns,
    double& out_succ_strcmp,
    double& out_unsucc_strcmp) {
    using namespace std::chrono;
    constexpr int iters = 9;
    constexpr int lookups = 100000;

    out_insert_us = std::numeric_limits<long long>::max();
    out_succ_ns = std::numeric_limits<long long>::max();
    out_unsucc_ns = std::numeric_limits<long long>::max();

    std::mt19937 gen(42);
    std::uniform_int_distribution<std::size_t> idx_dist(0, keys.size() - 1);

    for (int iter = 0; iter < iters; ++iter) {
        Table table;

        // (1) 挿入時間
        auto t0 = steady_clock::now();
        for (const auto& k : keys) {
            if constexpr (std::is_same_v<Table, std::unordered_map<std::string, int>>) {
                table.emplace(k, 1);
            } else {

```

```

        table.insert(k, 1);
    }
}
auto t1 = steady_clock::now();
long long insert_us = duration_cast<microseconds>(t1 - t0).count();
out_insert_us = std::min(out_insert_us, insert_us);

auto do_lookup = [](const Table& tbl, const std::string& k) -> const int* {
    if constexpr (std::is_same_v<Table, std::unordered_map<std::string, int>>) {
        auto it = tbl.find(k);
        return (it != tbl.end()) ? &(it->second) : nullptr;
    } else {
        return tbl(k);
    }
};

// (2) 成功検索時間
long long found = 0;
t0 = steady_clock::now();
for (int i = 0; i < lookups; ++i) {
    const int* v = do_lookup(table, keys[idx_dist(gen)]);
    if (v != nullptr) {
        ++found;
    }
}
t1 = steady_clock::now();
long long succ_total_ns = duration_cast<nanoseconds>(t1 - t0).count();
out_succ_ns = std::min(out_succ_ns, succ_total_ns / lookups);
std::cerr << name << " L=" << keylen << " iter=" << iter
            << " found=" << found << std::endl;

// (3) 不成功検索時間
found = 0;
t0 = steady_clock::now();
for (int i = 0; i < lookups; ++i) {
    const int* v = do_lookup(table, missing_keys[i]);
    if (v != nullptr) {
        ++found;
    }
}
t1 = steady_clock::now();
long long unsucc_total_ns = duration_cast<nanoseconds>(t1 - t0).count();
out_unsucc_ns = std::min(out_unsucc_ns, unsucc_total_ns / lookups);
std::cerr << name << " L=" << keylen << " iter=" << iter
            << " unsucc_found=" << found << std::endl;
}

// (4) 決定的な指標: 検索 1 回あたりの文字列比較回数 (時間と違い環境に依存しない)
out_succ_strcmp = -1.0;
out_unsucc_strcmp = -1.0;
if constexpr (!std::is_same_v<Table, std::unordered_map<std::string, int>>) {
    Table table;
    for (const auto& k : keys) {
        table.insert(k, 1);
    }
    std::mt19937 g2(42);
    std::uniform_int_distribution<std::size_t> d2(0, keys.size() - 1);
    table.reset_counters();
    for (int i = 0; i < lookups; ++i) {
        (void)table[keys[d2(g2)]];
    }
    out_succ_strcmp = static_cast<double>(table strcmp_count()) / lookups;
    table.reset_counters();
    for (int i = 0; i < lookups; ++i) {
        (void)table[missing_keys[i]];
    }
    out_unsucc_strcmp = static_cast<double>(table strcmp_count()) / lookups;
}
}

template <class Table>
static void run_for_table(const std::string& name,
                        const std::vector<std::vector<std::string>>& all_keys,
                        const std::vector<std::vector<std::string>>& all_missing) {

```

```

const int lens[] = {12, 64, 256};
for (std::size_t i = 0; i < all_keys.size(); ++i) {
    long long insert_us, succ_ns, unsucc_ns;
    double succ_strcmp, unsucc_strcmp;
    measure<Table>(name, lens[i], all_keys[i], all_missing[i],
        insert_us, succ_ns, unsucc_ns, succ_strcmp, unsucc_strcmp);
    std::cout << name << ", " << lens[i] << ", "
        << insert_us << ", " << succ_ns << ", " << unsucc_ns << ", "
        << std::fixed << std::setprecision(5) << succ_strcmp << ", "
        << unsucc_strcmp << std::endl;
}
}

int main() {
    constexpr int N = 200000;
    constexpr int lookups = 100000;
    const int lens[] = {12, 64, 256};

    std::mt19937 gen_insert(42);
    std::mt19937 gen_missing(123456789);

    std::vector<std::vector<std::string>> all_keys(3);
    std::vector<std::vector<std::string>> all_missing(3);
    for (int i = 0; i < 3; ++i) {
        int L = lens[i];
        all_keys[i].reserve(N);
        all_missing[i].reserve(lookups);
        for (int j = 0; j < N; ++j) {
            all_keys[i].push_back(make_key(gen_insert, L));
        }
        for (int j = 0; j < lookups; ++j) {
            all_missing[i].push_back(make_key(gen_missing, L));
        }
    }

    std::cout << "structure,keylen,insert_total_us,succ_lookup_ns,unsucc_lookup_ns,succ_strcmp,unsucc_strcmp" << std::endl;
    run_for_table<HashTableBaseline<int>>("baseline", all_keys, all_missing);
    run_for_table<HashTableTagged<int>>("tagged", all_keys, all_missing);
    run_for_table<std::unordered_map<std::string, int>>("unordered_map", all_keys, all_missing);

    return 0;
}

```

A.16 exp_hashdos.cpp (実験 11: ハッシュ飽和攻撃)

```

#include <algorithm>
#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <iomanip>
#include <iostream>
#include <random>
#include <set>
#include <string>
#include <unordered_set>
#include <vector>

// =====
// fixed17: 加算ハッシュ、バケット数 17 固定のチェイン法
// 検索時のキー比較回数を返せる
// =====
class Fixed17Table {
public:
    struct Node {
        std::string key;
        int value;
        Node* next;
    };

    Fixed17Table() : size_(0) {
        for (int i = 0; i < 17; ++i) {
            buckets_[i] = nullptr;
        }
    }

```

```

    }
}

~Fixed17Table() {
    clear();
}

Fixed17Table(const Fixed17Table&) = delete;
Fixed17Table& operator=(const Fixed17Table&) = delete;

void insert(const std::string& key, int value) {
    const int h = hash_(key);
    buckets_[h] = new Node{key, value, buckets_[h]};
    ++size_;
}

// 発見したら true。比較回数は comparisons に書き込む。
bool find(const std::string& key, long long& comparisons) const {
    const int h = hash_(key);
    comparisons = 0;
    for (Node* p = buckets_[h]; p != nullptr; p = p->next) {
        ++comparisons;
        if (p->key == key) {
            return true;
        }
    }
    return false;
}

void clear() {
    for (int i = 0; i < 17; ++i) {
        Node* p = buckets_[i];
        while (p != nullptr) {
            Node* next = p->next;
            delete p;
            p = next;
        }
        buckets_[i] = nullptr;
    }
    size_ = 0;
}

int size() const {
    return size_;
}

int max_chain() const {
    int max_len = 0;
    for (int i = 0; i < 17; ++i) {
        int len = 0;
        for (Node* p = buckets_[i]; p != nullptr; p = p->next) {
            ++len;
        }
        if (len > max_len) {
            max_len = len;
        }
    }
    return max_len;
}

int used_buckets() const {
    int count = 0;
    for (int i = 0; i < 17; ++i) {
        if (buckets_[i] != nullptr) {
            ++count;
        }
    }
    return count;
}

private:
    static int hash_(const std::string& key) {
        int h = 0;
        for (unsigned char c : key) {

```

```

        h += static_cast<int>(c);
    }
    return h % 17;
}

Node* buckets_[17];
int size_;
};

// =====
// improved: FNV-1a 32bit + 負荷率 1.0 超で素数リハッシュ
// =====
class ImprovedTable {
public:
    struct Node {
        std::string key;
        int value;
        Node* next;
    };

    ImprovedTable() : buckets_(nullptr), bucket_count_(17), size_(0) {
        buckets_ = new Node*[bucket_count_];
        for (int i = 0; i < bucket_count_; ++i) {
            buckets_[i] = nullptr;
        }
    }

    ~ImprovedTable() {
        clear();
        delete[] buckets_;
    }

    ImprovedTable(const ImprovedTable&) = delete;
    ImprovedTable& operator=(const ImprovedTable&) = delete;

    void insert(const std::string& key, int value) {
        if (load_factor() > 1.0) {
            rehash(next_prime_(bucket_count_));
        }
        const int h = hash_(key);
        buckets_[h] = new Node{key, value, buckets_[h]};
        ++size_;
    }

    bool find(const std::string& key, long long& comparisons) const {
        const int h = hash_(key);
        comparisons = 0;
        for (Node* p = buckets_[h]; p != nullptr; p = p->next) {
            ++comparisons;
            if (p->key == key) {
                return true;
            }
        }
        return false;
    }

    void clear() {
        for (int i = 0; i < bucket_count_; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                delete p;
                p = next;
            }
            buckets_[i] = nullptr;
        }
        size_ = 0;
    }

    int size() const {
        return size_;
    }

    int bucket_count() const {

```

```

        return bucket_count_;
    }

    int max_chain() const {
        int max_len = 0;
        for (int i = 0; i < bucket_count_; ++i) {
            int len = 0;
            for (Node* p = buckets_[i]; p != nullptr; p = p->next) {
                ++len;
            }
            if (len > max_len) {
                max_len = len;
            }
        }
        return max_len;
    }

    int used_buckets() const {
        int count = 0;
        for (int i = 0; i < bucket_count_; ++i) {
            if (buckets_[i] != nullptr) {
                ++count;
            }
        }
        return count;
    }
}

private:
    static std::uint32_t fnv1a_(const std::string& key) {
        std::uint32_t h = 2166136261u;
        for (unsigned char c : key) {
            h ^= static_cast<std::uint32_t>(c);
            h *= 16777619u;
        }
        return h;
    }

    int hash_(const std::string& key) const {
        return static_cast<int>(
            fnv1a_(key) % static_cast<std::uint32_t>(bucket_count_));
    }

    static bool is_prime_(int n) {
        if (n < 2) return false;
        if (n == 2) return true;
        if (n % 2 == 0) return false;
        for (int i = 3; i * i <= n; i += 2) {
            if (n % i == 0) return false;
        }
        return true;
    }

    static int next_prime_(int current) {
        int candidate = current * 2 + 1;
        while (!is_prime_(candidate)) {
            candidate += 2;
        }
        return candidate;
    }

    void rehash(int new_bucket_count) {
        Node** new_buckets = new Node*[new_bucket_count];
        for (int i = 0; i < new_bucket_count; ++i) {
            new_buckets[i] = nullptr;
        }
        for (int i = 0; i < bucket_count_; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                const int h = static_cast<int>(
                    fnv1a_(p->key) % static_cast<std::uint32_t>(new_bucket_count));
                p->next = new_buckets[h];
                new_buckets[h] = p;
                p = next;
            }
        }
    }

```

```

    }
}
delete[] buckets_;
buckets_ = new_buckets;
bucket_count_ = new_bucket_count;
}

double load_factor() const {
    return static_cast<double>(size_) / static_cast<double>(bucket_count_);
}

Node** buckets_;
int bucket_count_;
int size_;
};

// =====
// キー生成
// =====
static int additive_hash_mod17(const std::string& key) {
    int h = 0;
    for (unsigned char c : key) {
        h += static_cast<int>(c);
    }
    return h % 17;
}

static std::vector<std::string> generate_attack_keys(int N) {
    std::string base = "aaabbbccdd";
    std::sort(base.begin(), base.end());

    std::vector<std::string> keys;
    keys.reserve(static_cast<std::size_t>(N));

    do {
        keys.push_back(base);
        if (static_cast<int>(keys.size()) >= N) {
            break;
        }
    } while (std::next_permutation(base.begin(), base.end()));

    if (static_cast<int>(keys.size()) < N) {
        std::cerr << "warning: could not generate enough attack keys ("
            << keys.size() << " / " << N << ")\n";
    }
    return keys;
}

static std::vector<std::string> generate_random_keys(int N) {
    static const char chars[] =
        "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
    static const int L = 62;

    std::mt19937 rng(42);
    std::uniform_int_distribution<int> dist(0, L - 1);

    std::unordered_set<std::string> seen;
    seen.reserve(static_cast<std::size_t>(N) * 2);

    std::vector<std::string> keys;
    keys.reserve(static_cast<std::size_t>(N));

    std::string key(12, ' ');
    while (static_cast<int>(keys.size()) < N) {
        for (int i = 0; i < 12; ++i) {
            key[i] = chars[dist(rng)];
        }
        if (seen.insert(key).second) {
            keys.push_back(key);
        }
    }
    return keys;
}

```

```

// =====
// 計測
// =====
struct MeasureResult {
    double avg_comparisons;
    double avg_lookup_ns;
};

template <class Table>
static MeasureResult measure(Table& table, const std::vector<std::string>& keys) {
    const int n = static_cast<int>(keys.size());
    const int lookup_repeats = 5;

    // 挿入
    for (int i = 0; i < n; ++i) {
        table.insert(keys[i], i);
    }

    // 決定的な平均キー比較回数
    long long total_comparisons = 0;
    long long found_count = 0;
    for (int r = 0; r < lookup_repeats; ++r) {
        for (int i = 0; i < n; ++i) {
            long long comp = 0;
            const bool found = table.find(keys[i], comp);
            if (found) {
                ++found_count;
            }
            total_comparisons += comp;
        }
    }
    const double avg_comparisons =
        static_cast<double>(total_comparisons) /
        static_cast<double>(n * lookup_repeats);

    if (found_count != static_cast<long long>(n) * lookup_repeats) {
        std::cerr << "error: some keys were not found during measurement\n";
    }

    // 平均検索時間: 9 回反復の最小値
    // コンパイラの最適化で検索ループが消えないよう、volatile sink に結果を加算
    using clock = std::chrono::steady_clock;
    long long best_total_ns = -1;
    const int time_repeats = 9;
    volatile long long time_sink = 0;
    for (int iter = 0; iter < time_repeats; ++iter) {
        const auto t0 = clock::now();
        for (int r = 0; r < lookup_repeats; ++r) {
            for (int i = 0; i < n; ++i) {
                long long comp = 0;
                table.find(keys[i], comp);
                time_sink += comp;
            }
        }
        const auto t1 = clock::now();
        const long long ns =
            std::chrono::duration_cast<std::chrono::nanoseconds>(t1 - t0).count();
        if (best_total_ns < 0 || ns < best_total_ns) {
            best_total_ns = ns;
        }
    }
    if (time_sink < 0) {
        std::cerr << time_sink << "\n";
    }
    const double avg_lookup_ns =
        static_cast<double>(best_total_ns) /
        static_cast<double>(n * lookup_repeats);

    return {avg_comparisons, avg_lookup_ns};
}

template <class Table>
static void report_stats(const std::string& name, const Table& table) {
    std::cerr << name << ": "

```

```

        << "size=" << table.size()
        << " buckets=" << table.used_buckets()
        << " max_chain=" << table.max_chain()
        << " load_factor=" << std::fixed << std::setprecision(3)
        << (static_cast<double>(table.size()) /
            static_cast<double>(table.used_buckets()))
        << "\n";
    }

// =====
// メイン
// =====
int main() {
    const int N = 2000;

    const auto attack_keys = generate_attack_keys(N);
    const auto random_keys = generate_random_keys(N);

    // attack キーの加算ハッシュが全て同一か std::set で検証
    std::set<int> attack_hashes;
    for (const auto& k : attack_keys) {
        attack_hashes.insert(additive_hash_mod17(k));
    }
    std::cerr << "attack keys additive hash (mod 17):";
    for (int h : attack_hashes) {
        std::cerr << " " << h;
    }
    std::cerr << (attack_hashes.size() == 1 ? " (all same)\n" : " (DIFFERENT!)\n");

    std::cerr << "attack keys: " << attack_keys.size()
        << ", random keys: " << random_keys.size() << "\n";

    // (a) fixed17 / attack
    Fixed17Table fixed17_attack;
    const auto r1 = measure(fixed17_attack, attack_keys);
    report_stats("fixed17/attack", fixed17_attack);

    // (b) fixed17 / random
    Fixed17Table fixed17_random;
    const auto r2 = measure(fixed17_random, random_keys);
    report_stats("fixed17/random", fixed17_random);

    // (c) improved / attack
    ImprovedTable improved_attack;
    const auto r3 = measure(improved_attack, attack_keys);
    report_stats("improved/attack", improved_attack);

    // (d) improved / random
    ImprovedTable improved_random;
    const auto r4 = measure(improved_random, random_keys);
    report_stats("improved/random", improved_random);

    std::cout << "table,keyset,n,avg_comparisons,avg_lookup_ns\n";
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "fixed17,attack," << N << "," << r1.avg_comparisons << ","
        << r1.avg_lookup_ns << "\n";
    std::cout << "fixed17,random," << N << "," << r2.avg_comparisons << ","
        << r2.avg_lookup_ns << "\n";
    std::cout << "improved,attack," << N << "," << r3.avg_comparisons << ","
        << r3.avg_lookup_ns << "\n";
    std::cout << "improved,random," << N << "," << r4.avg_comparisons << ","
        << r4.avg_lookup_ns << "\n";

    return 0;
}

```

A.17 exp_hashdos.cpp (実験 11: ハッシュ飽和攻撃の実証)

```

#include <algorithm>
#include <chrono>
#include <cstdint>
#include <cstdlib>

```

```

#include <iomanip>
#include <iostream>
#include <random>
#include <set>
#include <string>
#include <unordered_set>
#include <vector>

// =====
// fixed17: 加算ハッシュ、バケット数 17 固定のチェイン法
// 検索時のキー比較回数を返せる
// =====
class Fixed17Table {
public:
    struct Node {
        std::string key;
        int value;
        Node* next;
    };

    Fixed17Table() : size_(0) {
        for (int i = 0; i < 17; ++i) {
            buckets_[i] = nullptr;
        }
    }

    ~Fixed17Table() {
        clear();
    }

    Fixed17Table(const Fixed17Table&) = delete;
    Fixed17Table& operator=(const Fixed17Table&) = delete;

    void insert(const std::string& key, int value) {
        const int h = hash_(key);
        buckets_[h] = new Node{key, value, buckets_[h]};
        ++size_;
    }

    // 発見したら true。比較回数は comparisons に書き込む。
    bool find(const std::string& key, long long& comparisons) const {
        const int h = hash_(key);
        comparisons = 0;
        for (Node* p = buckets_[h]; p != nullptr; p = p->next) {
            ++comparisons;
            if (p->key == key) {
                return true;
            }
        }
        return false;
    }

    void clear() {
        for (int i = 0; i < 17; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) {
                Node* next = p->next;
                delete p;
                p = next;
            }
            buckets_[i] = nullptr;
        }
        size_ = 0;
    }

    int size() const {
        return size_;
    }

    int max_chain() const {
        int max_len = 0;
        for (int i = 0; i < 17; ++i) {
            int len = 0;
            for (Node* p = buckets_[i]; p != nullptr; p = p->next) {

```

```

        ++len;
    }
    if (len > max_len) {
        max_len = len;
    }
}
return max_len;
}

int used_buckets() const {
    int count = 0;
    for (int i = 0; i < 17; ++i) {
        if (buckets_[i] != nullptr) {
            ++count;
        }
    }
    return count;
}

private:
    static int hash_(const std::string& key) {
        int h = 0;
        for (unsigned char c : key) {
            h += static_cast<int>(c);
        }
        return h % 17;
    }

    Node* buckets_[17];
    int size_;
};

// =====
// improved: FNV-1a 32bit + 負荷率 1.0 超で素数リハッシュ
// =====
class ImprovedTable {
public:
    struct Node {
        std::string key;
        int value;
        Node* next;
    };

    ImprovedTable() : buckets_(nullptr), bucket_count_(17), size_(0) {
        buckets_ = new Node*[bucket_count_];
        for (int i = 0; i < bucket_count_; ++i) {
            buckets_[i] = nullptr;
        }
    }

    ~ImprovedTable() {
        clear();
        delete[] buckets_;
    }

    ImprovedTable(const ImprovedTable&) = delete;
    ImprovedTable& operator=(const ImprovedTable&) = delete;

    void insert(const std::string& key, int value) {
        if (load_factor() > 1.0) {
            rehash(next_prime_(bucket_count_));
        }
        const int h = hash_(key);
        buckets_[h] = new Node{key, value, buckets_[h]};
        ++size_;
    }

    bool find(const std::string& key, long long& comparisons) const {
        const int h = hash_(key);
        comparisons = 0;
        for (Node* p = buckets_[h]; p != nullptr; p = p->next) {
            ++comparisons;
            if (p->key == key) {
                return true;
            }
        }
    }

```

```

    }
}
return false;
}

void clear() {
    for (int i = 0; i < bucket_count_; ++i) {
        Node* p = buckets_[i];
        while (p != nullptr) {
            Node* next = p->next;
            delete p;
            p = next;
        }
        buckets_[i] = nullptr;
    }
    size_ = 0;
}

int size() const {
    return size_;
}

int bucket_count() const {
    return bucket_count_;
}

int max_chain() const {
    int max_len = 0;
    for (int i = 0; i < bucket_count_; ++i) {
        int len = 0;
        for (Node* p = buckets_[i]; p != nullptr; p = p->next) {
            ++len;
        }
        if (len > max_len) {
            max_len = len;
        }
    }
    return max_len;
}

int used_buckets() const {
    int count = 0;
    for (int i = 0; i < bucket_count_; ++i) {
        if (buckets_[i] != nullptr) {
            ++count;
        }
    }
    return count;
}

private:
static std::uint32_t fnv1a_(const std::string& key) {
    std::uint32_t h = 2166136261u;
    for (unsigned char c : key) {
        h ^= static_cast<std::uint32_t>(c);
        h *= 16777619u;
    }
    return h;
}

int hash_(const std::string& key) const {
    return static_cast<int>(
        fnv1a_(key) % static_cast<std::uint32_t>(bucket_count_));
}

static bool is_prime_(int n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (int i = 3; i * i <= n; i += 2) {
        if (n % i == 0) return false;
    }
    return true;
}
}

```

```

static int next_prime_(int current) {
    int candidate = current * 2 + 1;
    while (!is_prime_(candidate)) {
        candidate += 2;
    }
    return candidate;
}

void rehash(int new_bucket_count) {
    Node** new_buckets = new Node*[new_bucket_count];
    for (int i = 0; i < new_bucket_count; ++i) {
        new_buckets[i] = nullptr;
    }
    for (int i = 0; i < bucket_count_; ++i) {
        Node* p = buckets_[i];
        while (p != nullptr) {
            Node* next = p->next;
            const int h = static_cast<int>(<
                fnv1a_(p->key) % static_cast<std::uint32_t>(new_bucket_count));
            p->next = new_buckets[h];
            new_buckets[h] = p;
            p = next;
        }
    }
    delete[] buckets_;
    buckets_ = new_buckets;
    bucket_count_ = new_bucket_count;
}

double load_factor() const {
    return static_cast<double>(size_) / static_cast<double>(bucket_count_);
}

Node** buckets_;
int bucket_count_;
int size_;
};

// =====
// キー生成
// =====
static int additive_hash_mod17(const std::string& key) {
    int h = 0;
    for (unsigned char c : key) {
        h += static_cast<int>(c);
    }
    return h % 17;
}

static std::vector<std::string> generate_attack_keys(int N) {
    std::string base = "aaabbbcccddd";
    std::sort(base.begin(), base.end());

    std::vector<std::string> keys;
    keys.reserve(static_cast<std::size_t>(N));

    do {
        keys.push_back(base);
        if (static_cast<int>(keys.size()) >= N) {
            break;
        }
    } while (std::next_permutation(base.begin(), base.end()));

    if (static_cast<int>(keys.size()) < N) {
        std::cerr << "warning: could not generate enough attack keys ("
            << keys.size() << " / " << N << ")\\n";
    }
    return keys;
}

static std::vector<std::string> generate_random_keys(int N) {
    static const char chars[] =
        "ABCDEFGHGIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";

```

```

static const int L = 62;

std::mt19937 rng(42);
std::uniform_int_distribution<int> dist(0, L - 1);

std::unordered_set<std::string> seen;
seen.reserve(static_cast<std::size_t>(N) * 2);

std::vector<std::string> keys;
keys.reserve(static_cast<std::size_t>(N));

std::string key(12, ' ');
while (static_cast<int>(keys.size()) < N) {
    for (int i = 0; i < 12; ++i) {
        key[i] = chars[dist(rng)];
    }
    if (seen.insert(key).second) {
        keys.push_back(key);
    }
}
return keys;
}

// =====
// 計測
// =====
struct MeasureResult {
    double avg_comparisons;
    double avg_lookup_ns;
};

template <class Table>
static MeasureResult measure(Table& table, const std::vector<std::string>& keys) {
    const int n = static_cast<int>(keys.size());
    const int lookup_repeats = 5;

    // 挿入
    for (int i = 0; i < n; ++i) {
        table.insert(keys[i], i);
    }

    // 決定的な平均キー比較回数
    long long total_comparisons = 0;
    long long found_count = 0;
    for (int r = 0; r < lookup_repeats; ++r) {
        for (int i = 0; i < n; ++i) {
            long long comp = 0;
            const bool found = table.find(keys[i], comp);
            if (found) {
                ++found_count;
            }
            total_comparisons += comp;
        }
    }
    const double avg_comparisons =
        static_cast<double>(total_comparisons) /
        static_cast<double>(n * lookup_repeats);

    if (found_count != static_cast<long long>(n) * lookup_repeats) {
        std::cerr << "error: some keys were not found during measurement\n";
    }

    // 平均検索時間: 9 回反復の最小値
    // コンパイラの最適化で検索ループが消えないよう、volatile sink に結果を加算
    using clock = std::chrono::steady_clock;
    long long best_total_ns = -1;
    const int time_repeats = 9;
    volatile long long time_sink = 0;
    for (int iter = 0; iter < time_repeats; ++iter) {
        const auto t0 = clock::now();
        for (int r = 0; r < lookup_repeats; ++r) {
            for (int i = 0; i < n; ++i) {
                long long comp = 0;
                table.find(keys[i], comp);
            }
        }
        time_sink += clock::now() - t0;
    }
    best_total_ns = time_sink;
}

```

```

        time_sink += comp;
    }
}
const auto t1 = clock::now();
const long long ns =
    std::chrono::duration_cast<std::chrono::nanoseconds>(t1 - t0).count();
if (best_total_ns < 0 || ns < best_total_ns) {
    best_total_ns = ns;
}
}
if (time_sink < 0) {
    std::cerr << time_sink << "\n";
}
const double avg_lookup_ns =
    static_cast<double>(best_total_ns) /
    static_cast<double>(n * lookup_repeats);

return {avg_comparisons, avg_lookup_ns};
}

template <class Table>
static void report_stats(const std::string& name, const Table& table) {
    std::cerr << name << ": "
        << "size=" << table.size()
        << " buckets=" << table.used_buckets()
        << " max_chain=" << table.max_chain()
        << " load_factor=" << std::fixed << std::setprecision(3)
        << (static_cast<double>(table.size()) /
            static_cast<double>(table.used_buckets()))
        << "\n";
}

// =====
// メイン
// =====
int main() {
    const int N = 2000;

    const auto attack_keys = generate_attack_keys(N);
    const auto random_keys = generate_random_keys(N);

    // attack キーの加算ハッシュが全て同一か std::set で検証
    std::set<int> attack_hashes;
    for (const auto& k : attack_keys) {
        attack_hashes.insert(additive_hash_mod17(k));
    }
    std::cerr << "attack keys additive hash (mod 17):";
    for (int h : attack_hashes) {
        std::cerr << " " << h;
    }
    std::cerr << (attack_hashes.size() == 1 ? " (all same)\n" : " (DIFFERENT!)\n");

    std::cerr << "attack keys: " << attack_keys.size()
        << ", random keys: " << random_keys.size() << "\n";

    // (a) fixed17 / attack
    Fixed17Table fixed17_attack;
    const auto r1 = measure(fixed17_attack, attack_keys);
    report_stats("fixed17/attack", fixed17_attack);

    // (b) fixed17 / random
    Fixed17Table fixed17_random;
    const auto r2 = measure(fixed17_random, random_keys);
    report_stats("fixed17/random", fixed17_random);

    // (c) improved / attack
    ImprovedTable improved_attack;
    const auto r3 = measure(improved_attack, attack_keys);
    report_stats("improved/attack", improved_attack);

    // (d) improved / random
    ImprovedTable improved_random;
    const auto r4 = measure(improved_random, random_keys);
    report_stats("improved/random", improved_random);
}

```

```

std::cout << "table,keyset,n,avg_comparisons,avg_lookup_ns\n";
std::cout << std::fixed << std::setprecision(2);
std::cout << "fixed17,attack," << N << ", " << r1.avg_comparisons << ", "
    << r1.avg_lookup_ns << "\n";
std::cout << "fixed17,random," << N << ", " << r2.avg_comparisons << ", "
    << r2.avg_lookup_ns << "\n";
std::cout << "improved,attack," << N << ", " << r3.avg_comparisons << ", "
    << r3.avg_lookup_ns << "\n";
std::cout << "improved,random," << N << ", " << r4.avg_comparisons << ", "
    << r4.avg_lookup_ns << "\n";

return 0;
}

```

A.18 make_figures.py (図の生成スクリプト)

```

#!/usr/bin/env python3
"""実験 CSV からレポート用の白黒図 (PDF) を生成する."""
import csv
import math
from collections import defaultdict
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

BASE = Path(__file__).resolve().parent
DATA = BASE / "data"
FIGS = BASE / "figures"
FIGS.mkdir(exist_ok=True)

# 白黒印刷で判別できるスタイル (グレースケール+線種+マーカー)
plt.rcParams.update({
    "font.family": "Hiragino Sans",
    "font.size": 10,
    "axes.grid": True,
    "grid.color": "0.85",
    "grid.linewidth": 0.5,
    "figure.dpi": 150,
    "savefig.dpi": 200,
    "savefig.bbox": "tight",
})
STYLES = {
    "add": dict(color="0.0", linestyle="-", marker="o", markersize=4),
    "djb2": dict(color="0.35", linestyle="--", marker="s", markersize=4),
    "fnv1a": dict(color="0.55", linestyle="-.", marker="^", markersize=4),
    "stdhash": dict(color="0.7", linestyle=":", marker="d", markersize=4),
}
LABELS = {
    "add": "加算ハッシュ (hash.c)",
    "djb2": "djb2",
    "fnv1a": "FNV-1a",
    "stdhash": "std:hash",
}

def read_csv(path):
    with open(path) as f:
        return list(csv.DictReader(f))

def fig_collision(m, outname):
    """バケットごとの要素数分布 (ソート済み) を 4 関数で比較."""
    rows = read_csv(DATA / f"collision_m{m}.csv")
    counts = defaultdict(list)
    for r in rows:
        counts[r["hash"]].append(int(r["count"]))
    fig, ax = plt.subplots(figsize=(6.4, 4.0))
    n = sum(counts["add"])

```

```

expected = n / m
for name in ["add", "djb2", "fnv1a", "stdhash"]:
    ys = sorted(counts[name], reverse=True)
    ax.plot(range(len(ys)), ys, label=LABELS[name], linewidth=1.2,
            **{k: v for k, v in STYLES[name].items()},
            markevery=max(1, len(ys) // 25))
    ax.axhline(expected, color="0.2", linewidth=0.8, linestyle=(0, (1, 3)))
    ax.annotate(f"一様分布の期待値 {expected:.1f}", xy=(len(counts['add']) * 0.55, expected),
               xytext=(0, 5), textcoords="offset points", fontsize=9, color="0.2")
    ax.set_xlabel(f"バケット順位 (要素数の降順, 全 {m} バケット)")
    ax.set_ylabel(f"バケット内の要素数 [個]")
    ax.legend()
    fig.savefig(FIGS / outname)
    plt.close(fig)
    print(f"wrote {outname}")

def fig_loadfactor():
    """負荷率と平均比較回数 (実測 vs 理論)."""
    rows = read_csv(DATA / "loadfactor.csv")
    series = defaultdict(lambda: ([], [], [])) # hash -> (alpha, succ, unsucc)
    for r in rows:
        a, s, u = series[r["hash"]]
        a.append(float(r["alpha"]))
        s.append(float(r["succ_avg"]))
        u.append(float(r["unsucc_avg"]))
    fig, axes = plt.subplots(1, 2, figsize=(9.6, 4.0), sharex=True)
    alphas = series["add"][0]
    theory_x = [a for a in alphas]
    for ax, idx, title, theory in (
        (axes[0], 1, "成功検索", [1 + a / 2 for a in theory_x]),
        (axes[1], 2, "不成功検索", theory_x),
    ):
        for name in ["add", "fnv1a"]:
            a = series[name][0]
            y = series[name][idx]
            ax.plot(a, y, label=LABELS[name], linewidth=1.2, **STYLES[name])
        ax.plot(theory_x, theory, color="0.0", linestyle=(0, (1, 2)), linewidth=1.8,
               label="理論値 + (" $1+\\alpha/2$" if idx == 1 else " $\\alpha$"))
        ax.set_xlabel(f"負荷率 $\\alpha$")
        ax.set_title(title, fontsize=10)
        ax.set_xscale("log")
        ax.set_yscale("log")
        ax.legend(fontsize=9)
    axes[0].set_ylabel("平均キー比較回数 [回]")
    fig.savefig(FIGS / "loadfactor.png")
    plt.close(fig)
    print("wrote loadfactor.png")

def fig_timing():
    """要素数と挿入/検索時間 (中央値) の両対数プロット."""
    path = DATA / "timing.csv"
    if not path.exists():
        print("timing.csv not found; skip")
        return
    rows = read_csv(path)
    agg = defaultdict(lambda: defaultdict(list)) # structure -> N -> [(ins, lk)]
    for r in rows:
        agg[r["structure"]][int(r["N"])].append(
            (float(r["insert_total_us"]), float(r["lookup_avg_ns"])))
    st_styles = {
        "fixed17": dict(color="0.0", linestyle="-", marker="o", markersize=4),
        "improved": dict(color="0.35", linestyle="--", marker="s", markersize=4),
        "unordered_map": dict(color="0.55", linestyle="-.", marker="^", markersize=4),
        "map": dict(color="0.7", linestyle=":", marker="d", markersize=4),
    }
    st_labels = {
        "fixed17": "提出版 ($m=17$ 固定)",
        "improved": "改良版 (FNV-1a+リハッシュ)",
        "unordered_map": "std::unordered_map",
        "map": "std::map",
    }
    def median(xs):

```

```

xs = sorted(xs)
return xs[len(xs) // 2]

for idx, (ylabel, outname) in enumerate(
    [("挿入合計時間 [ $\mu$ s]", "timing_insert.png"),
     ("1 回あたり平均検索時間 [ns]", "timing_lookup.png")]):
    fig, ax = plt.subplots(figsize=(6.4, 4.2))
    for st in ["fixed17", "improved", "unordered_map", "map"]:
        if st not in agg:
            continue
        ns = sorted(agg[st].keys())
        ys = [median([v[idx] for v in agg[st][n]]) for n in ns]
        label = st_labels[st].replace("\\_", "_")
        ax.plot(ns, ys, label=label, linewidth=1.2, **st_styles[st])
    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.set_xlabel("要素数  $N$  [個]")
    ax.set_ylabel(ylabel)
    ax.legend(fontsize=9)
    fig.savefig(FIGS / outname)
    plt.close(fig)
    print(f"wrote {outname}")

def fig_rehash():
    """挿入レイテンシの時系列（リハッシュのスパイク）と累積平均."""
    path = DATA / "rehash.csv"
    if not path.exists():
        print("rehash.csv not found; skip")
        return
    rows = read_csv(path)
    samples = [(int(r["i"]), float(r["value_ns"])) for r in rows if r["kind"] == "sample"]
    spikes = [(int(r["i"]), float(r["value_ns"])) for r in rows if r["kind"] == "spike"]
    rehashes = [int(r["i"]) for r in rows if r["kind"] == "rehash"]
    cumavg = [(int(r["i"]), float(r["value_ns"])) for r in rows if r["kind"] == "cumavg"]

    fig, axes = plt.subplots(2, 1, figsize=(7.2, 6.0), sharex=True,
                             gridspec_kw={"height_ratios": [2, 1]})

    ax = axes[0]
    ax.plot([i for i, _ in samples], [v for _, v in samples],
            color="0.6", linewidth=0.5, label="挿入レイテンシ (100 件ごと)")
    ax.plot([i for i, _ in spikes], [v for _, v in spikes],
            linestyle="none", marker="x", color="0.0", markersize=5,
            label="スパイク (50  $\mu$ s 超)")
    for k, i in enumerate(rehashes):
        ax.axvline(i, color="0.3", linewidth=0.6, linestyle="--",
                  label="リハッシュ発生" if k == 0 else None)
    ax.set_yscale("log")
    ax.set_ylabel("挿入 1 件のレイテンシ [ns]")
    ax.legend(fontsize=9, loc="upper left")

    ax2 = axes[1]
    ax2.plot([i for i, _ in cumavg], [v for _, v in cumavg],
            color="0.0", linewidth=1.2, label="累積平均挿入時間")
    ax2.set_xlabel("挿入番号  $i$  [件目]")
    ax2.set_ylabel("累積平均 [ns]")
    ax2.set_ylim(bottom=0)
    ax2.legend(fontsize=9)
    fig.savefig(FIGS / "rehash.png")
    plt.close(fig)
    print("wrote rehash.png")

def fig_memory():
    """1 要素あたりメモリ使用量の比較（棒グラフ）."""
    path = DATA / "memory.csv"
    if not path.exists():
        print("memory.csv not found; skip")
        return
    rows = []
    with open(path) as f:
        for line in f:
            parts = line.strip().split(",")
            if len(parts) == 5:

```

```

        rows.append(parts)
labels_map = {"improved": "改良版", "unordered_map": "std::unordered_map",
              "map": "std::map"}
ns = [100000, 1000000]
structs = ["improved", "unordered_map", "map"]
fig, ax = plt.subplots(figsize=(6.4, 4.0))
width = 0.35
hatches = {100000: "", 1000000: "///"}
grays = {100000: "0.55", 1000000: "0.85"}
for j, n in enumerate(ns):
    vals = []
    for st in structs:
        delta = next(int(r[4]) for r in rows if r[0] == st and int(r[1]) == n)
        vals.append(delta / n)
    xs = [k + (j - 0.5) * width for k in range(len(structs))]
    ax.bar(xs, vals, width=width, color=grays[n], edgecolor="0.0",
           hatch=hatches[n], label=f"$N = 10^{{{len(str(n)) - 1}}}$")
    for x, v in zip(xs, vals):
        ax.annotate(f"{v:.1f}", xy=(x, v), xytext=(0, 2),
                    textcoords="offset points", ha="center", fontsize=9)
ax.set_xticks(range(len(structs)))
ax.set_xticklabels([labels_map[s] for s in structs])
ax.set_ylabel("1 要素あたりの増分メモリ [バイト/個]")
ax.legend()
fig.savefig(FIGS / "memory.png")
plt.close(fig)
print("wrote memory.png")

def fig_openaddr():
    """開放番地法（線形走査）とチェイン法のプローブ数比較."""
    path = DATA / "openaddr.csv"
    if not path.exists():
        print("openaddr.csv not found; skip")
        return
    rows = read_csv(path)
    series = defaultdict(lambda: defaultdict(list)) # method -> field -> list
    for r in rows:
        m = r["method"]
        series[m]["alpha"].append(float(r["alpha"]))
        series[m]["succ"].append(float(r["succ_probes"]))
        series[m]["unsucc"].append(float(r["unsucc_probes"]))
    m_styles = {
        "chained": dict(color="0.0", linestyle="-", marker="o", markersize=4),
        "openaddr": dict(color="0.45", linestyle="--", marker="s", markersize=4),
    }
    m_labels = {"chained": "チェイン法", "openaddr": "開放番地法（線形走査）"}
    alphas_t = [0.25 + 0.01 * i for i in range(71)]
    fig, axes = plt.subplots(1, 2, figsize=(9.6, 4.0), sharex=True)
    for ax, key, title, theories in (
        (axes[0], "succ", "成功検索",
         [("チェイン法理論  $1 + \frac{\alpha}{2}$ ", [1 + a / 2 for a in alphas_t], "0.0"),
          ("線形走査理論  $1 + \frac{1}{1 - \alpha}$ ",
           [(1 + 1 / (1 - a)) / 2 for a in alphas_t], "0.45")]),
        (axes[1], "unsucc", "不成功検索",
         [("チェイン法理論  $1 + \alpha$ ", [1 + a for a in alphas_t], "0.0"),
          ("線形走査理論  $1 + \frac{1}{(1 - \alpha)^2}$ ",
           [(1 + 1 / (1 - a) ** 2) / 2 for a in alphas_t], "0.45")]),
    ):
        for m in ["chained", "openaddr"]:
            ax.plot(series[m]["alpha"], series[m][key], linestyle="none",
                    marker=m_styles[m]["marker"], markersize=5,
                    color=m_styles[m]["color"], label=m_labels[m] + "（実測）")
        for lbl, ys, col in theories:
            ax.plot(alphas_t, ys, color=col, linewidth=1.0,
                    linestyle=(0, (1, 2)), label=lbl)
        ax.set_xlabel("負荷率  $\alpha$ ")
        ax.set_title(title, fontsize=10)
        ax.set_yscale("log")
        ax.legend(fontsize=8)
    axes[0].set_ylabel("平均プローブ数 [回]")
    fig.savefig(FIGS / "openaddr.png")
    plt.close(fig)
    print("wrote openaddr.png")

```

```

def fig_keylen():
    """キー長とハッシュ計算・検索時間の関係."""
    path = DATA / "keylen.csv"
    if not path.exists():
        print("keylen.csv not found; skip")
        return
    rows = read_csv(path)
    ls = [int(r["keylen"]) for r in rows]
    fig, ax = plt.subplots(figsize=(6.4, 4.0))
    ax.plot(ls, [float(r["fnv1a_hash_ns"]) for r in rows], label="FNV-1a ハッシュ計算",
            **STYLES["fnv1a"], linewidth=1.2)
    ax.plot(ls, [float(r["add_hash_ns"]) for r in rows], label="加算ハッシュ計算",
            **STYLES["add"], linewidth=1.2)
    ax.plot(ls, [float(r["lookup_ns"]) for r in rows], label="成功検索 (FNV-1a 表)",
            **STYLES["djb2"], linewidth=1.2)
    ax.set_xscale("log", base=2)
    ax.set_xlabel("キー長 $L$ [文字]")
    ax.set_ylabel("1 回あたり平均時間 [ns]")
    ax.legend(fontsize=9)
    fig.savefig(FIGS / "keylen.png")
    plt.close(fig)
    print("wrote keylen.png")

if __name__ == "__main__":
    fig_collision(17, "collision_m17.png")
    fig_collision(1031, "collision_m1031.png")
    fig_loadfactor()
    fig_timing()
    fig_rehash()
    fig_memory()
    fig_openaddr()
    fig_keylen()

```

B 付録: 出力結果

以下に、実際に実行した際の出力を無加工で添付する。

B.1 hashtable_v1 の出力

```

--- insert ---
takimoto=42, katsurada=122, matsuzawa=35, ohmura=12 を挿入しました
--- find (success) ---
takimoto: 42
katsurada: 122
matsuzawa: 35
ohmura: 12
--- find (failure) ---
nonexistent: none
--- erase ---
katsurada を削除しました
katsurada (after erase): none
takimoto (after erase): 42

```

B.2 hashtable の出力

```

=== (a) HashTable<int> ===
insert 後の size: 4
bucket[0]: (empty)
bucket[1]: (empty)
bucket[2]: (empty)
bucket[3]: (matsuzawa -> 35)
bucket[4]: (empty)
bucket[5]: (takimoto -> 42)

```

```

bucket[6]: (ohmura -> 12)
bucket[7]: (empty)
bucket[8]: (katsurada -> 122)
bucket[9]: (empty)
bucket[10]: (empty)
bucket[11]: (empty)
bucket[12]: (empty)
bucket[13]: (empty)
bucket[14]: (empty)
bucket[15]: (empty)
bucket[16]: (empty)
find takimoto: 42
find katsurada: 122
find nonexistent: none
katsurada 削除後の size: 3
find katsurada after erase: none

=== (b) HashTable<double> ===
insert 後の size: 3
find matsuzawa: 175.000000
find ohmura: none

=== (c) HashTable<std::string> ===
insert 後の size: 3
find greeting: こんにちは
find subject: 情報科学演習
find farewell: none

=== (d) 存在しないキーと削除後の検索 ===
find absent (before erase present): none
find present (after erase present): none
find absent (after erase present): none

```

B.3 hashtable_improved の出力

```

=== 大量挿入前 ===
bucket_count: 17, size: 0, load_factor: 0
=== key0000 ~ key0999 を 0 ~ 999 とともに挿入 ===
拡張: bucket_count 17 -> 37 (size=19, load_factor=0.513514)
拡張: bucket_count 37 -> 79 (size=39, load_factor=0.493671)
拡張: bucket_count 79 -> 163 (size=81, load_factor=0.496933)
拡張: bucket_count 163 -> 331 (size=165, load_factor=0.498489)
拡張: bucket_count 331 -> 673 (size=333, load_factor=0.494799)
拡張: bucket_count 673 -> 1361 (size=675, load_factor=0.495959)
=== 挿入後 ===
bucket_count: 1361, size: 1000, load_factor: 0.734754
=== 検索テスト ===
find key0000: 0
find key0123: 123
find key0999: 999
find key1000: none
=== 削除テスト ===
key0123 を削除しました
find key0123 after erase: none
size: 999, load_factor: 0.734019
=== 先頭 30 バケットの内容 (拡張後も整合性確認) ===
bucket[0]: (empty)
bucket[1]: (key0400 -> 400) -> (key0649 -> 649)
bucket[2]: (key0890 -> 890) -> (key0170 -> 170) -> (key0228 -> 228)
bucket[3]: (empty)
bucket[4]: (empty)
bucket[5]: (key0280 -> 280)
bucket[6]: (empty)
bucket[7]: (key0804 -> 804) -> (key0309 -> 309) -> (key0482 -> 482)
bucket[8]: (empty)
bucket[9]: (key0202 -> 202)
bucket[10]: (empty)
bucket[11]: (empty)
bucket[12]: (key0369 -> 369)
bucket[13]: (key0941 -> 941)
bucket[14]: (empty)
bucket[15]: (key0347 -> 347)

```

```

bucket[16]: (empty)
bucket[17]: (key0755 -> 755)
bucket[18]: (empty)
bucket[19]: (empty)
bucket[20]: (empty)
bucket[21]: (key0119 -> 119) -> (key0241 -> 241)
bucket[22]: (empty)
bucket[23]: (key0070 -> 70)
bucket[24]: (key0660 -> 660) -> (key0062 -> 62)
bucket[25]: (key0902 -> 902) -> (key0588 -> 588)
bucket[26]: (empty)
bucket[27]: (empty)
bucket[28]: (empty)
bucket[29]: (empty)
... (残り 1331 バケットは省略)

```

B.4 exp_anagram の出力

key	add_hash(m=17)	fnv1a_hash(m=17)

abc	5	1
acb	5	1
bac	5	2
bca	5	1
cab	5	5
cba	5	4

stop	12	13
tops	12	7
pots	12	0
spot	12	3
opts	12	8
post	12	9

listen	9	11
silent	9	10
enlist	9	5
tinsel	9	12

[まとめ] すべてのグループで add は全キーが同一バケットになり、fnv1a は異なるバケットに分散している。

B.5 exp_collision の要約出力

$m = 17$ の要約統計:

Hash collision distribution summary (m=17, n=5000)					
function	empty	longest	avg(non-empty)	variance	chi2

add	0	329	294.12	311.16	17.99
djb2	0	325	294.12	295.04	17.05
fnv1a	0	315	294.12	253.75	14.67
stdhash	0	321	294.12	210.10	12.14

$m = 1031$ の要約統計:

Hash collision distribution summary (m=1031, n=5000)					
function	empty	longest	avg(non-empty)	variance	chi2

add	57	20	5.13	14.54	3091.29
djb2	7	13	4.88	4.67	991.76
fnv1a	6	12	4.88	4.78	1016.09
stdhash	8	15	4.89	4.93	1048.26

なお、バケットごとの度数を列挙した CSV（標準出力）は $m = 1031$ で 4000 行を超えるため、本付録には $m = 17$ の分のみを添付する（ $m = 1031$ の分布は図 5 と上記要約統計に反映されている）。
 $m = 17$ の度数 CSV:

```
hash,bucket,count
add,0,285
add,1,299
add,2,305
add,3,270
add,4,313
add,5,297
add,6,305
add,7,288
add,8,301
add,9,278
add,10,306
add,11,329
add,12,305
add,13,271
add,14,308
add,15,258
add,16,282
djb2,0,316
djb2,1,325
djb2,2,319
djb2,3,288
djb2,4,264
djb2,5,300
djb2,6,277
djb2,7,297
djb2,8,284
djb2,9,299
djb2,10,292
djb2,11,283
djb2,12,268
djb2,13,307
djb2,14,310
djb2,15,295
djb2,16,276
fnv1a,0,258
fnv1a,1,304
fnv1a,2,288
fnv1a,3,302
fnv1a,4,277
fnv1a,5,289
fnv1a,6,315
fnv1a,7,296
fnv1a,8,268
fnv1a,9,307
fnv1a,10,313
fnv1a,11,277
fnv1a,12,306
fnv1a,13,299
fnv1a,14,309
fnv1a,15,287
fnv1a,16,305
stdhash,0,310
stdhash,1,302
stdhash,2,321
stdhash,3,291
stdhash,4,284
stdhash,5,302
stdhash,6,267
stdhash,7,279
stdhash,8,292
stdhash,9,295
stdhash,10,319
stdhash,11,275
stdhash,12,275
stdhash,13,299
stdhash,14,297
stdhash,15,297
stdhash,16,295
```

B.6 exp_loadfactor の出力 (CSV)

```
hash,n,alpha,succ_avg,unsucc_avg
add,8,0.470588,1.125000,0.500000
add,17,1.000000,1.411765,0.705882
add,34,2.000000,1.882353,2.117647
add,51,3.000000,2.392157,3.215686
add,85,5.000000,3.329412,5.011765
add,170,10.000000,5.882353,10.129412
add,340,20.000000,11.161765,19.894118
add,510,30.000000,16.213725,29.960784
add,850,50.000000,26.188235,50.389412
add,1700,100.000000,50.872941,100.271765
fnv1a,8,0.470588,1.250000,0.500000
fnv1a,17,1.000000,1.411765,0.823529
fnv1a,34,2.000000,2.029412,1.735294
fnv1a,51,3.000000,2.509804,2.823529
fnv1a,85,5.000000,3.529412,4.929412
fnv1a,170,10.000000,6.252941,9.770588
fnv1a,340,20.000000,11.082353,19.770588
fnv1a,510,30.000000,16.033333,29.752941
fnv1a,850,50.000000,25.818824,50.097647
fnv1a,1700,100.000000,50.828824,100.301765
```

B.7 exp_timing の出力 (CSV)

```
structure,N,rep,insert_total_us,lookup_avg_ns
fixed17,1000,1,40.0,191.5
fixed17,1000,2,398.0,1109.3
fixed17,1000,3,35.0,567.9
fixed17,1000,4,18.0,766.7
fixed17,1000,5,15.0,194.2
improved,1000,1,74.0,55.8
improved,1000,2,52.0,513.7
improved,1000,3,62.0,554.0
improved,1000,4,38.0,133.7
improved,1000,5,36.0,133.5
unordered_map,1000,1,54.0,12.2
unordered_map,1000,2,44.0,11.5
unordered_map,1000,3,35.0,130.4
unordered_map,1000,4,91.0,407.0
unordered_map,1000,5,96.0,26.6
map,1000,1,169.0,397.3
map,1000,2,120.0,288.8
map,1000,3,155.0,331.6
map,1000,4,121.0,402.9
map,1000,5,255.0,128.5
fixed17,3162,1,116.0,1153.5
fixed17,3162,2,93.0,1363.4
fixed17,3162,3,96.0,1076.1
fixed17,3162,4,48.0,626.9
fixed17,3162,5,59.0,1072.5
improved,3162,1,3941.0,30.9
improved,3162,2,233.0,72.8
improved,3162,3,147.0,41.7
improved,3162,4,126.0,35.5
improved,3162,5,222.0,217.1
unordered_map,3162,1,137.0,236.7
unordered_map,3162,2,240.0,85.3
unordered_map,3162,3,240.0,293.2
unordered_map,3162,4,140.0,27.3
unordered_map,3162,5,181.0,22.7
map,3162,1,460.0,843.1
map,3162,2,1217.0,229.7
map,3162,3,562.0,215.4
map,3162,4,571.0,170.7
map,3162,5,557.0,179.2
fixed17,10000,1,376.0,3488.8
fixed17,10000,2,306.0,3021.7
fixed17,10000,3,294.0,3440.7
fixed17,10000,4,176.0,3716.8
fixed17,10000,5,308.0,3337.8
```

improved,10000,1,603.0,37.7
 improved,10000,2,734.0,36.8
 improved,10000,3,592.0,88.3
 improved,10000,4,352.0,26.9
 improved,10000,5,321.0,37.6
 unordered_map,10000,1,526.0,27.9
 unordered_map,10000,2,863.0,39.3
 unordered_map,10000,3,783.0,54.9
 unordered_map,10000,4,822.0,28.3
 unordered_map,10000,5,612.0,21.2
 map,10000,1,2435.0,240.6
 map,10000,2,5650.0,300.8
 map,10000,3,3356.0,215.1
 map,10000,4,11840.0,238.5
 map,10000,5,3825.0,275.6
 fixed17,31623,1,1319.0,15475.0
 fixed17,31623,2,2248.0,18979.1
 fixed17,31623,3,3607.0,16495.9
 fixed17,31623,4,2168.0,14777.9
 fixed17,31623,5,943.0,19999.4
 improved,31623,1,1202.0,85.2
 improved,31623,2,3676.0,47.7
 improved,31623,3,2404.0,52.0
 improved,31623,4,5252.0,360.7
 improved,31623,5,5107.0,277.7
 unordered_map,31623,1,3917.0,36.9
 unordered_map,31623,2,10875.0,69.4
 unordered_map,31623,3,16621.0,85.6
 unordered_map,31623,4,3630.0,25.2
 unordered_map,31623,5,5084.0,95.2
 map,31623,1,13808.0,428.8
 map,31623,2,28546.0,487.6
 map,31623,3,9827.0,541.3
 map,31623,4,16844.0,359.8
 map,31623,5,8048.0,332.2
 fixed17,100000,1,3294.0,227284.1
 fixed17,100000,2,1672.0,484731.5
 fixed17,100000,3,3487.0,303822.0
 fixed17,100000,4,8990.0,374498.1
 fixed17,100000,5,6726.0,743101.0
 improved,100000,1,13283.0,27.8
 improved,100000,2,31653.0,178.6
 improved,100000,3,10417.0,23.0
 improved,100000,4,4453.0,39.4
 improved,100000,5,24479.0,25.9
 unordered_map,100000,1,11777.0,123.8
 unordered_map,100000,2,20205.0,31.6
 unordered_map,100000,3,28873.0,26.6
 unordered_map,100000,4,5407.0,24.3
 unordered_map,100000,5,21847.0,225.1
 map,100000,1,128625.0,1969.2
 map,100000,2,162097.0,686.6
 map,100000,3,57548.0,776.7
 map,100000,4,95797.0,872.1
 map,100000,5,71972.0,2339.4
 improved,316228,1,41385.0,76.6
 improved,316228,2,67185.0,72.4
 improved,316228,3,154527.0,447.1
 improved,316228,4,194349.0,235.8
 improved,316228,5,54041.0,85.5
 unordered_map,316228,1,59407.0,173.3
 unordered_map,316228,2,72512.0,256.8
 unordered_map,316228,3,122751.0,147.1
 unordered_map,316228,4,117644.0,249.5
 unordered_map,316228,5,169379.0,273.8
 map,316228,1,483318.0,4004.3
 map,316228,2,535042.0,4478.3
 map,316228,3,1686559.0,2921.7
 map,316228,4,1139795.0,4930.5
 map,316228,5,1054905.0,7659.1
 improved,1000000,1,1751084.0,273.8
 improved,1000000,2,1869361.0,1642.7
 improved,1000000,3,1382041.0,335.9
 improved,1000000,4,1365438.0,1087.5

```

improved,1000000,5,1496228.0,952.6
unordered_map,1000000,1,2060447.0,1321.6
unordered_map,1000000,2,3803852.0,1570.4
unordered_map,1000000,3,4391572.0,2244.6
unordered_map,1000000,4,3584558.0,956.4
unordered_map,1000000,5,2562546.0,372.5
map,1000000,1,13337805.0,12907.7
map,1000000,2,6074825.0,11252.8
map,1000000,3,4635425.0,24731.0
map,1000000,4,17204663.0,15431.1
map,1000000,5,3776930.0,10615.6

```

B.8 exp_memory の出力 (CSV)

```

improved,100000,3833856,11649024,7815168
improved,1000000,25427968,96239616,70811648
unordered_map,100000,3833856,9519104,5685248
unordered_map,1000000,25427968,81739776,56311808
map,100000,3817472,10256384,6438912
map,1000000,25427968,89735168,64307200

```

B.9 exp_tablesize の出力 (CSV)

```

hash,m,n,empty,longest,mean_nonempty,variance,chi2
add,16,5000,0,353,312.50,284.62,14.57
fnv1a,16,5000,0,339,312.50,244.00,12.49
add,17,5000,0,329,294.12,311.16,17.99
fnv1a,17,5000,0,315,294.12,253.75,14.67
add,256,5000,0,32,19.53,20.44,267.87
fnv1a,256,5000,0,31,19.53,19.89,260.70
add,257,5000,0,33,19.46,20.84,275.28
fnv1a,257,5000,0,31,19.46,17.91,236.63
add,1024,5000,54,21,5.15,14.77,3097.79
fnv1a,1024,5000,11,13,4.94,4.75,996.54
add,1031,5000,57,20,5.13,14.54,3091.29
fnv1a,1031,5000,6,12,4.88,4.78,1016.09

```

B.10 exp_rehash の統計出力

挿入ごとのレイテンシ CSV（標準出力）は 2000 行を超えるため添付を省略する（全データは図 11 に反映されている）。以下は標準エラーに出力された統計の全文である。

```
Generating 200000 unique keys...
```

```

progress: 0/200000
progress: 10000/200000
progress: 20000/200000
progress: 30000/200000
progress: 40000/200000
progress: 50000/200000
progress: 60000/200000
progress: 70000/200000
progress: 80000/200000
progress: 90000/200000
progress: 100000/200000
progress: 110000/200000
progress: 120000/200000
progress: 130000/200000
progress: 140000/200000
progress: 150000/200000
progress: 160000/200000
progress: 170000/200000
progress: 180000/200000
progress: 190000/200000

```

```
-----
Final stats (N=200000)
```

```
total insert time: 106292154 ns
average latency:   531.461 ns
max latency:      23832416 ns
rehash count:     14
final bucket_count: 350899
final load_factor: 0.569965
```

B.11 test_hashtable の出力 (全文)

```
OK: empty lookup returns nullptr
OK: empty size is 0
OK: erase on empty table is safe
OK: single insert/lookup
OK: all anagram keys found
OK: anagram values are correct
OK: erase head removes it
OK: head erase leaves middle
OK: head erase leaves tail
OK: erase middle removes it
OK: middle erase leaves head
OK: middle erase leaves tail
OK: erase tail removes it
OK: tail erase leaves head
OK: tail erase leaves middle
OK: erase missing key keeps size
OK: table intact after missing erase
OK: table intact after missing erase 2
OK: after erase key absent
OK: re-inserted key found
OK: duplicate insert returns newer value
OK: one erase reveals old value
OK: initial size 0
OK: size after 2 inserts
OK: size after 1 erase
OK: size after all erased
OK: size unchanged by missing erase
OK: 1000 inserted keys all found
OK: remaining 500 keys still found
OK: erased 500 keys return nullptr
OK: size after 500 erases is 500
OK: string empty lookup returns nullptr
OK: string empty size is 0
OK: string erase on empty table is safe
OK: string single insert/lookup
OK: string anagram keys found
OK: string anagram values correct
OK: string erase head removes it
OK: string head erase leaves middle
OK: string head erase leaves tail-ish
OK: string erase tail removes it
OK: string tail erase leaves head
OK: string tail erase leaves middle
OK: string erase middle removes it
OK: string middle erase leaves one side
OK: string middle erase leaves other side
OK: string table intact after missing erase
OK: string table intact after missing erase 2
OK: string after erase key absent
OK: string re-inserted key found
passed 50 / 50
```

B.12 exp_openaddr の出力 (CSV)

```
method,alpha,n,succ_probes,unsucc_probes,succ_ns,unsucc_ns
chained,0.25,32768,1.12585,1.25,87.6109,42.5135
openaddr,0.25,32768,1.1666,1.38959,775.262,167.805
chained,0.5,65536,1.24974,1.50014,814.335,2720.91
openaddr,0.5,65536,1.50391,2.51172,3652.69,1097.03
chained,0.7,91750,1.34897,1.70043,67.9187,1278.56
```

```

openaddr,0.7,91750,2.17365,6.11531,2251.94,1942.38
chained,0.8,104858,1.39974,1.79996,1158.11,974.102
openaddr,0.8,104858,2.95143,12.9226,1024,311.865
chained,0.9,117965,1.45118,1.90035,120.575,453.572
openaddr,0.9,117965,5.53371,49.3506,1214.9,640.8
chained,0.95,124518,1.47101,1.95075,86.5936,2322.49
openaddr,0.95,124518,10.6894,176.76,3175,16621

```

B.13 exp_keylen の出力 (CSV)

```

keylen,fnv1a_hash_ns,add_hash_ns,lookup_ns
8,5.0084,2.2292,22.1
16,11.15,2.7916,32.6417
32,26.6792,3.7,62.9209
64,70.5708,2.7917,118.737
128,194.058,3.6209,278.217
256,529.867,6.9292,567.862

```

B.14 hashtable_tagged の出力

```

=== 大量挿入前 ===
bucket_count: 17, size: 0, load_factor: 0
=== key0000 ~ key0999 を 0 ~ 999 とともに挿入 ===
  拡張: bucket_count 17 -> 37 (size=19, load_factor=0.513514)
  拡張: bucket_count 37 -> 79 (size=39, load_factor=0.493671)
  拡張: bucket_count 79 -> 163 (size=81, load_factor=0.496933)
  拡張: bucket_count 163 -> 331 (size=165, load_factor=0.498489)
  拡張: bucket_count 331 -> 673 (size=333, load_factor=0.494799)
  拡張: bucket_count 673 -> 1361 (size=675, load_factor=0.495959)
=== 挿入後 ===
bucket_count: 1361, size: 1000, load_factor: 0.734754
=== 検索テスト ===
find key0000: 0
find key0123: 123
find key0999: 999
find key1000: none
=== 全キー検索でのタグ照合効果 ===
1000 件全キー検索: tag comparisons = 1333, string comparisons = 1000
-> タグ不一致により文字列比較を 333 回省略しました
=== 削除テスト ===
key0123 を削除しました
find key0123 after erase: none
size: 999, load_factor: 0.734019
=== 先頭 30 バケットの内容 (拡張後も整合性確認) ===
bucket[0]: (empty)
bucket[1]: (key0400 -> 400) -> (key0649 -> 649)
bucket[2]: (key0890 -> 890) -> (key0170 -> 170) -> (key0228 -> 228)
bucket[3]: (empty)
bucket[4]: (empty)
bucket[5]: (key0280 -> 280)
bucket[6]: (empty)
bucket[7]: (key0804 -> 804) -> (key0309 -> 309) -> (key0482 -> 482)
bucket[8]: (empty)
bucket[9]: (key0202 -> 202)
bucket[10]: (empty)
bucket[11]: (empty)
bucket[12]: (key0369 -> 369)
bucket[13]: (key0941 -> 941)
bucket[14]: (empty)
bucket[15]: (key0347 -> 347)
bucket[16]: (empty)
bucket[17]: (key0755 -> 755)
bucket[18]: (empty)
bucket[19]: (empty)
bucket[20]: (empty)
bucket[21]: (key0119 -> 119) -> (key0241 -> 241)
bucket[22]: (empty)
bucket[23]: (key0070 -> 70)
bucket[24]: (key0660 -> 660) -> (key0062 -> 62)
bucket[25]: (key0902 -> 902) -> (key0588 -> 588)

```

```

bucket[26]: (empty)
bucket[27]: (empty)
bucket[28]: (empty)
bucket[29]: (empty)
... (残り 1331 バケットは省略)

```

B.15 exp_tag の出力 (CSV)

CSV の末尾 2 列が検索 1 回あたりの文字列比較回数であり (-1 は計測対象外の `std::unordered_map`), 時間の列は実行環境のばらつきを含む参考値である.

```

structure,keylen,insert_total_us,succ_lookup_ns,unsucc_lookup_ns,succ_strcmp,unsucc_strcmp
baseline,12,50891,292,113,1.28728,0.56954
baseline,64,301337,1404,372,1.28554,0.56925
baseline,256,967241,2415,1340,1.28809,0.56863
tagged,12,42960,2115,101,1.00000,0.00006
tagged,64,57715,2126,567,1.00001,0.00007
tagged,256,381603,4116,4068,1.00002,0.00004
unordered_map,12,74850,967,95,-1.00000,-1.00000
unordered_map,64,151800,1013,197,-1.00000,-1.00000
unordered_map,256,79376,2588,680,-1.00000,-1.00000

```

B.16 exp_hashdos の出力 (CSV)

```

table,keyset,n,avg_comparisons,avg_lookup_ns
fixed17,attack,2000,1000.50,9656.94
fixed17,random,2000,59.89,250.43
improved,attack,2000,1.36,31.26
improved,random,2000,1.38,30.75

```

C 付録: 参考にした hash.c

本課題の出発点として与えられた C 言語のチェーン法実装を参考として添付する (本文 2 節・3 節・5.8 節で参照).

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define HASHSIZE 17

struct member_list *hashtable[HASHSIZE];

struct member_list{
    char name[16];
    int score;
    struct member_list *next;
};

int hash(char *key){
    int hashval = 0;
    while(*key!=0){
        hashval += *key;
        key++;
    }

    return hashval % HASHSIZE;
}

void add(char *c, int score){
    int hashval;
    struct member_list *p, *ptr;

```

```

    hashval = hash(c);

    ptr = malloc(sizeof(struct member_list));
    strcpy(ptr->name,c);
    ptr->score = score;
    ptr->next=hashtable[hashval];
    hashtable[hashval]=ptr;
}

struct member_list *find(char *c){
    struct member_list *p;
    int hashval;

    hashval = hash(c);
    p = hashtable[hashval];

    while(p!=NULL){
        if(strcmp(c,p->name)==0){
            return p;
        }
        p = p->next;
    }
    return NULL;
}

int main(){
    char word[1024];
    int i;
    struct member_list *ptr;

    for(i=0;i<17;i++){
        hashtable[i] = NULL;
    }
    add("takimoto",42);
    add("katsurada",122);
    add("matsuzawa",35);
    add("ohmura",12);

    while(scanf("%s",word)!=EOF){
        ptr = find(word);
        if(ptr!=NULL){
            printf("%s, %i\n",ptr->name,ptr->score);
        }else{
            printf("none\n");
        }
    }
}

```